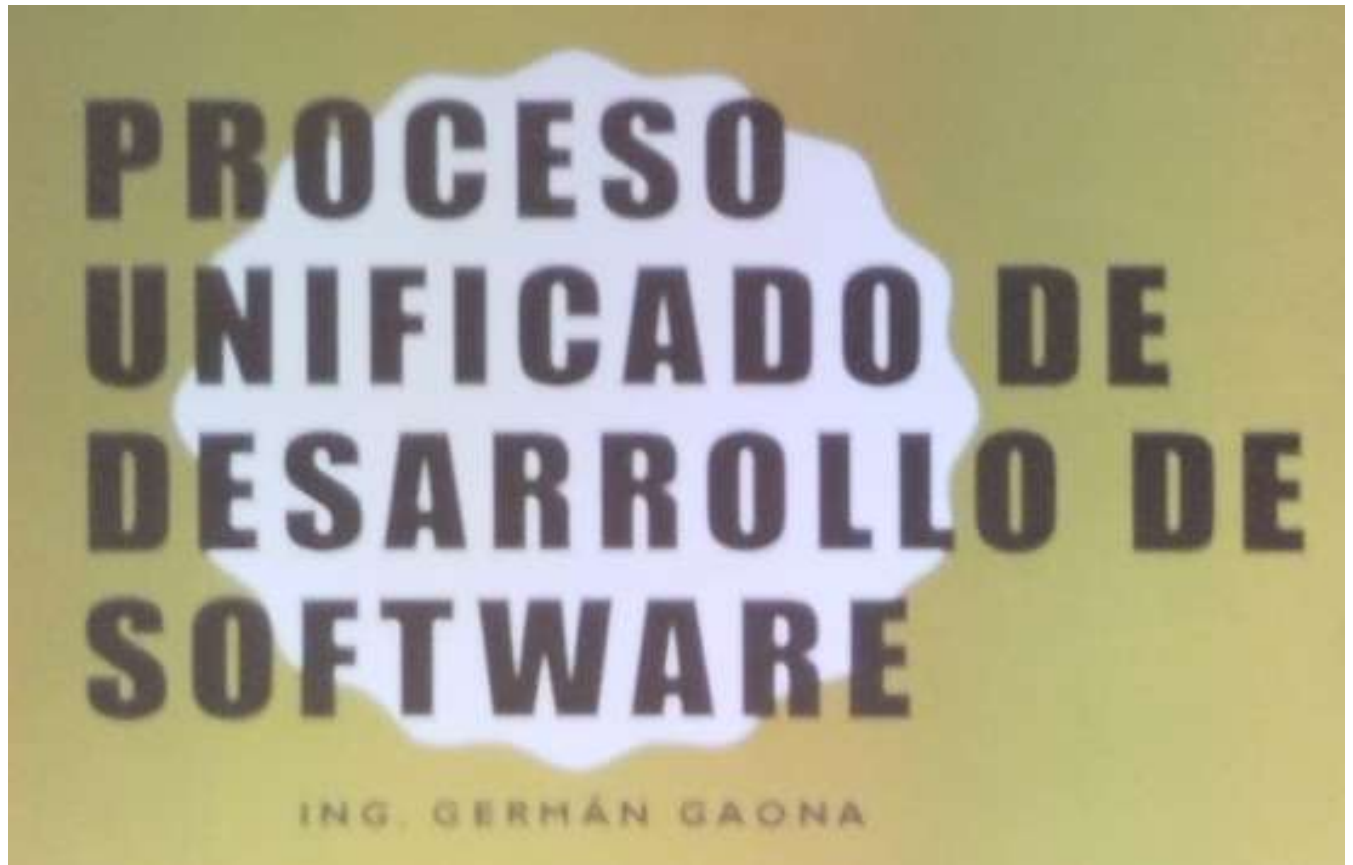


CLASE - 24/09

PROCESO UNIFICADO DE DESARROLLO DE SOFTWARE



- guiado por CU
- iterativo e incremental
- centrado en arquitectura
- es un framework de un proceso
- muchas veces está asociado con una metodología prescriptiva. Me dice **exactamente** qué es lo que tengo que hacer
- esta cargado de muchos modelos, al contrario de Scrum o XP
- el PU en su forma natural es prescriptivo, no hace falta tomar todo lo que dice el proceso
- el PU fue pensado para proyectos que se tienen que ejecutar con cierto rigor, no nos podemos permitir muchos errores
- el otro argumento que puede existir en contra del PU es que hoy en día casi no se lo usa
- ofrece una perspectiva de cómo pueden hacerse las cosas

BIBLIOGRAFÍA



- el manual de referencia
- guía de usuario
- no hubo versión 2.0 de PU, solamente de UML
- este último libro da una perspectiva más práctica así que el profe lo recomienda para eso, vamos a tener un poco más de como se hacen más las Realizaciones de CU, partes importantes de un Diagrama de Secuencia. No hay un capítulo exacto porque hay en distintas partes del libro, pero sirve para pegarle un vistazo
- PU y UML son los mismos autores, aunque es confuso que aparezca UML. El PU usa UML para hacer sus diagramas

UN PROCESO DE DESARROLLO DE SW

UN PROCESO DE DESARROLLO DE SW

- Especifica:
 - **quién** hace (trabajador),
 - **qué** hace (actividad),
 - **qué produce** (artefacto),
 - **cuándo** lo hace y **cómo** lo hace,
 - con el **objetivo** de crear o mejorar software.
- Proporciona **directrices y buenas prácticas**
 - para el **desarrollo eficiente** de software de calidad,
 - **reduciendo riesgos** y aumentando la **previsibilidad**.
- Sirve como **guía** para todos los participantes
 - clientes, usuarios, desarrolladores y gestores.

An illustration showing a person standing on a small stage or platform, gesturing with their arms raised, addressing a group of people seated at desks in a meeting room. The scene is lit with warm, golden light, suggesting a presentation or a motivational speech.

-que debería tener un proceso de desarrollo

-necesitamos alguien que nos guíe

-se necesita un proceso que nos ayude a organizar las tareas que se van hacer

-quien hace las cosas

-que hacen(actividades que se van hacer)

-que se produce(cuales son las cosas que vamos a producir en cada actividad que vamos a estar trabajando)

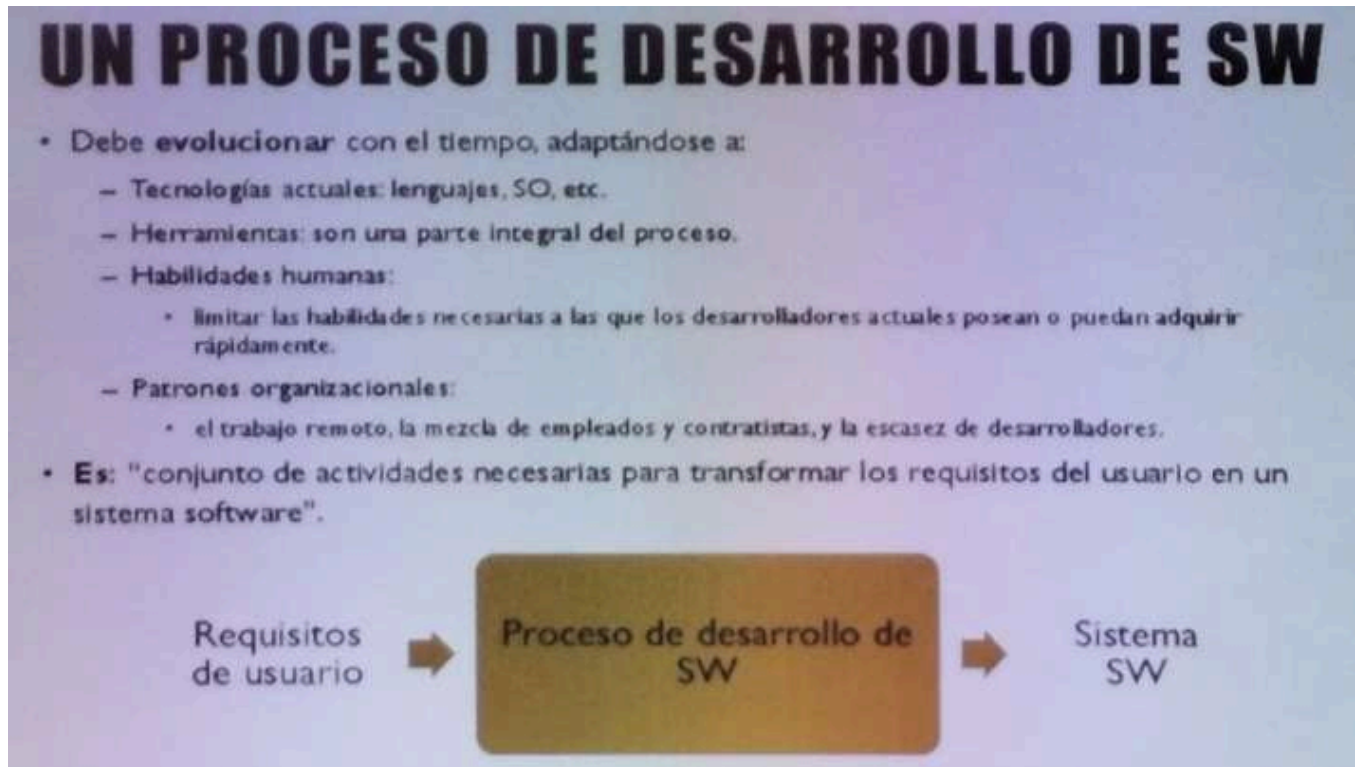
-el cuando tiene que ver básicamente con una cuestión de secuencialidad,

-el cómo sería demasiado prescriptivo ya (y no todos los procesos lo incluyen)

-los autores recomiendan hacer cierto tipo de actividad que van a terminar en cierto tipos de artefactos

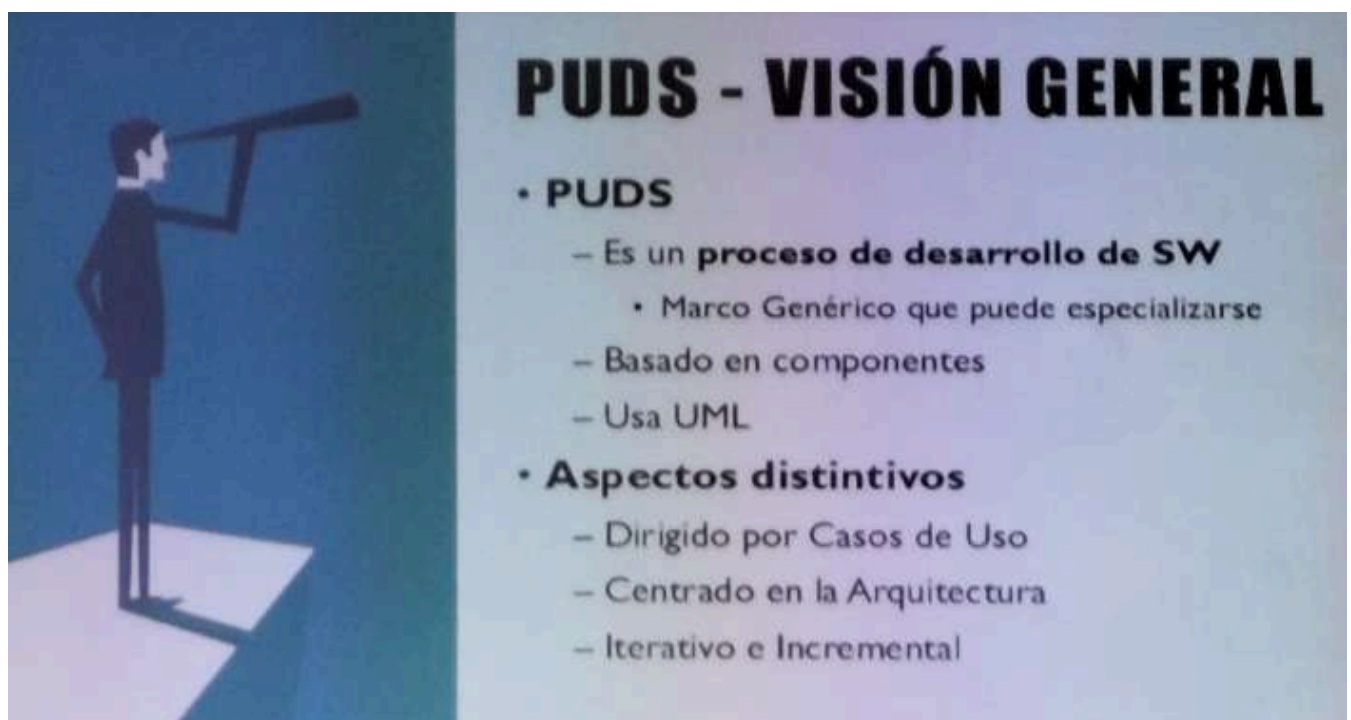
-estamos estableciendo una guía de cómo el equipo se va a comportar

(otro power)



- el proceso debe ser capaz de evolucionar, debe ser capaz de adaptarse a las diferentes tecnologías que van surgiendo en el mercado, no debe estar atado a una tecnología en particular
- convertir los requisitos en...

PUDS - VISIÓN GENERAL



-es un marco de trabajo general

-estos son 3 autores que se unieron para armar UML y PU. El PU viene más de la mano de Jacobson.

1) sirve para especializarse en diferentes proyectos de sw, no uno en particular. Está pensado para grandes proyectos

2) esto quiere decir que tiene que ver con el sw, desde un punto de vista de componente y sus interfaces. El concepto de componente puede llegar hacer algo contradictorio

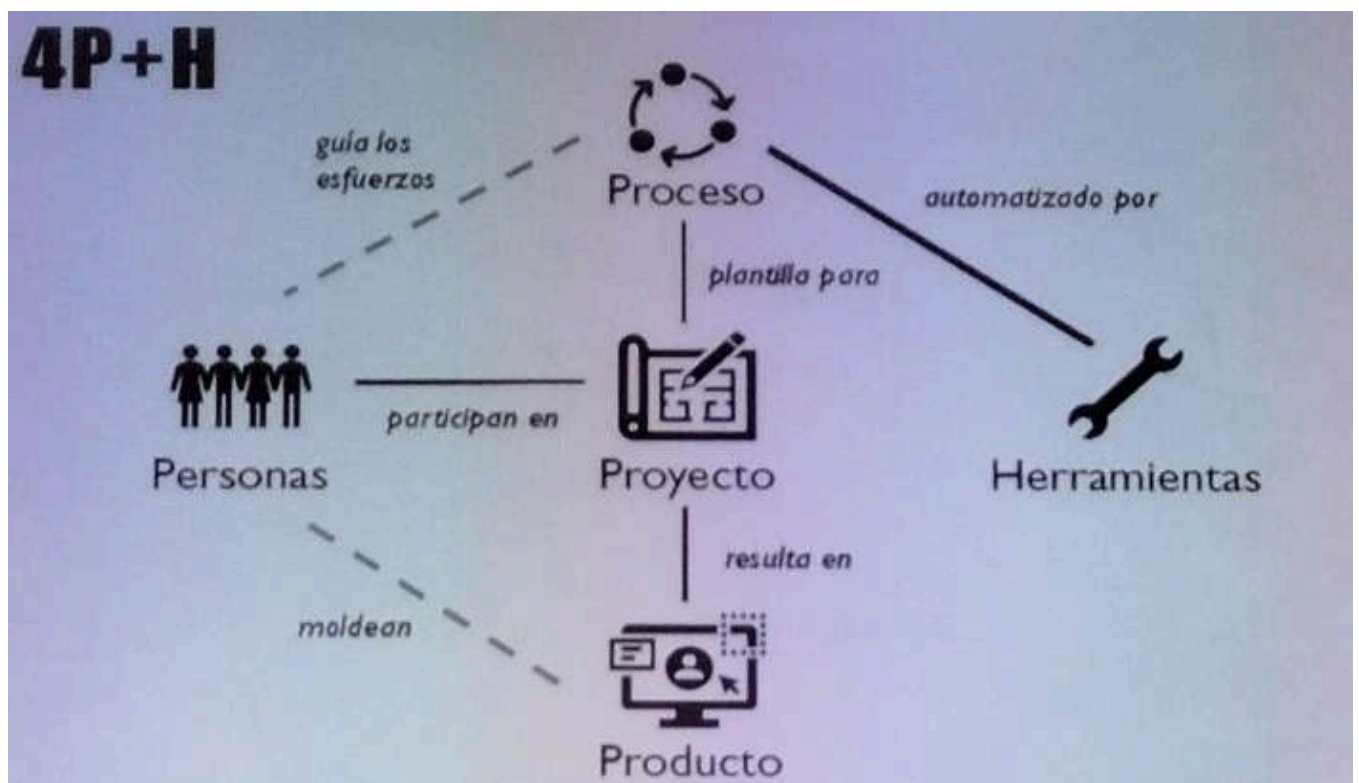
3) la versión que está en el libro de UML es 1.4

(muchas veces cuando ves un Componente escrito en algún lugar se refiere a un Artefacto que estoy implementando de algún Componente)

-Aspectos distintivos

-estas 3 características son innegociable

4P + H



(todos estos temas en particular se hablan más en el Capítulo 2. No hace falta entrar tanto en detalle sino saber los conceptos fundamentales)

-estos conceptos interesan para entender las generalidades

-el proceso sirve como **plantilla** al proyecto que vamos a ir elaborando

-voy a tener el PU que voy a tener que especializar para poder ejecutar

-en ese proyecto van a participar muchas personas que van a tener diferentes roles. Nos interesan los diferentes roles que tienen, no las personas en sí.

-el término que utiliza de manera abstracta para referirse a los roles es **Trabajador(worker)**

-ese producto normalmente va a ser ejecutable, pero a veces se requiere que haya algo más que serían los modelos

-se necesitan herramientas porque es inviable hacer todo a mano, se hace tedioso. El proceso debería poder ser automatizado con alguna herramienta

-las personas son los que terminan de moldear el Proyecto

- **Personas.** Los principales autores de un proyecto software son los arquitectos, desarrolladores, ingenieros de prueba, y el personal de gestión que les da soporte, además de los usuarios, clientes, y otros interesados. Las personas son realmente seres humanos, a diferencia del término abstracto *trabajadores*, que introduciremos más adelante.
- **Proyecto.** Elemento organizativo a través del cual se gestiona el desarrollo de software. El resultado de un proyecto es una versión de un producto.
- **Producto.** Artefactos que se crean durante la vida del proyecto, como los **modelos** (Apéndice A), código fuente, ejecutables, y documentación.
- **Proceso.** Un proceso de ingeniería de software es una definición del conjunto completo de actividades necesarias para transformar los requisitos de usuario en un producto. Un proceso es una plantilla para crear proyectos.
- **Herramientas.** Software que se utiliza para automatizar las actividades definidas en el proceso.

-El proceso que guía este desarrollo debe orientarse a las personas, es decir, debe funcionar bien para las personas que lo utilizan.

2.3. El producto es más que código

En el contexto del Proceso Unificado, el producto que se obtiene es un sistema software. El término *producto* aquí hace referencia al sistema entero, y no sólo al código que se entrega.

2.4.1. El proceso: una plantilla

En el Proceso Unificado, *proceso* hace referencia a un contexto que sirve como plantilla que puede reutilizarse para crear instancias de ella. Es comparable a una clase, que puede utilizarse para crear objetos en el paradigma de la orientación a objetos. *Instancia del proceso* es un sinónimo de *proyecto*.

En este libro, un *proceso de desarrollo de software* es una definición del conjunto completo de actividades necesarias para convertir los requisitos de usuario en un conjunto consistente de artefactos que conforman un producto software, y para convertir los cambios sobre esos requisitos en un nuevo conjunto consistente de artefactos.

Un proceso es una definición de un conjunto de actividades, no su ejecución.

2.5.2. El proceso dirige las herramientas

El proceso, tanto si se define explícita como implícitamente, especifica la funcionalidad de las herramientas, es decir, los casos de uso de las herramientas. El hecho de que existe un proceso es, por supuesto, el único motivo para necesitar herramientas. Las herramientas están ahí para automatizar tanto como sea posible del proceso.

de nuevo para cada alternativa probada. En resumen, es esencial que las herramientas sean capaces tanto de soportar la automatización de actividades repetitivas como de gestionar la información representada por la serie de modelos y artefactos, fomentando y soportando las actividades creativas que son el núcleo fundamental del desarrollo.

2.5.3. El equilibrio entre el proceso y las herramientas

Por tanto, desarrollar un proceso sin pensar cómo se automatizará es un mero ejercicio académico. Desarrollar herramientas sin saber qué proceso (marco de trabajo) van a soportar puede ser una experimentación infructuosa. Debe haber un equilibrio entre proceso y herramientas.

Por un lado, el proceso dirige el desarrollo de las herramientas. Por otro lado, las herramientas dirigen el desarrollo del proceso. El desarrollo del proceso y su soporte por herramientas debe tener lugar de manera simultánea. En cada versión del proceso debe haber también una versión de las herramientas. En cada versión debe darse este equilibrio. Alcanzar casi el ideal de este equilibrio nos llevará varias iteraciones, y las sucesivas iteraciones deben guiarse por la retroalimentación de los usuarios entre versiones.

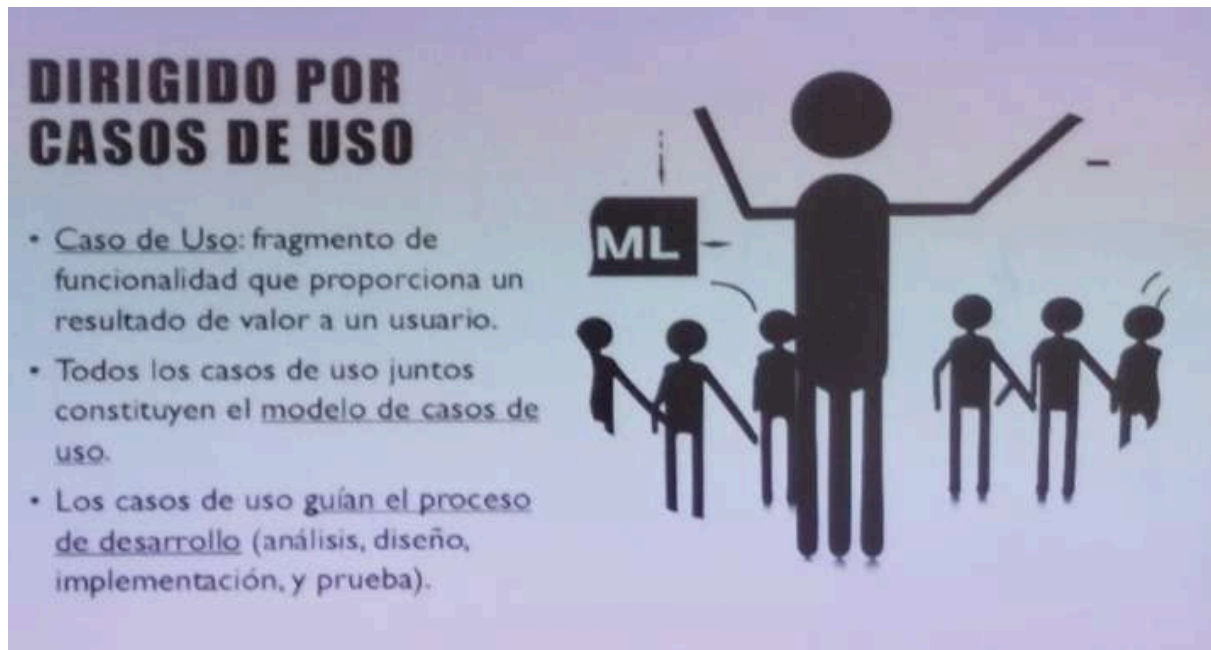
2.5.5. Las herramientas dan soporte al ciclo de vida completo

Hay herramientas que soportan todos los aspectos del **ciclo de vida del software** (Apéndice C):

- *Gestión de requisitos*: Se utiliza para almacenar, examinar, revisar, hacer el seguimiento y navegar por los diferentes requisitos de un proyecto software. Un requisito debería tener un estado asociado, y la herramienta debería permitir hacer el seguimiento de un requisito desde otros artefactos del ciclo de vida, como un caso de uso o un caso de prueba (véase el Capítulo 11).
- *Modelado visual*: Se utiliza para automatizar el uso de UML, es decir, para modelar y ensamblar una aplicación visualmente. Con esta herramienta conseguimos la integración con entornos de programación y aseguramos que el modelo y la implementación siempre son consistentes.
- *Herramientas de programación*: Proporciona una gama de herramientas, incluyendo editores, compiladores, depuradores, detectores de errores y analizadores de rendimiento.
- *Aseguramiento de la calidad*: Se utiliza para probar aplicaciones y componentes, es decir, para registrar y ejecutar casos de prueba que dirigen la prueba de un IGU y de las interfaces de un componente. En un ciclo de vida iterativo, la prueba de regresión es aún más esencial que en el desarrollo convencional. La automatización de los casos de prueba es esencial para conseguir una productividad alta. Además, muchas aplicaciones también ne-

Además de estas herramientas orientadas a la funcionalidad, hay otras herramientas que abarcan todo el ciclo de vida. Estas herramientas incluyen las de control de versiones, gestión de la configuración, seguimiento de defectos, documentación, gestión de proyecto y automatización de procesos.

DIRIGIDO POR CASOS DE USO



(esta imagen hizo con IA, no se guien)

-hablar de PU sin CU es imposible, hay cosas que se pueden obviar pero estas 3 características son innegociable

-un CU un resultado de valor para un Actor

-los Actores pueden ser personas o sistemas

-a partir de los CU voy a desarrollar el resto de disciplinas

-van a tomar un CU y van a tener que ver las partes estáticas y dinámicas

-la Realización de CU es una parte importante pero pequeña dentro de todo el CU

-me permite hacer los requerimientos, diseño, implementación y luego las pruebas

Aunque es cierto que los casos de uso guían el proceso, no se desarrollan aisladamente. Se desarrollan a la vez que la arquitectura del sistema. Es decir, los casos de uso guían la arquitectura del sistema y la arquitectura del sistema influye en la selección de los casos de uso. Por tanto, tanto la arquitectura del sistema como los casos de uso maduran según avanza el ciclo de desarrollo.

CENTRADO EN LA ARQUITECTURA

CENTRADO EN LA ARQUITECTURA

- **Arquitectura**
 - conjunto de decisiones significativas acerca de la organización de un sistema software, la selección de los elementos estructurales a partir de los cuales se compone el sistema, las interfaces entre ellos, su comportamiento, sus colaboraciones, y su composición.
- Se describe mediante diferentes **Vistas**.
- Incluye los **aspectos estáticos y dinámicos** más significativos.



-(este concepto está relacionado con el que ya vimos pero acá habla de decisiones significativas, cuales son los requerimientos principales de mi sw)

-aca habla de estructura, habla de componentes, habla de interfaces, no menciona explícitamente el tema de módulos pero vamos a ver mas adelante, y también menciona el tema de vistas

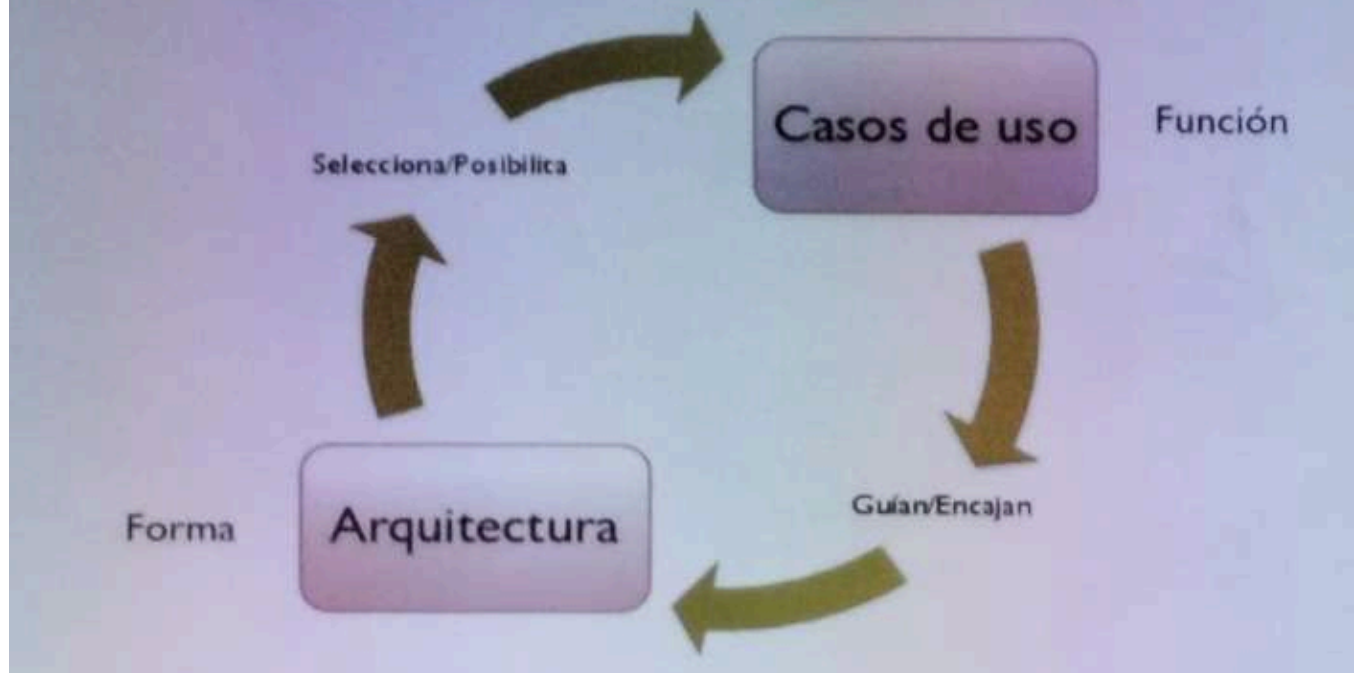
-la arquitectura es el conjunto de estas decisiones

-toman el aspecto dinámico y el aspecto estático

-de cada modelo extrae los elementos principales, de cada uno toma los modelos estructurales

(otro power)

CENTRADO EN LA ARQUITECTURA



- los CU en esencia son funcionalidades
- no es que la funcionalidad no importe, sino que hay muchas formas que pueden dar soporte a las.....
- se recomienda que se empiece bosquejando la arquitectura en general
- definimos aquellos CU que son más importantes(tres o cuatro), no solo del punto de vista de funcionalidad. Sino...
- si en un ejemplo voy a tener muchos usuarios podría pensar usar una arquitectura de Microservicios
- la arquitectura se va construyendo de a poco, en conjunto con todo el sistema. La arquitectura me presenta las características principales

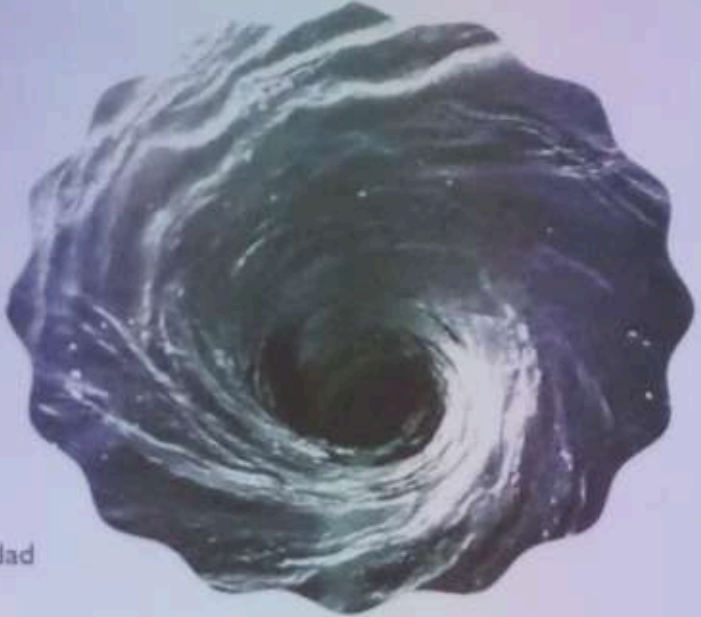
Funciones fundamentales del sistema. De manera resumida, podemos decir que el arquitecto:

- Crea un esquema en borrador de la arquitectura, comenzando por la parte de la arquitectura que no es específica de los casos de uso (por ejemplo, la plataforma). Aunque esta parte de la arquitectura es independiente de los casos de uso, el arquitecto debe poseer una comprensión general de los casos de uso antes de comenzar la creación del esquema arquitectónico.
- A continuación, el arquitecto trabaja con un subconjunto de los casos de uso especificados, con aquellos que representen las funciones clave del sistema en desarrollo. Cada caso de uso seleccionado se especifica en detalle y se realiza en términos de subsistemas (Apéndice A; véase también Sección 3.4.4), clases (Apéndice A), y componentes (Apéndice A).
- A medida que los casos de uso se especifican y maduran, se descubre más de la arquitectura. Esto, a su vez, lleva a la maduración de más casos de uso.

ITERATIVO E INCREMENTAL

ITERATIVO E INCREMENTAL

- Cada Fase se divide en Iteraciones (mini proyectos)
- Cada Iteración resulta en un Incremento
- Iteración controlada (planeada):
 - lo que se incluye en la iteración depende de:
 - Casos de usos extienden la usabilidad del producto.
 - Se atacan los riesgos más importantes.



-dentro de cada fase vamos a tener iteraciones que van pasando por diferentes flujos de trabajo

-después de cada iteración **no** voy a tener necesariamente una nueva funcionalidad, puede ser una modificar, refactorizar (mejorar) o incluso reemplazar partes del trabajo que ya habías hecho. **Un incremento no necesariamente es aditivo.**

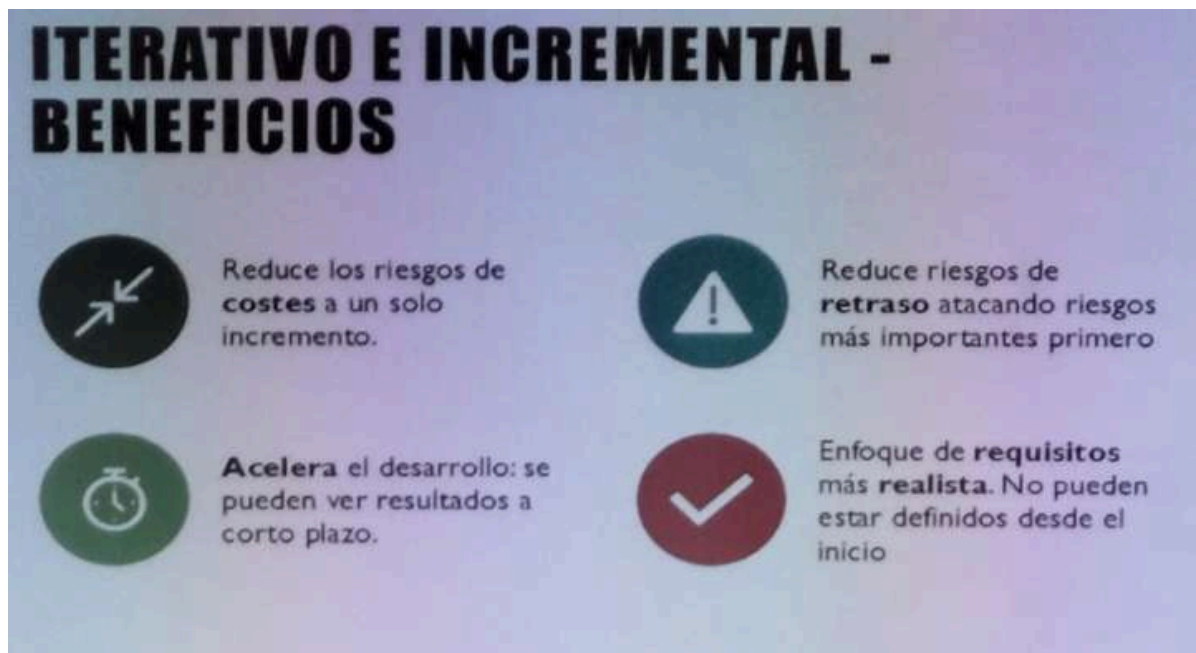
3) esto es bien específico del PU

-vamos a tener una iteración controlada

-la planificación va a depender de los CU que vamos a elegir

-el hecho de que sea iterativo e incremental, hace que esté dirigido por riesgo. Si tenemos uno más riesgoso vamos a tratar de resolver ese primero

BENEFICIOS



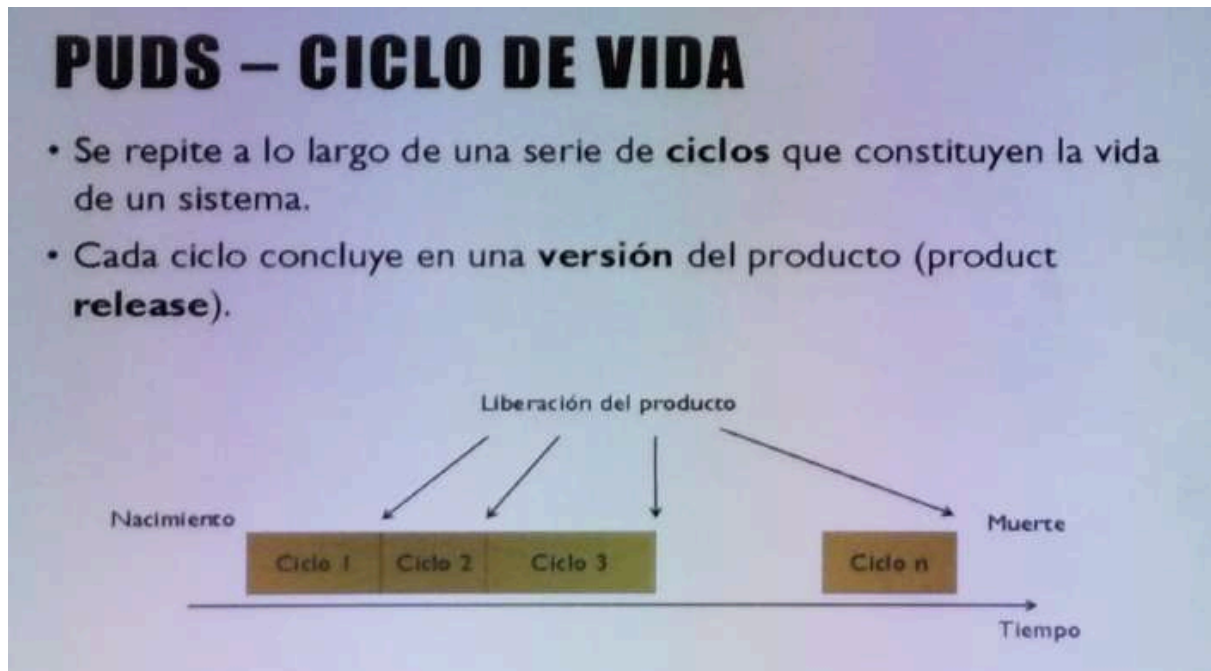
- si nosotros corregimos errores al principio va a ser mucho más barato que si detectamos estos al final, cuando ya tenemos todo el sistema
- todo proyecto de sw va a sufrir retraso, hay que tratar de disminuirlo
- tratar de gestionar los cambios
- con este enfoque siempre volvemos hacer requerimientos, ver los cambios que surgen e ir agregando en nuestro producto. Es más realista porque esos requisitos van a cambiar

Retroalimentación Temprana y Continua (en cada ciclo)

Mayor Calidad del Producto (se corrigen errores constantemente)

Mayor Adaptabilidad al Cambio (El modelo está diseñado para aceptar y gestionar el cambio de forma controlada)

PUDS - CICLO DE VIDA

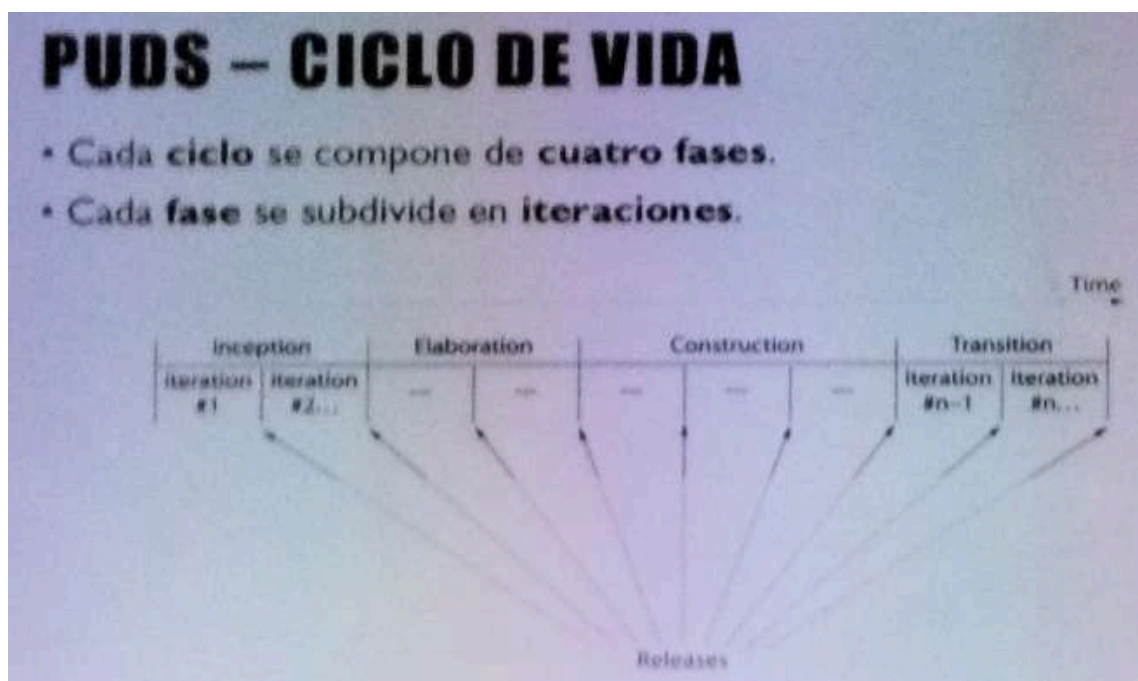


-esto es una perspectiva teórica

-desarrollamos un sistema en el primer ciclo, que termina en una release(versión utilizable) de producto. Esto se va a poder utilizar en producción. Ese mismo producto va surgiendo cambios hasta que llega el momento que ese sw no se va a utilizar más, denominado muerte del sw

-el ciclo de sw está compuesto por varios ciclos, en cada uno hay una versión del producto

Ciclo - Fase - Iteración



-dentro de cada fase existen iteraciones, por eso es iterativo e incremental

- vamos a tener liberaciones de productos
- de un proyecto de sw se obtiene como resultado un producto
- en PU el producto más allá del sw ejecutable, abarca el resto de los modelos
- en la primer fase vamos a tener una release que no va hacer tan ejecutable pero igualmente se la considera así
- en la fase de transición no es la fase de mantenimiento, pero se realizan correcciones para que el producto sea utilizable

-El resultado de cada iteración. Estas son versiones Internas o Incrementos. No se entregan al cliente, pero le permiten al equipo ver el progreso, probar y obtener feedback.

Release

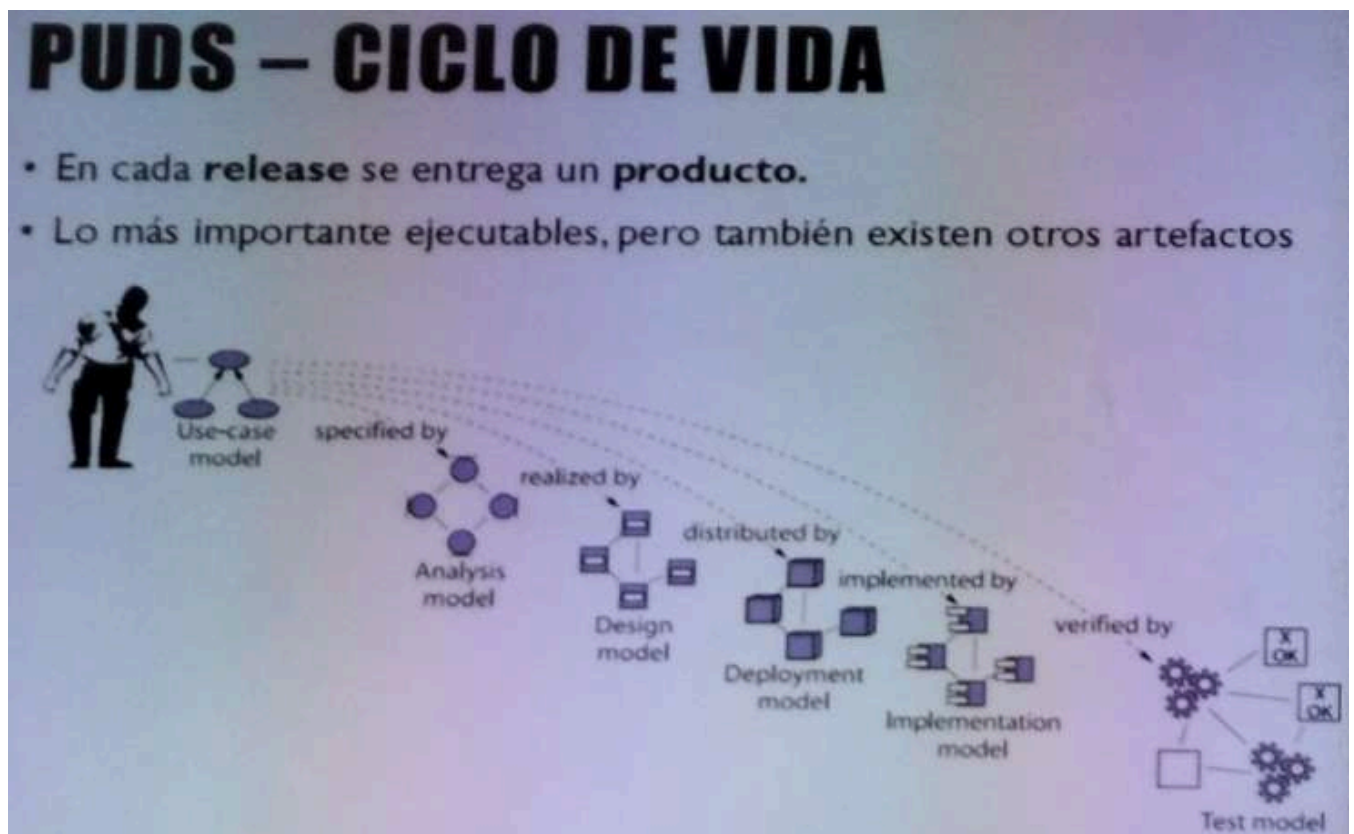


Figura 1.4. Modelo del Proceso Unificado. Existen dependencias entre muchos de los modelos. Como ejemplo, se indican las dependencias entre el modelo de casos de uso y los demás modelos.

- el PU me guía el proyecto de sw. El cual me brinda el producto, el cual va a tener más

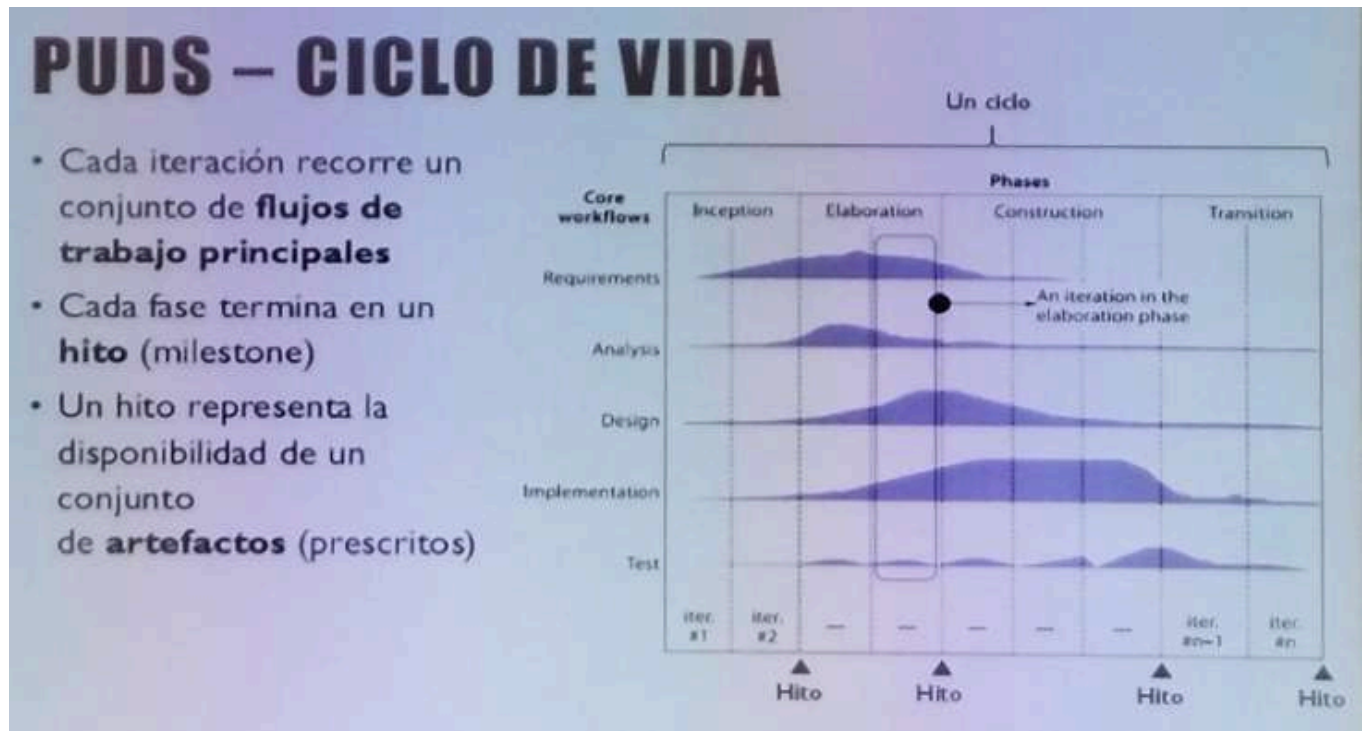
- Un modelo de casos de uso, con todos los casos de uso y su relación con los usuarios.
- Un modelo de análisis, con dos propósitos: refinar los casos de uso con más detalle y establecer la asignación inicial de funcionalidad del sistema a un conjunto de objetos que proporcionan el comportamiento.
- Un modelo de diseño que define: (a) la estructura estática del sistema en la forma de subsistemas, clases e interfaces y (b) los casos de uso reflejados como **colaboraciones** (Apéndice A; véase también la Sección 3.1) entre los subsistemas, clases, e interfaces.
- Un modelo de implementación, que incluye componentes (que representan al código fuente) y la correspondencia de las clases con los componentes.
- Un modelo de despliegue², que define los nodos físicos (ordenadores) y la correspondencia de los componentes con esos nodos.
- Un modelo de prueba, que describe los casos de prueba que verifican los casos de uso.
- Y, por supuesto, una representación de la arquitectura.

El sistema también debe tener un modelo del dominio o modelo del negocio que describa el contexto del negocio en el que se halla el sistema.

Todos estos modelos están relacionados. Juntos, representan al sistema como un todo. Los elementos de un modelo poseen dependencias de **traza** (Apéndice A; véase también la Sección 2.3.7) hacia atrás y hacia adelante, mediante enlaces hacia otros modelos. Por ejemplo, podemos hacer el seguimiento de un caso de uso (en el modelo de casos de uso) hasta una realización de caso de uso (en el modelo de diseño) y hasta un caso de prueba (en el modelo de prueba). La trazabilidad facilita la comprensión y el cambio.

Diagrama

(Diagrama muy importante, resume prácticamente todo)



-(gaonita hizo unas anotaciones en castellano al gráfico que es del libro)

-a la izquierda están las actividades que vamos a ir poder ejecutando en ese tiempo

-al fin de cada iteración, van a estar los hitos menores

-las release vamos a tener **release externas** para presentar al cliente y **release internas** son las que quedan dentro del equipo de trabajo

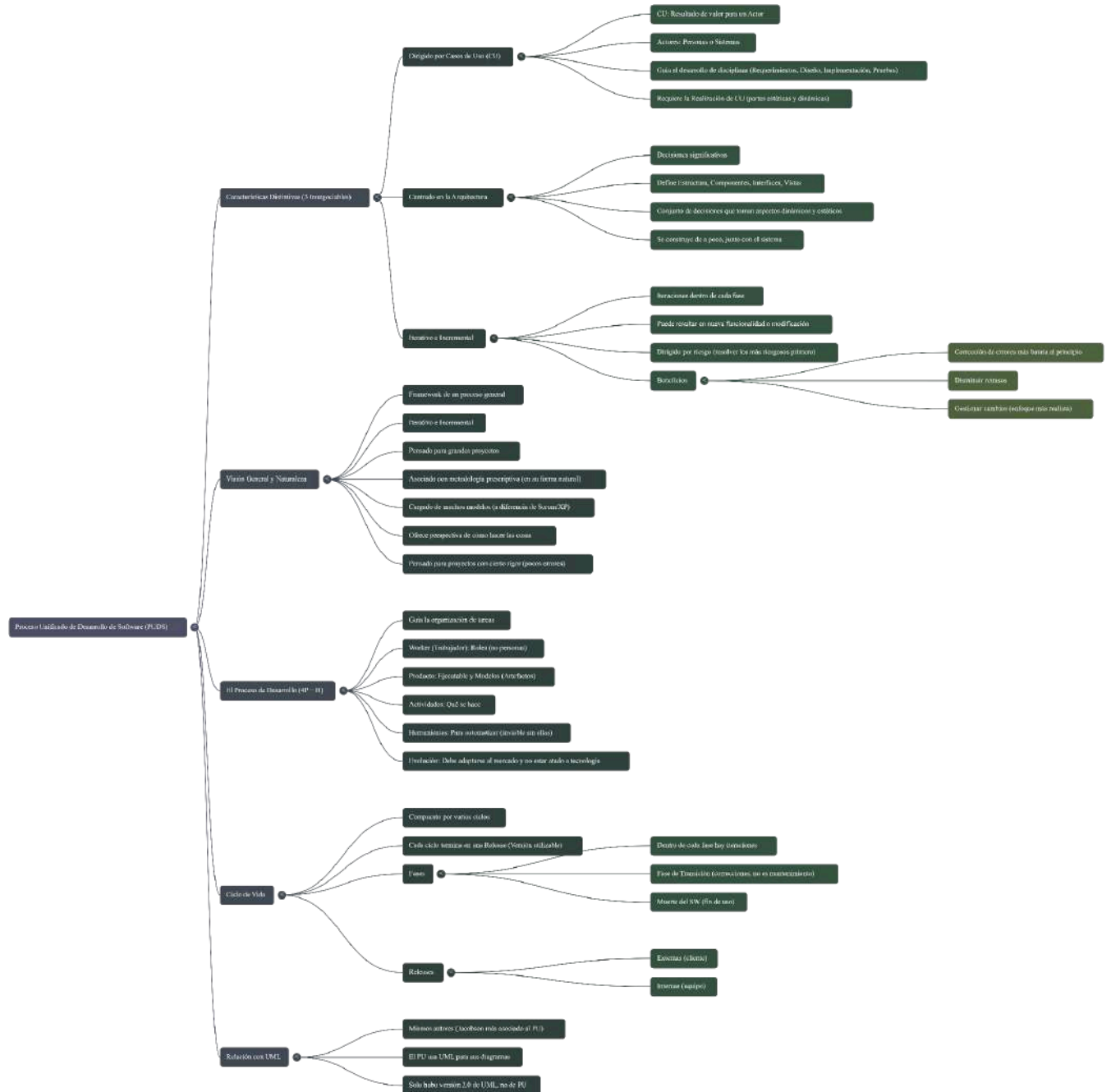
-tenemos ciertos artefactos pero también podemos decir que tan completos están

-el hito es un punto de control

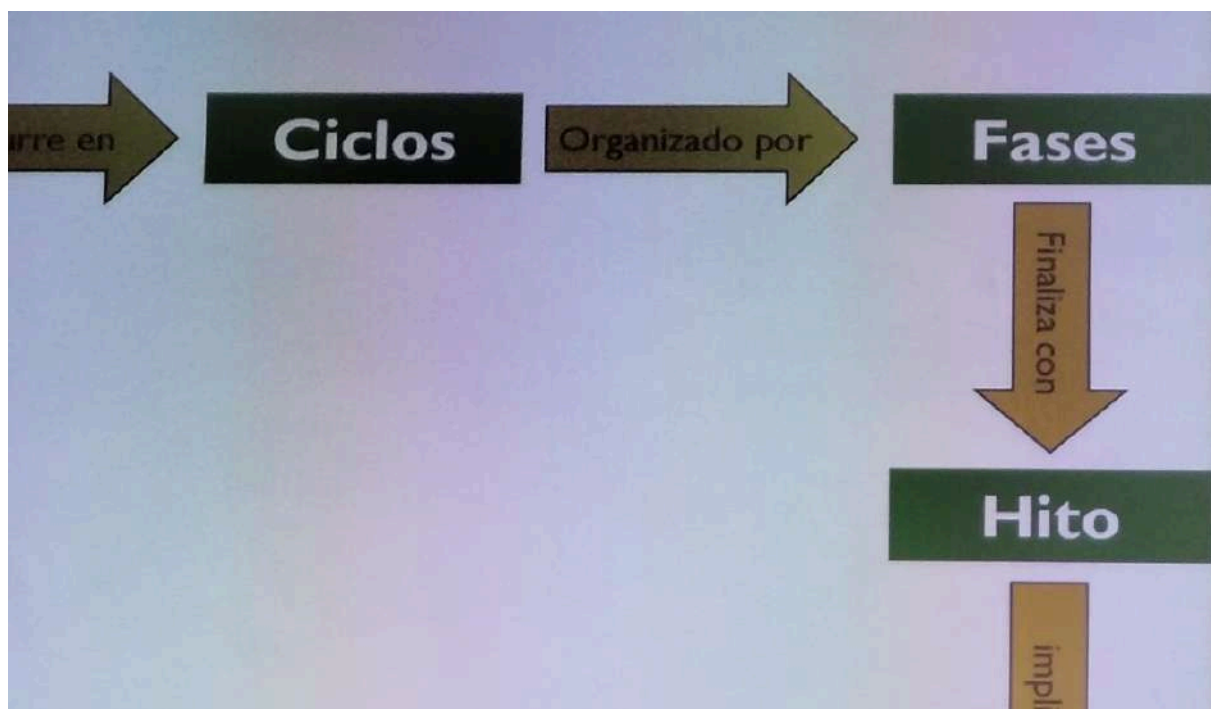
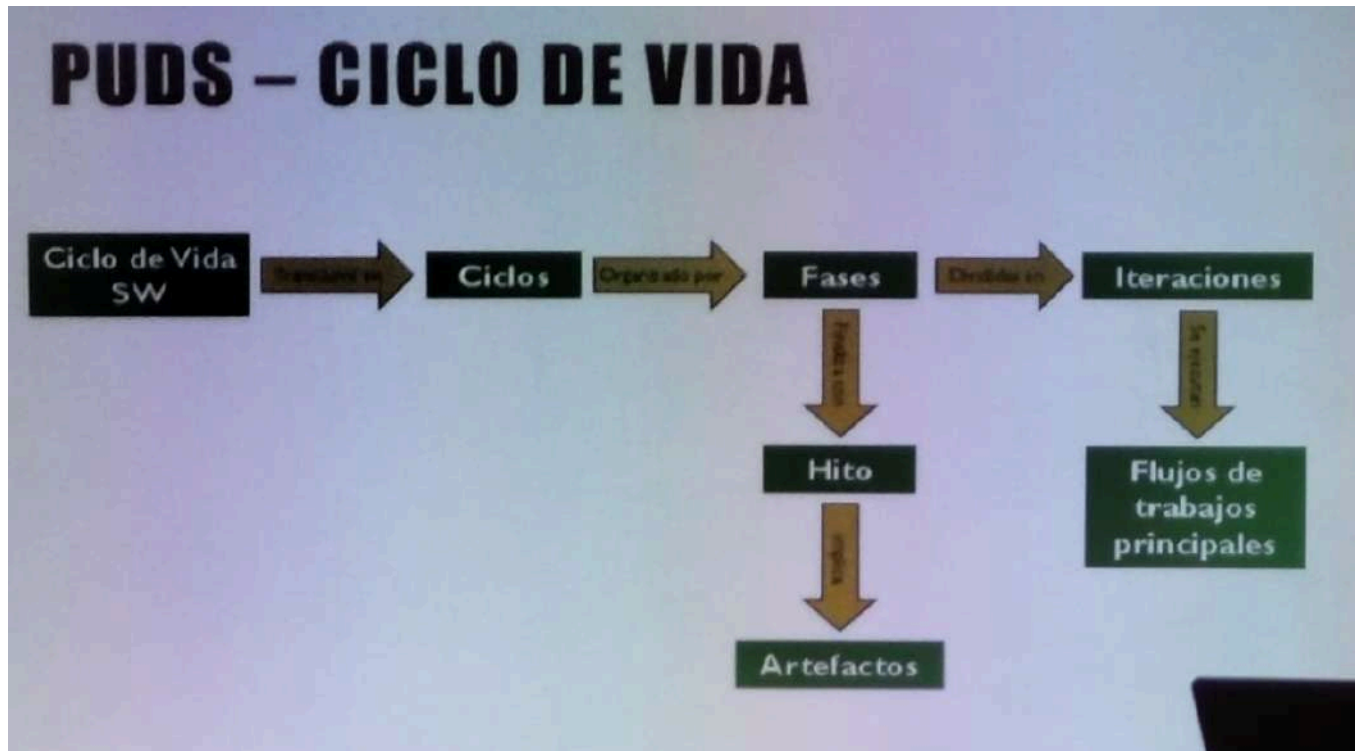
Mapa mental - CONCEPTOS CLAVE

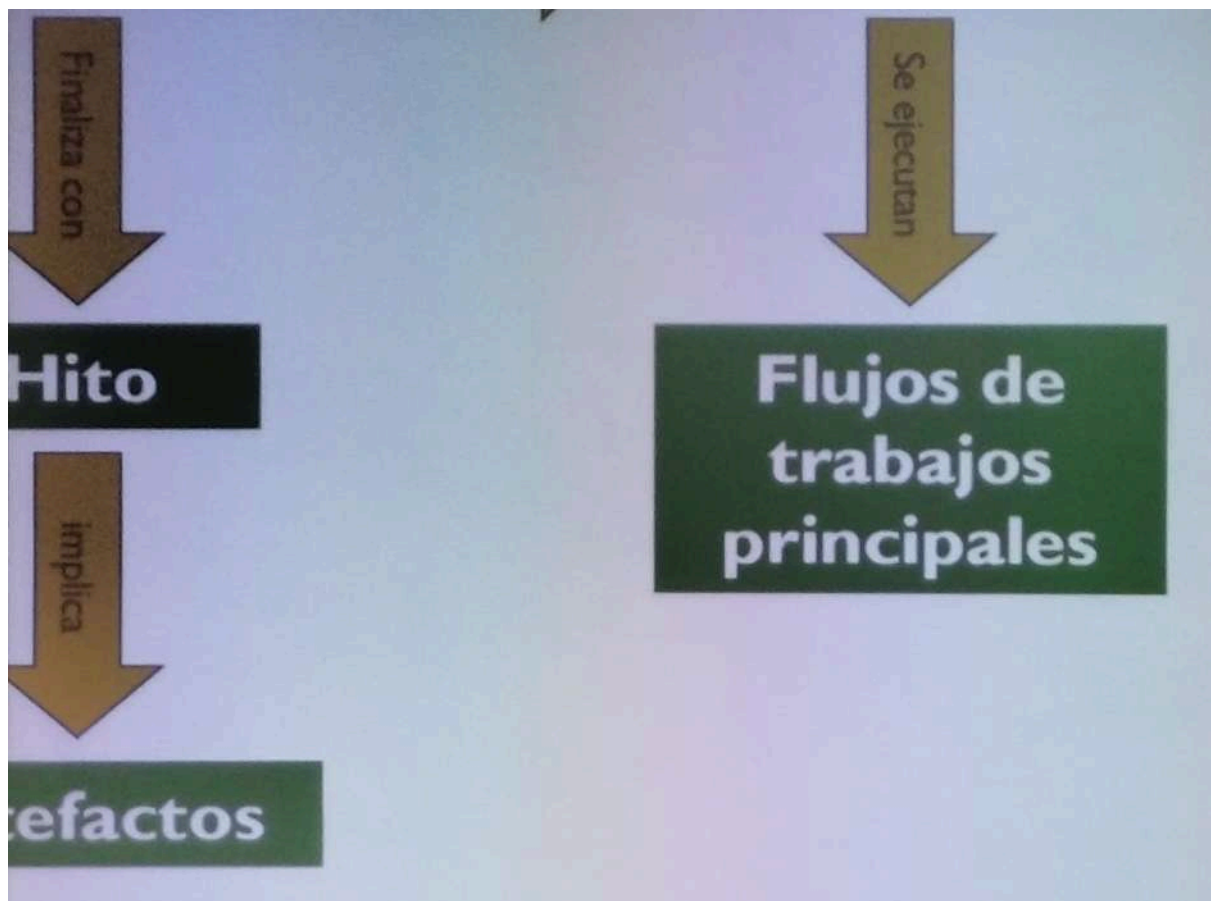
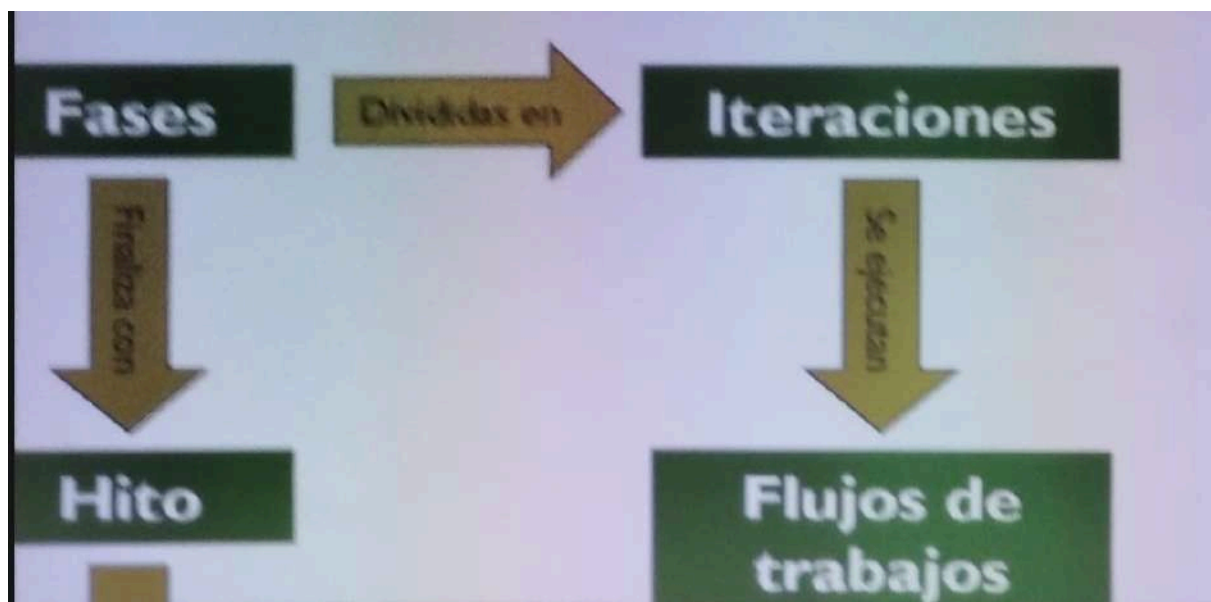
(Gaona nos dijo que hagamos un mapa mental de lo visto)

le tire el pdf a notebook LM y me realizó el siguiente mapa



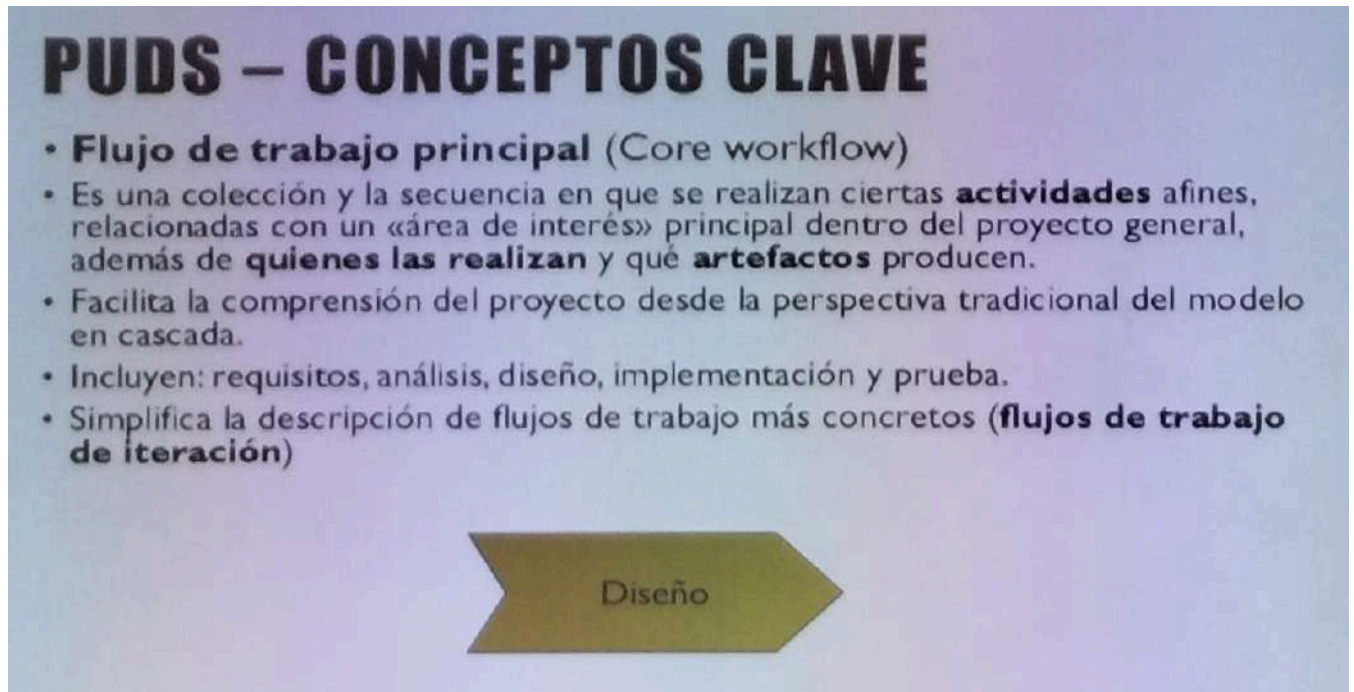
(gráfico del profe)





PUDS - CONCEPTOS CLAVE

Flujo de trabajo principal (Core workflow)



2) en principio es un agrupamiento de actividades, que no tiene una secuencia como tal. Pero cuando la ejecutamos si debe tener una secuencia.

-quienes las realizan serían los Trabajadores

4) flujos de trabajo

-el modelo en cascada es fácil de entender, pero no es realista. Entonces el PU toma las partes de las disciplinas del mismo pero lo implementa de otra forma

-flujos de trabajo de la iteración

-cuando se hablaba de un **flujo de trabajo fundamental** desde el punto de vista de diseño esa es la propuesta genérica que tenemos, pero luego como vamos a ejecutar en una iteración dentro de una fase particular va a depender de cada proyecto

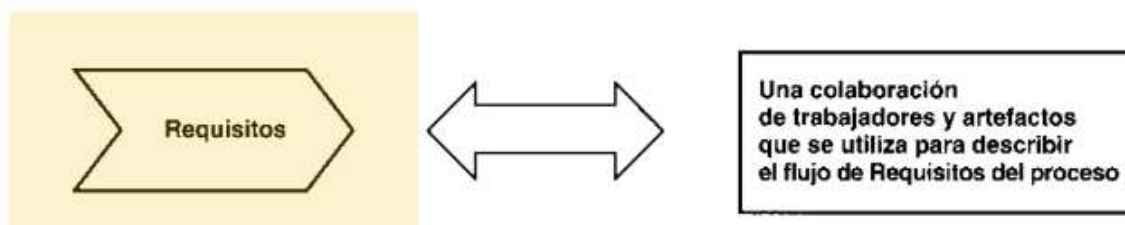


Figura 2.7. La notación “en forma de pez” es la abreviatura de un flujo de trabajo.

Actividad (Activity)

PUDS – CONCEPTOS CLAVE

- **Actividad (Activity)**
 - Una unidad de trabajo tangible realizada por un **trabajador** en un **flujo de trabajo** que:
 - implica una responsabilidad bien definida para el trabajador;
 - produce un resultado bien definido (artefactos) basado en una entrada bien definida (otros artefactos) y
 - probablemente se mencionará en un plan de proyecto cuando las tareas se asignan a individuos.
 - Las actividades pueden estar descompuestas en **pasos**.



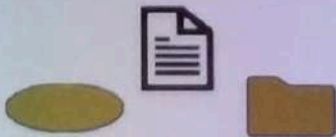
Diseñar caso de uso

- actividades dentro del flujo de trabajo
- vamos a tener una responsabilidad bien definida para cada trabajador
- va a tener artefactos de entrada que van a estar ocupando y artefactos de salida que son los que va a estar produciendo
- fijense que además estas actividades van a depender del plan de proyecto. Dentro de ese plan en el contexto de PU va a tener un Ingeniero de CU pero dentro de esa actividad se le va asignar a una persona en específico

Artefacto (Artifact)

PUDS – CONCEPTOS CLAVE

- **Artefacto (Artifact)**
 - Una pieza tangible de información que:
 - es creada, modificada y utilizada por los trabajadores al realizar actividades,
 - representa un área de responsabilidad y
 - es probable que se ponga bajo un control de versiones separado.
 - Puede ser un modelo, un elemento de modelo o un documento.



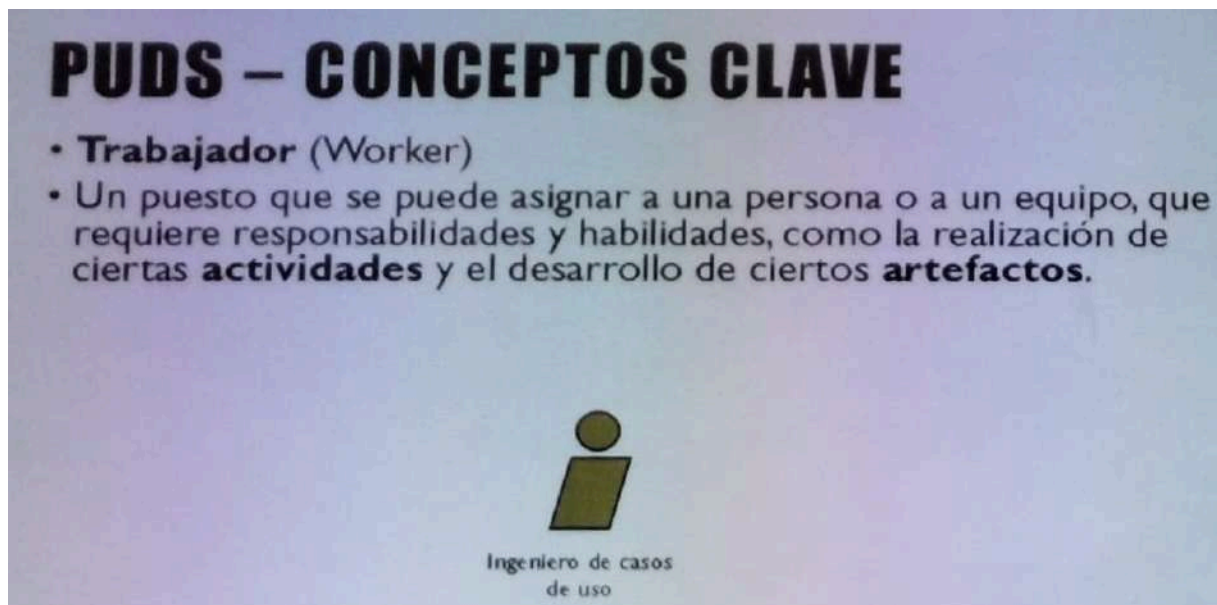
- un Artefacto puede ser cualquier cosa que se produzca básicamente va a ser un artefacto

- cada artefacto va a estar dentro de un control de versiones.
- se puede tener otros modelos versionados

Básicamente, hay dos tipos de artefactos: artefactos de ingeniería y artefactos de gestión. Este libro se centra en los artefactos de ingeniería creados durante las distintas fases del proceso (requisitos, análisis, diseño, implementación y prueba).

Sin embargo, el desarrollo de software también requiere artefactos de gestión. Varios de estos artefactos tienen un tiempo de vida corto —sólo duran lo que dure la vida del proyecto. A

Trabajador (Worker)



-dentro de lo que es PU se le puso ese nombre a ese trabajador. Hay algunos más conocidos y otros son más específicos de este proceso

Cada **trabajador** (un trabajador concreto) es responsable de un conjunto completo de actividades, como las actividades necesarias para el diseño de un subsistema. Para trabajar eficazmente, los trabajadores **necesitan la información** requerida para llevar a cabo sus actividades. **Necesitan comprender cuáles son sus roles** en relación con los de otros trabajadores. Al mismo tiempo, si tienen que hacer su trabajo, **las herramientas que emplean deben ser las adecuadas**. Las herramientas no sólo deben ayudar a los trabajadores a llevar a cabo sus propias actividades, sino que también deben aislarles de la información que no les sea relevante. Para lograr estos objetivos, el Proceso Unificado describe formalmente los puestos —es decir, los trabajadores— que las personas pueden asumir en el proceso.

La Figura 2.2 ilustra cómo **una persona puede ser varios trabajadores dentro de un proyecto**.

Un trabajador también puede representar a un conjunto de personas que trabajan juntas. Por ejemplo, un trabajador arquitecto puede ser un grupo de arquitectura.

Cada **trabajador tiene un conjunto de responsabilidades y lleva a cabo un conjunto de actividades** en el desarrollo de software.

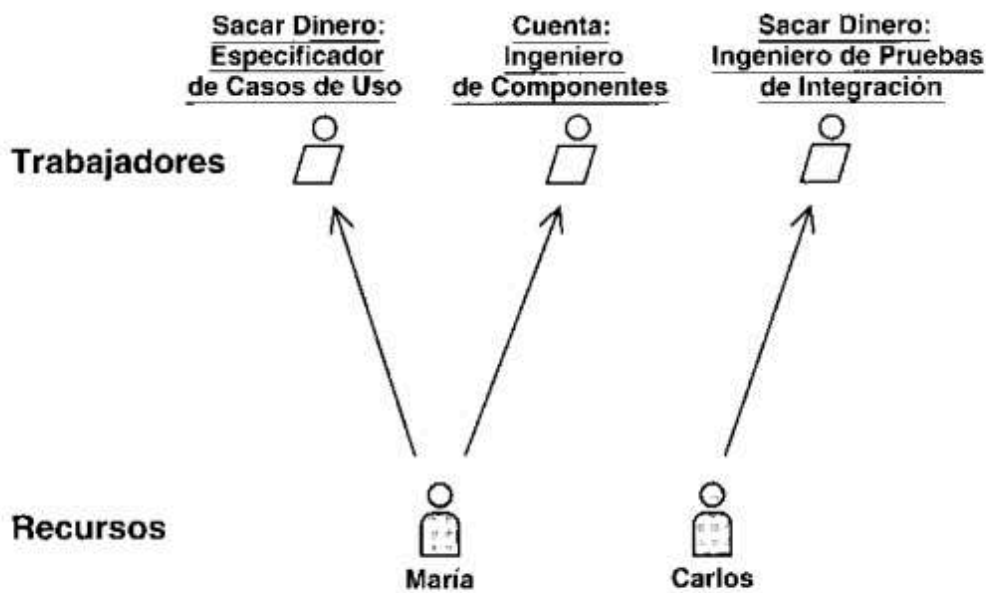
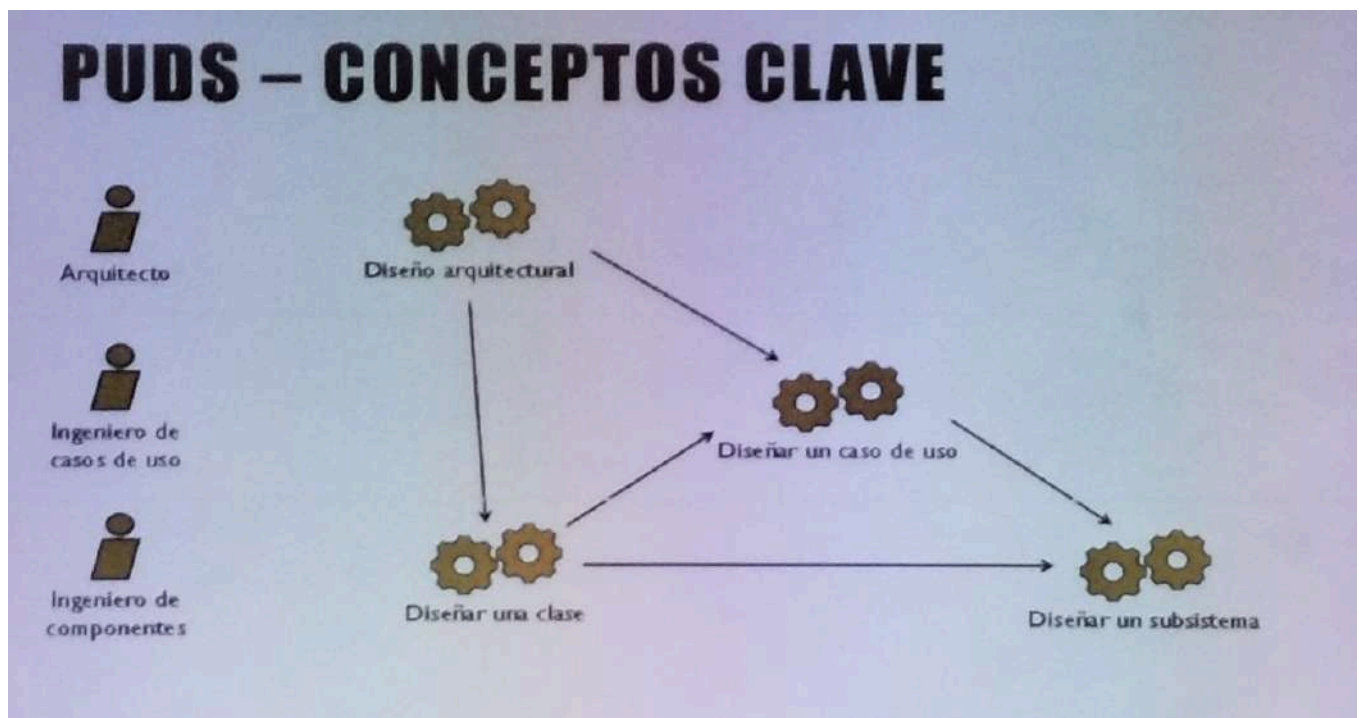


Figura 2.2. Trabajadores y recursos que los realizan.

Ejemplo - Flujo de trabajo de Diseño



(faltan las líneas dividiendo a cada uno)

-se tienen los flujos que no necesariamente se debe dar de esta forma. Es un flujo posible

Modelo

La construcción de un sistema es por tanto un proceso de construcción de modelos, utilizando distintos modelos para describir todas las perspectivas diferentes del sistema. La elec-



Figura 2.4. El conjunto fundamental de modelos del Proceso Unificado¹.

2.3.4. ¿Qué es un modelo?

Un modelo es una abstracción del sistema, especificando el sistema modelado desde un cierto punto de vista y en un determinado nivel de abstracción [1]. Un punto de vista es, por ejemplo, una vista de especificación o una vista de diseño del sistema.

Los modelos son abstracciones del sistema que construyen los arquitectos y desarrolladores. Por ejemplo, los trabajadores que modelan los requisitos funcionales siguen en el sistema con

2.3.5. Cada modelo es una vista autocontenida del sistema

Un modelo es una abstracción semánticamente cerrada del sistema. Es una vista autocontenida en el sentido de que un usuario de un modelo no necesita para interpretarlo más información (de otros modelos).

Si bien los modelos tienen relaciones de traza entre sí, no necesitamos de la información de otro para poder entender/comprender el que estamos leyendo. Esto quiere decir que es Autocontenida, tiene la información suficiente para representarse por sí solo

En cambio (y también por tener relaciones de traza), si dependemos del modelo anterior para poder realizar el siguiente.

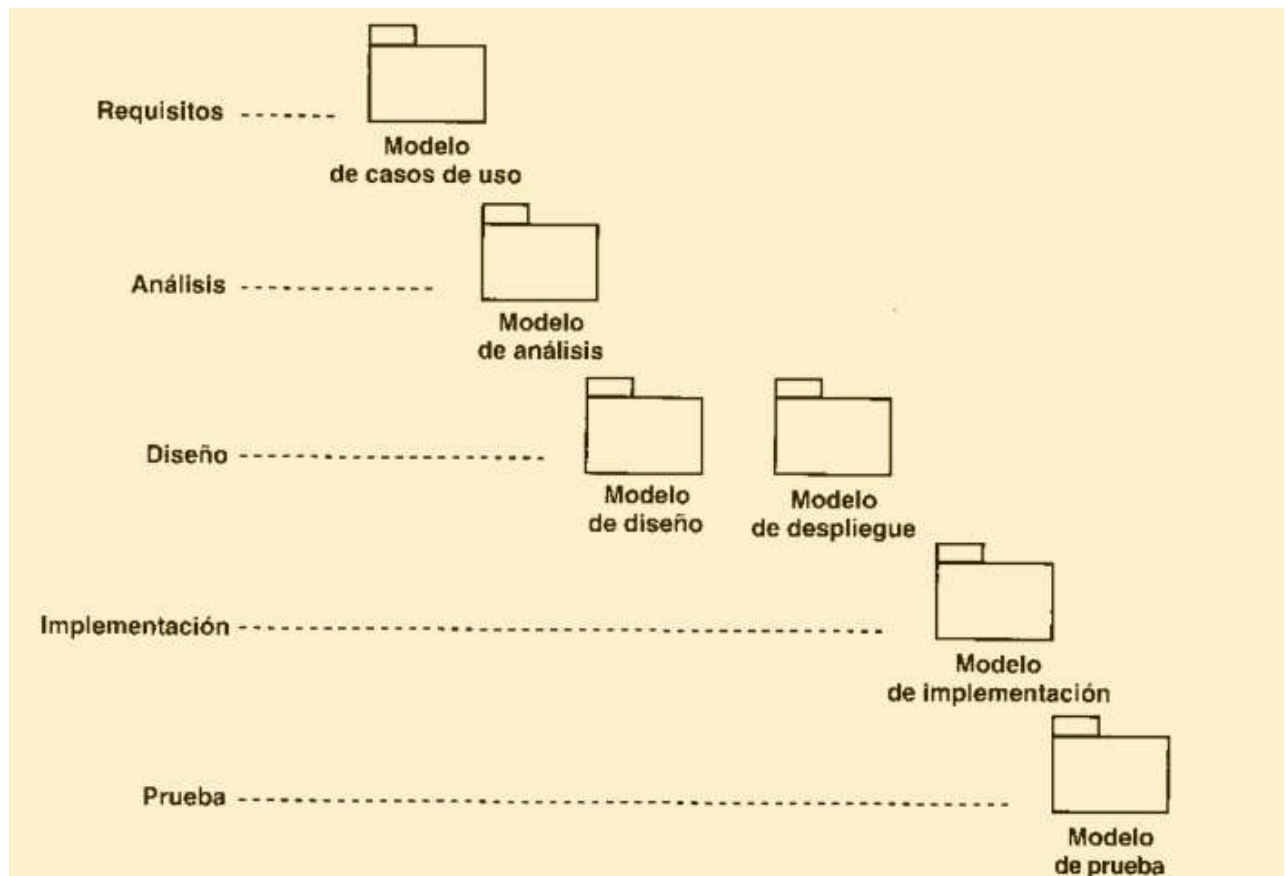
2.3.6. Dentro de un modelo

Un modelo siempre identifica el sistema que está modelando. Este elemento del sistema es por tanto el contenedor de los demás elementos. El subsistema de más alto nivel (Apéndice B) representa al sistema en construcción. En el modelo de casos de uso, el sistema contiene casos



Figura 2.5. Los modelos están estrechamente enlazados unos con otros mediante trazas.

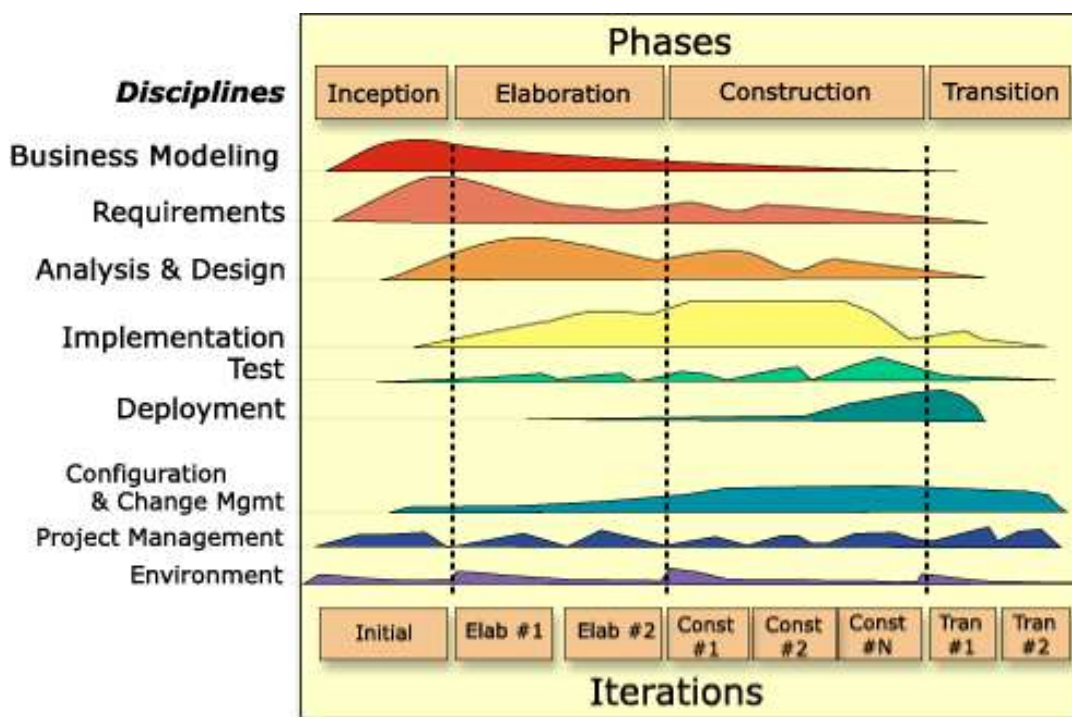
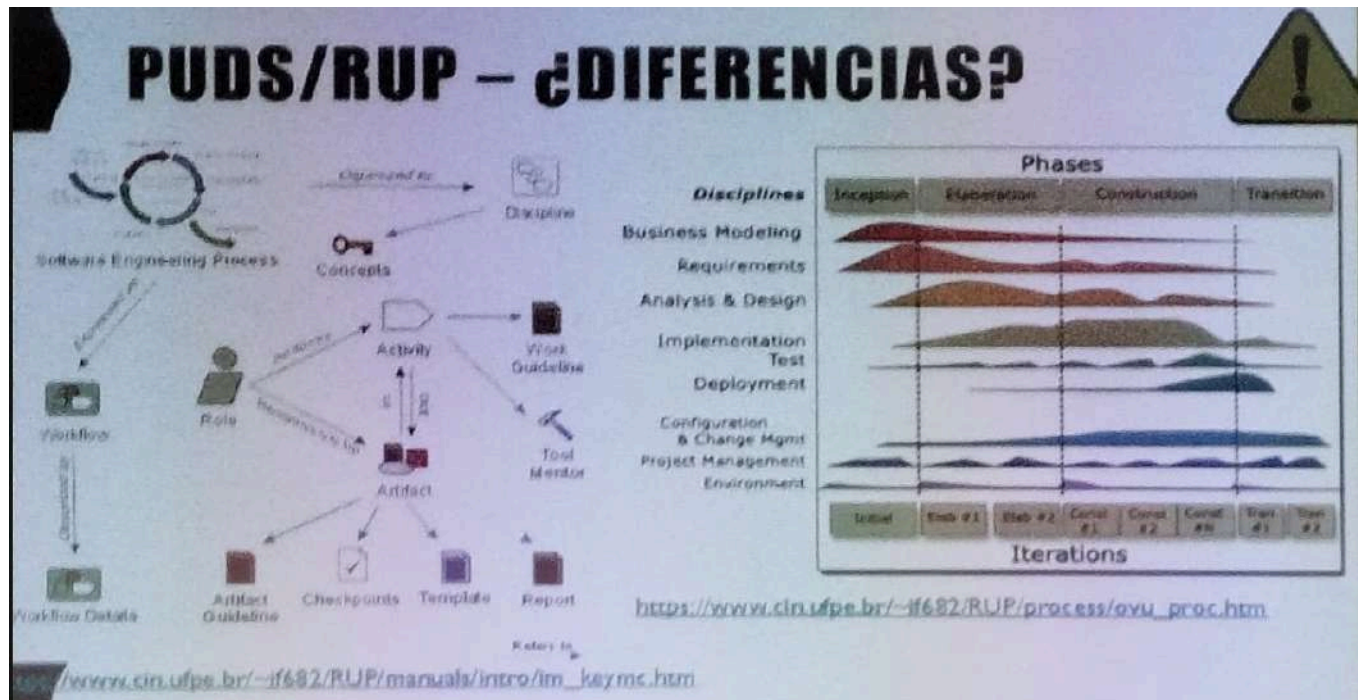
Una relación de traza no añade información a los elementos involucrados, sino que simplemente los une. Si añadiera información, no podríamos decir que los modelos son Autocontenidos



- Un modelo de casos de uso, con todos los casos de uso y su relación con los usuarios.
- Un modelo de análisis, con dos propósitos: refinar los casos de uso con más detalle y establecer la asignación inicial de funcionalidad del sistema a un conjunto de objetos que proporcionan el comportamiento.
- Un modelo de diseño que define: (a) la estructura estática del sistema en la forma de subsistemas, clases e interfaces y (b) los casos de uso reflejados como **colaboraciones** (Apéndice A; véase también la Sección 3.1) entre los subsistemas, clases, e interfaces.
- Un modelo de implementación, que incluye componentes (que representan al código fuente) y la correspondencia de las clases con los componentes.
- Un modelo de despliegue², que define los nodos físicos (ordenadores) y la correspondencia de los componentes con esos nodos.
- Un modelo de prueba, que describe los casos de prueba que verifican los casos de uso.
- Y, por supuesto, una representación de la arquitectura.

El sistema también debe tener un modelo del dominio o modelo del negocio que describa el contexto del negocio en el que se halla el sistema.

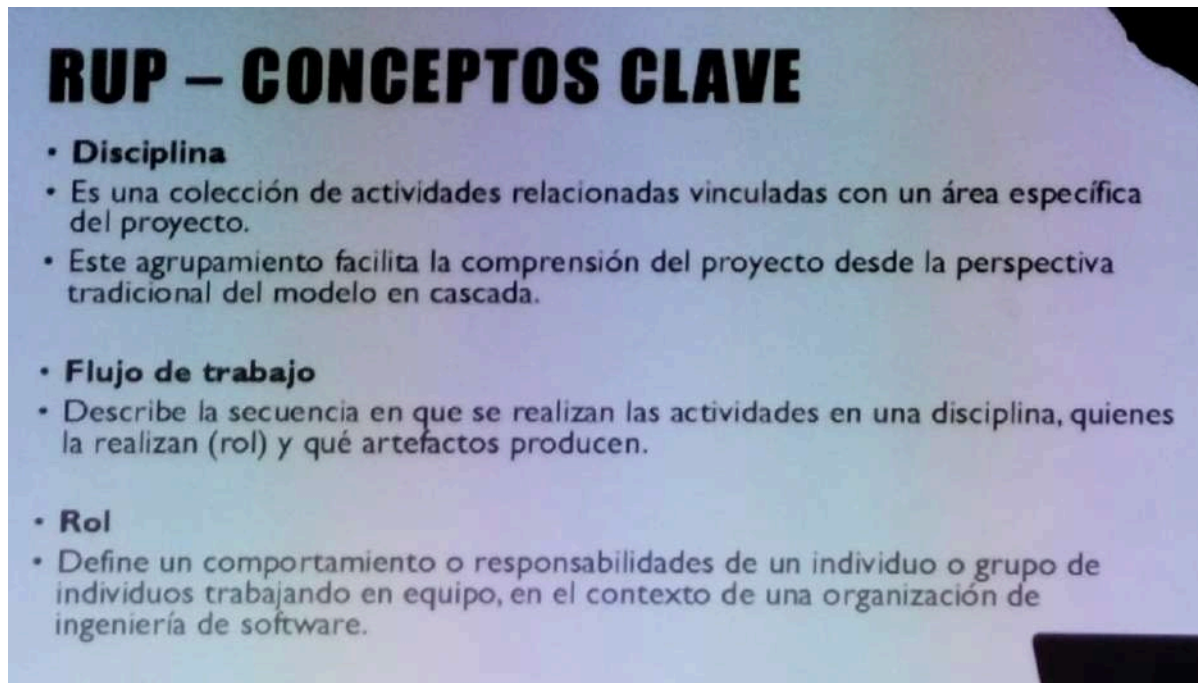
PUDS/RUP - DIFERENCIAS?



- se parece mucho pero tiene algunos detalles de diferencia
- algunos conceptos pueden llegar a traer inconvenientes. Lo que se conoce como Rol en RUP en el PU se conoce como Trabajador.
- contienen más disciplinas. Las últimas disciplinas extras no se tratan en el PU directamente. El RUP juntan algunas disciplinas del PU
- el RUP tiene mucho más detalle a comparación del PU
- en este enlace se puede tener acceso a todas las plantillas

CLASE - 01/10

RUP - CONCEPTOS



RUP – CONCEPTOS CLAVE

- **Disciplina**
 - Es una colección de actividades relacionadas vinculadas con un área específica del proyecto.
 - Este agrupamiento facilita la comprensión del proyecto desde la perspectiva tradicional del modelo en cascada.
- **Flujo de trabajo**
 - Describe la secuencia en que se realizan las actividades en una disciplina, quienes la realizan (rol) y qué artefactos producen.
- **Rol**
 - Define un comportamiento o responsabilidades de un individuo o grupo de individuos trabajando en equipo, en el contexto de una organización de ingeniería de software.

RUP → a veces se lo menciona como proceso unificado, porque uno deriva del otro. se hizo una versión más genérica. Aunque se parezcan mucho no son lo mismo

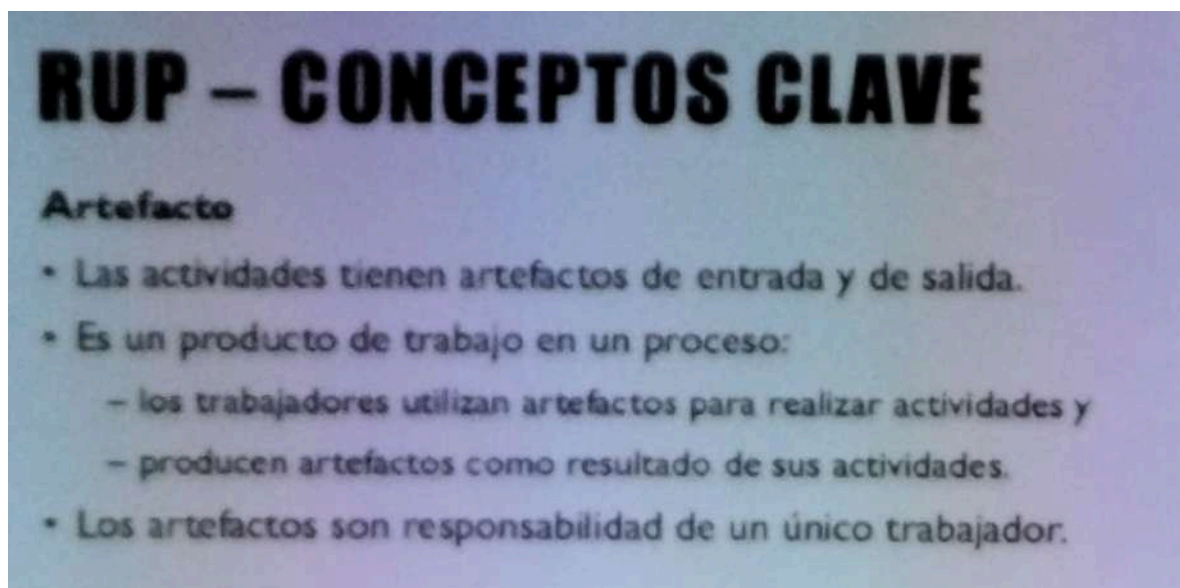
Disciplina → flujo de trabajo fundamental en PU

Rol → trabajador

Artefacto → similar

Ciclo de vida → desde que nace

Ciclo → serie de fases



RUP – CONCEPTOS CLAVE

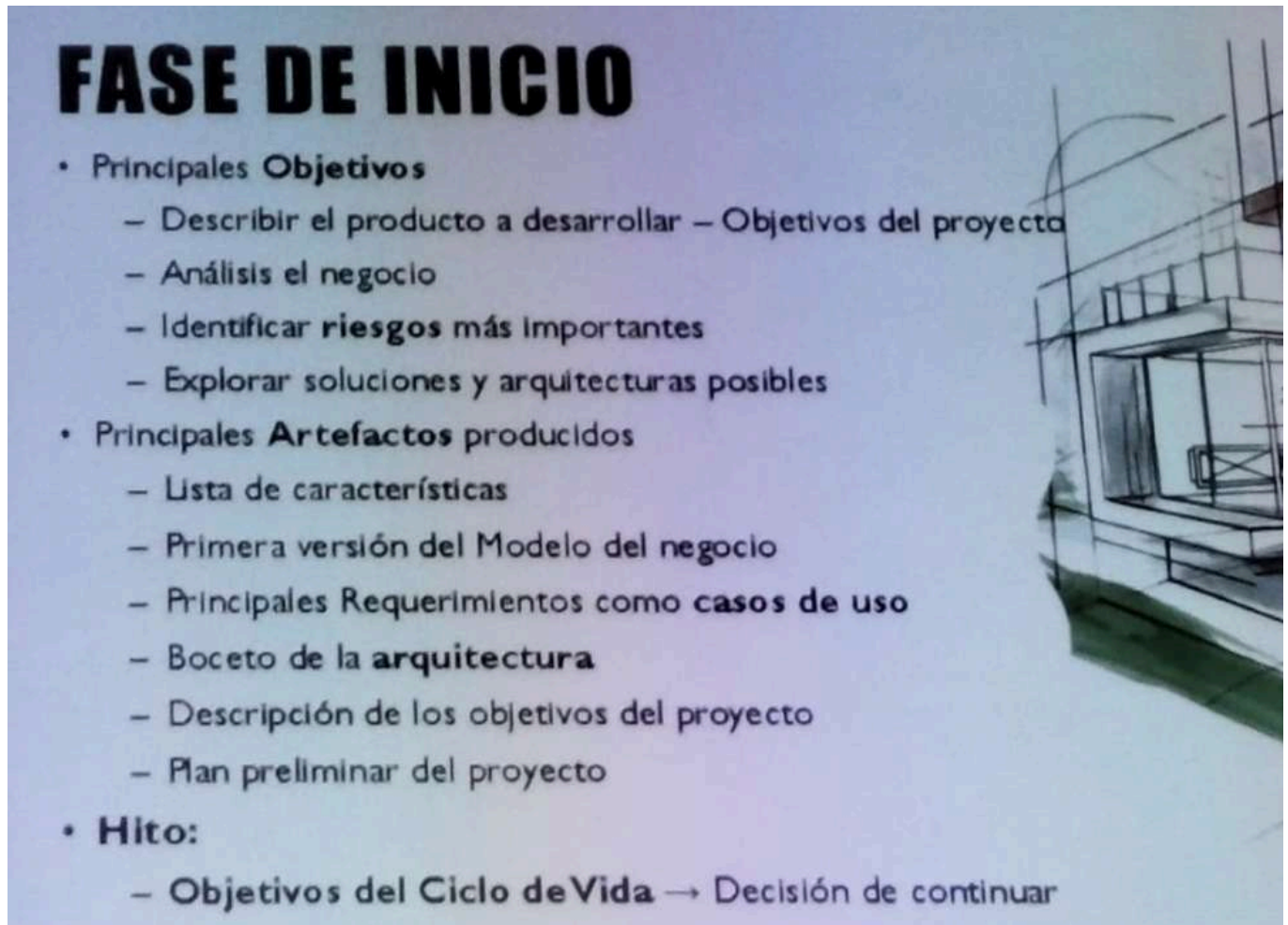
Artefacto

- Las actividades tienen artefactos de entrada y de salida.
- Es un producto de trabajo en un proceso:
 - los trabajadores utilizan artefactos para realizar actividades y
 - producen artefactos como resultado de sus actividades.
- Los artefactos son responsabilidad de un único trabajador.

FASES



FASE DE INICIO



FASE DE INICIO

- Principales **Objetivos**
 - Describir el producto a desarrollar – Objetivos del proyecto
 - Análisis el negocio
 - Identificar **riesgos** más importantes
 - Explorar soluciones y arquitecturas posibles
- Principales **Artefactos** producidos
 - Lista de características
 - Primera versión del Modelo del negocio
 - Principales Requerimientos como **casos de uso**
 - Boceto de la **arquitectura**
 - Descripción de los objetivos del proyecto
 - Plan preliminar del proyecto
- **Hito:**
 - **Objetivos del Ciclo de Vida** → Decisión de continuar

-nosotros definimos si completamos una fase con los hitos

-feature

-la **diferencia** de requisitos y característica es que la característica puede englobar varios requisitos. Ej: gestión de clientes, gestión de stock, gestión de productos

-uno de los objetivos es identificar la factibilidad del proyecto

-el hecho de que sea iterativo e incremental hace que sea menos riesgoso

-vista de características

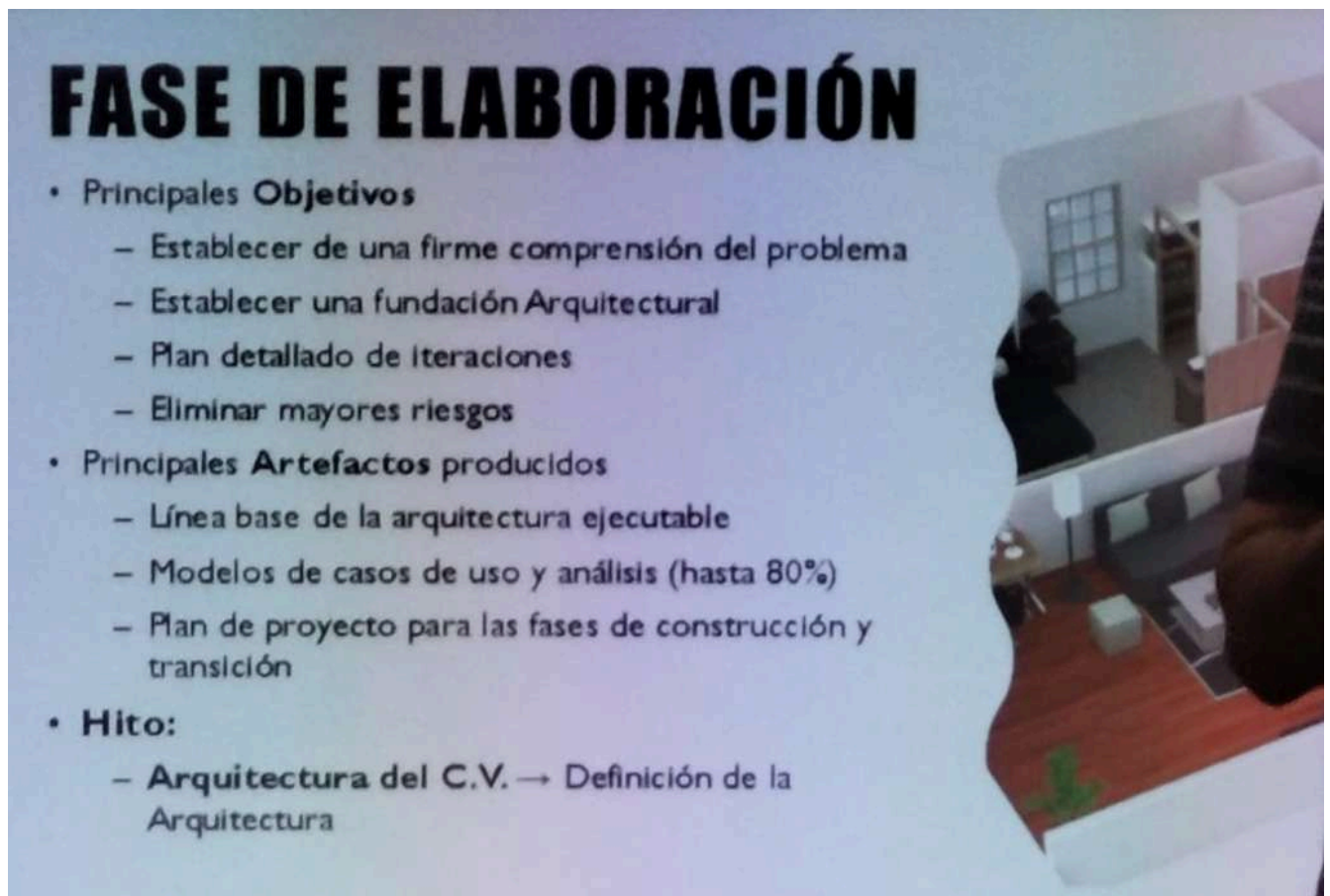
-boceto de arquitectura

-el modelo de negocio nos sirve para mencionar en alto nivel el dominio del problema

-un caso especial del modelo de negocio es el Modelo de dominio que modelamos a través de clases conceptuales

-los CU son formas de modelar procesos de negocios

ELABORACIÓN



FASE DE ELABORACIÓN

- Principales **Objetivos**
 - Establecer de una firme comprensión del problema
 - Establecer una fundación Arquitectural
 - Plan detallado de iteraciones
 - Eliminar mayores riesgos
- Principales **Artefactos** producidos
 - Línea base de la arquitectura ejecutable
 - Modelos de casos de uso y análisis (hasta 80%)
 - Plan de proyecto para las fases de construcción y transición
- **Hito:**
 - **Arquitectura del C.V.** → Definición de la Arquitectura

-tener definida la línea base de la arquitectura, es una línea que consideramos relativamente estable (no va a cambiar). El PU esta preparado para el cambio de requisitos, pero lo que pide es que al final de la fase de elaboración es tener la arquitectura definida o que se parezca mucho a la final ya que cambiar la arquitectura va a costar mucho

-tenemos que tener definido el esqueleto del sistema

-tenemos que tener el QUÉ pero no todavía el CÓMO

-vamos a mirar un poco más a largo plazo lo que es el proyecto

-cuando tenemos un enfoque ágil, la planificación se hace muy poco

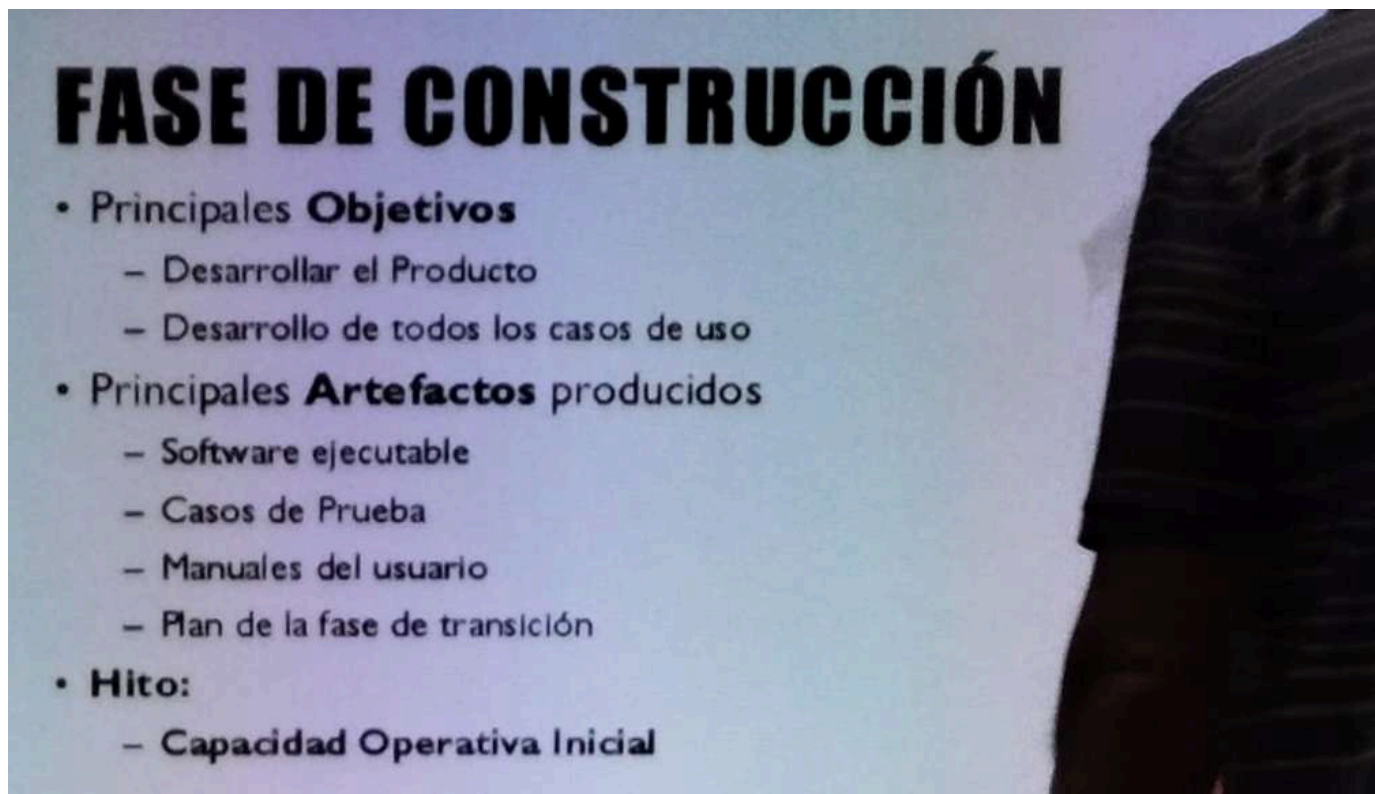
-lo que siempre son difíciles de diferenciar es la fase de construcción de la de elaboración

-la fase de elaboración tiene que ver mucho con construir CU

-la fase de construcción tiene que ver más con el sistema final y la implementación

-Arquitectura del Ciclo de Vida

FASE DE CONSTRUCCIÓN



-en esencia es desarrollar el producto de sw

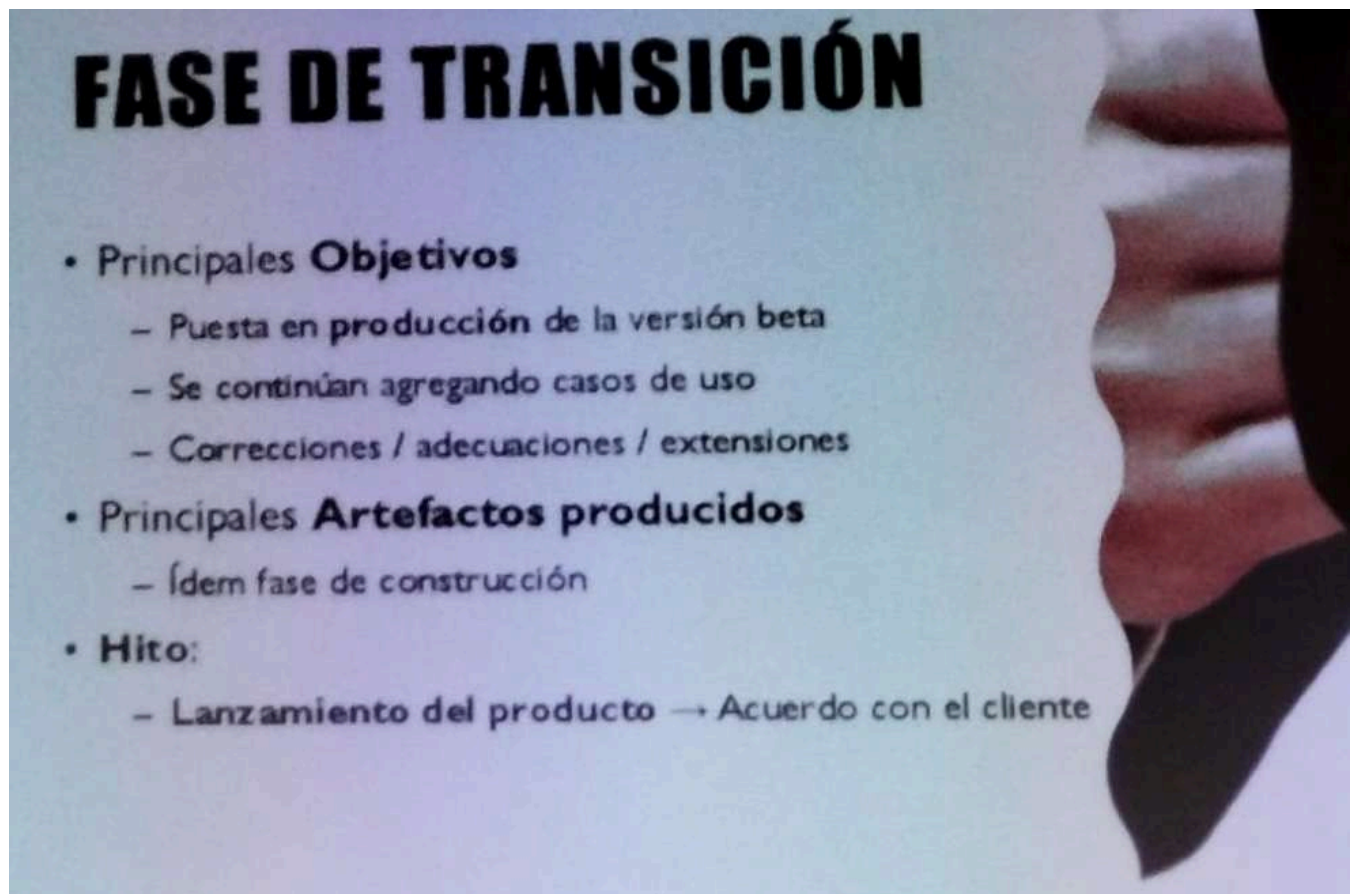
-deberían estar definidos todos los CU

-el artefacto más importante de todos es el sw ejecutable

-tenemos una versión funcional completa del sistema, pero no está libre de errores

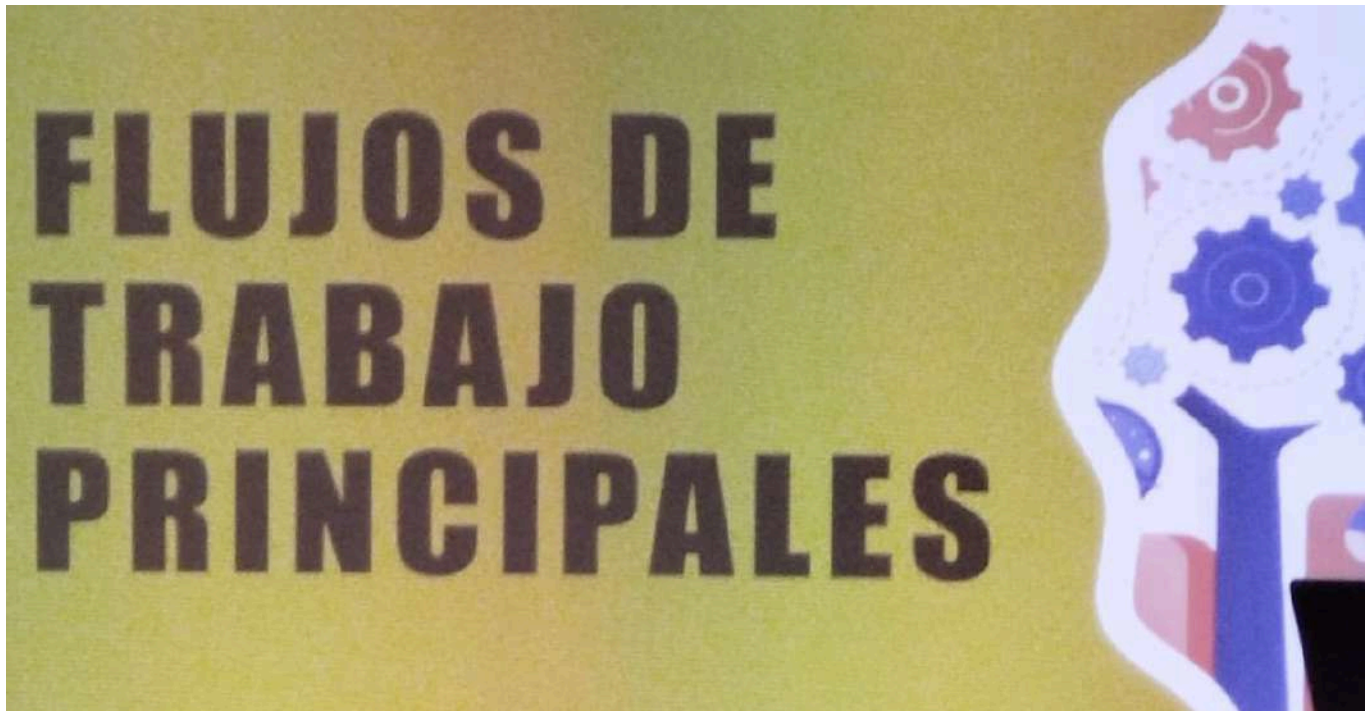
-al final de esta fase tenemos el sistema desarrollado casi en su totalidad

FASE DE TRANSICIÓN



- los artefactos son iguales a la fase anterior
- la diferencia es que el producto va a estar instalado en el lugar que debe funcionar
- esta siendo usado por los usuarios finales
- es una fase donde se detecta correcciones o agregados, aparecen esos detalles que no se vieron mientras se estaba construyendo
- puede ser que se agreguen CU pero ya van a ser menores
- al final de esta fase vamos a tener el producto funcional en el sitio del cliente
- cuando termina esta fase termina un ciclo

FLUJOS DE TRABAJO PRINCIPALES



(en el capítulo 3 de PU, explica que significa que el PUD sea dirigido por CU. Tiene un buen resumen de todas las disciplinas)

- los CU son el punto de inicio y conductor de todo el proceso

- los modelos de CU son todos los CU, aunque no se obtienen todos de una vez. Se va formando durante el flujo de trabajo de requisitos

- una vez que tengamos eso vamos a intentar transformarlos en la etapa de análisis. Se realiza un modelo conceptual sin tanto detalle

- una vez que pasamos la etapa de análisis pasamos a la de diseño. Que va a ver cómo tomar esas clases y plasmarlas en una tecnología en particular. Se analiza qué lenguajes usar, ventanas, que framework usar, todo lo que tenga que ver con tecnología se ve aca

- el modelo de diseño es más bien física

- en el modelo de implementación tomamos aquello que diseñamos. Ya vamos a tener ejecutables de como agrupamos eso que antes vimos

- todo eso que tenemos se ve al final con casos y procedimientos de prueba

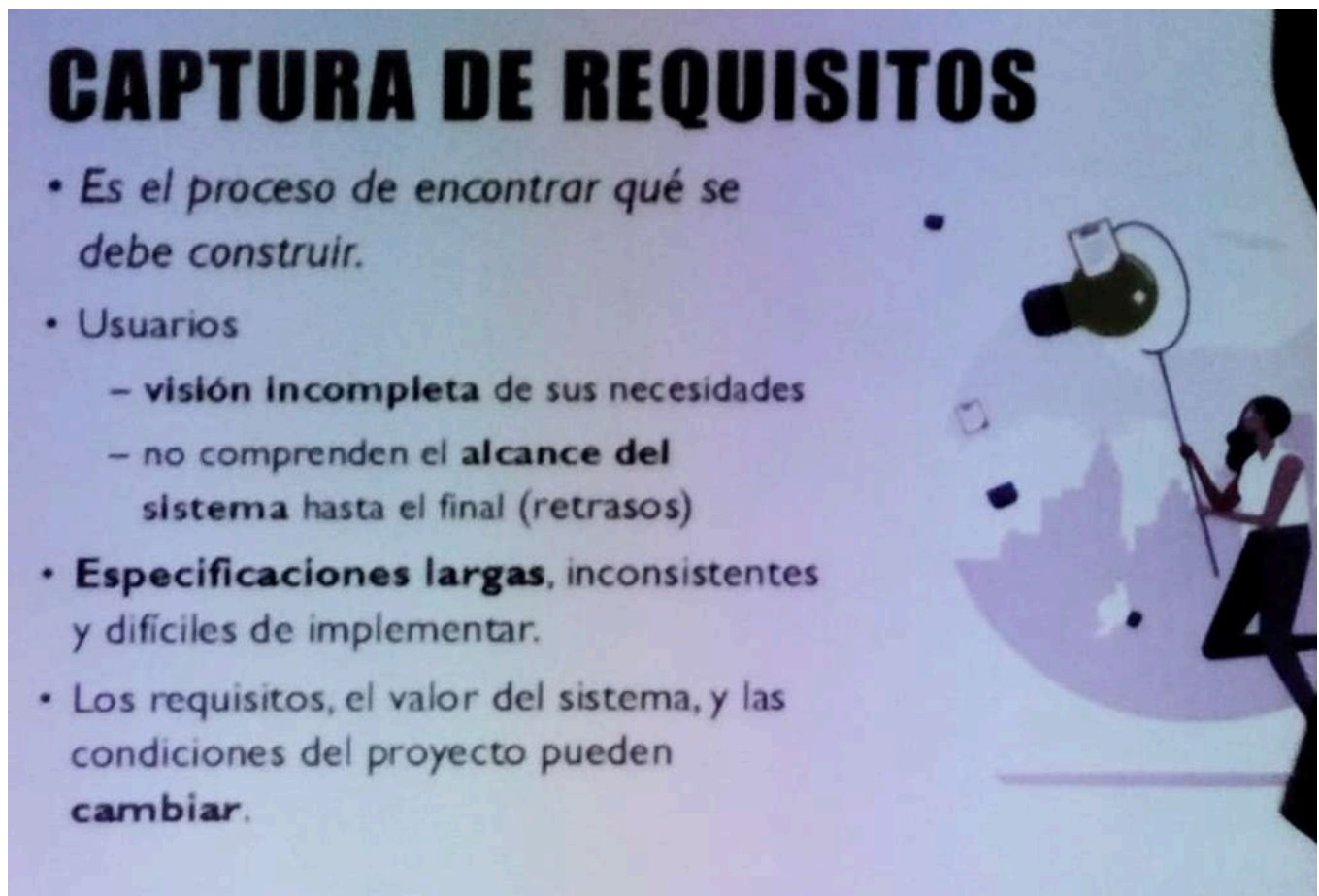
- en el PU los Componentes son lo que veíamos en UML como Artefacto

- vamos a ver como transformar requisitos en un sw ejecutable

- el PU es un proceso de desarrollo para sistemas complejos y un desarrollo más formal

- no se puede considerar PU a algo que no usa CU

CAPTURA DE REQUISITOS



CAPTURA DE REQUISITOS

- *Es el proceso de encontrar qué se debe construir.*
- Usuarios
 - **visión incompleta** de sus necesidades
 - no comprenden el **alcance del sistema** hasta el final (retrasos)
- **Especificaciones largas**, inconsistentes y difíciles de implementar.
- Los requisitos, el valor del sistema, y las condiciones del proyecto pueden **cambiar**.

The slide features an illustration on the right side showing a person in a white shirt and dark pants using a long-handled magnifying glass to inspect a globe. Several small, floating icons representing documents or data points are scattered around the globe.

-vamos agregar un paso previo que no está muy formalizado en el PU, se puede poner como sinónimo a lo que es **Modelado de negocio**

-no forma parte como tal del PU, pero está ahí

-tratar de ver cuales son los requisitos que tenemos que desarrollar en nuestro proceso

2) POR QUÉ SE CONSIDERA DIFÍCIL HACER LA CAPTURA DE REQUISITOS

(lo que aparece abajo son las dificultades que nos podemos encontrar)

-el desarrollo de sw es una disciplina socio-técnica, nosotros tenemos que saber lidiar con las personas. Y en que parte, que sería en la parte de contacto con los clientes

-en sistemas muy grandes los usuarios no reconocen todo lo que quieren

-al tener una visión incompleta tampoco tienen un alcance global

-cuando tenemos sistemas complejos, las especificaciones de los requisitos tienden a ser más largos

-tenemos que saber que los requisitos van a cambiar y cómo lidiar con eso

-es la parte crucial, de sacar lo que el usuario quiere de manera correcta

-la listas muy grandes de requisitos si no están bien organizadas pueden ser difíciles de leer

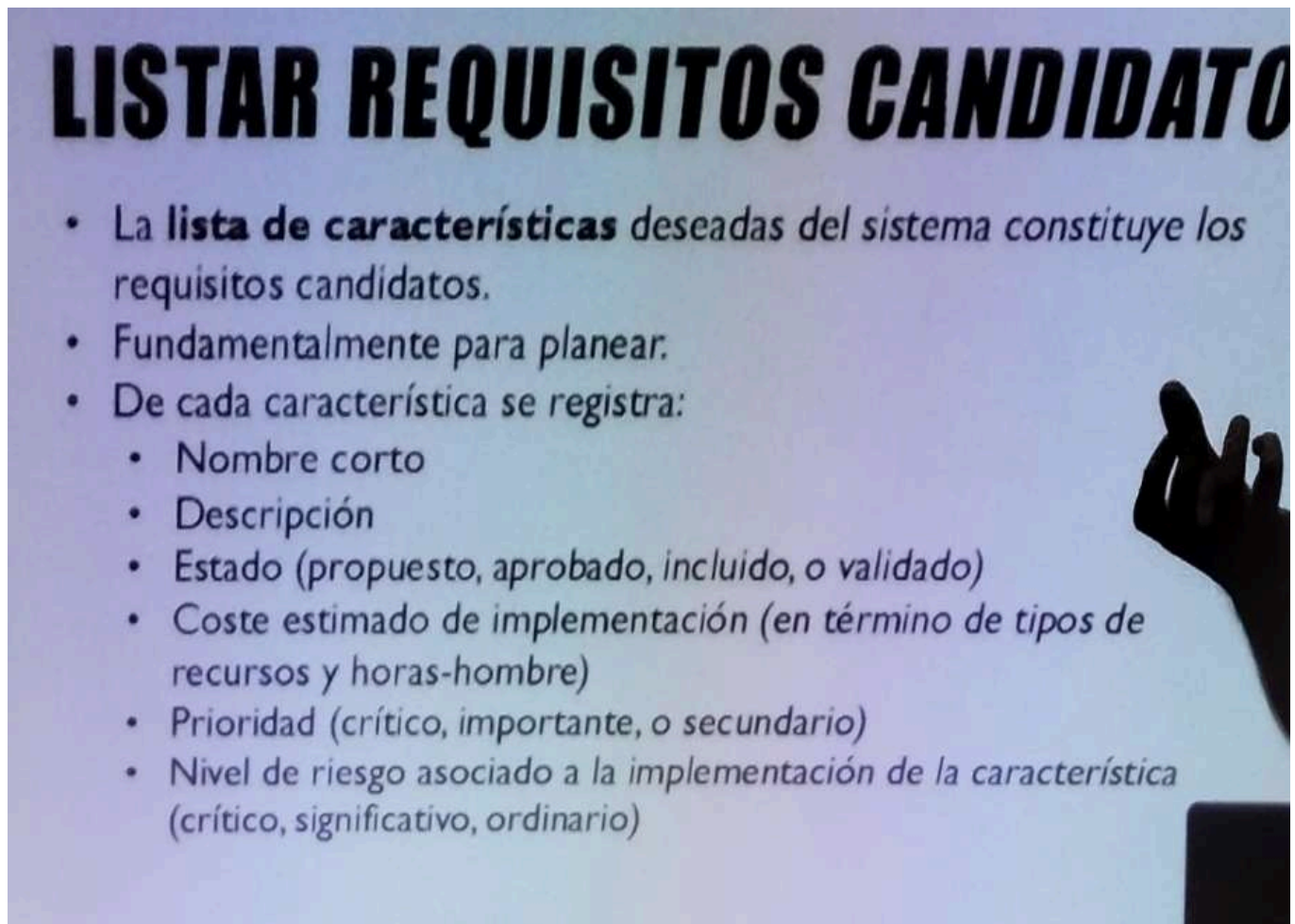
(otro power)

CAPTURA DE REQUISITOS

- **Flujo de trabajo:**
 - **Listar los requisitos candidatos**
Lista de Características.
 - **Comprender el contexto del sistema**
Modelo del Dominio / Negocio.
 - **Capturar requisitos funcionales**
Modelo de Casos de Uso.
 - **Capturar requisitos no funcionales**
Lista Especificaciones adicionales.

-la mejor forma es tratar de capturar eso requisitos como CU

Listar requisitos candidatos



LISTAR REQUISITOS CANDIDATO

- La **lista de características** deseadas del sistema constituye los requisitos candidatos.
- Fundamentalmente para planear.
- De cada característica se registra:
 - Nombre corto
 - Descripción
 - Estado (propuesto, aprobado, incluido, o validado)
 - Coste estimado de implementación (en término de tipos de recursos y horas-hombre)
 - Prioridad (crítico, importante, o secundario)
 - Nivel de riesgo asociado a la implementación de la característica (crítico, significativo, ordinario)

-aca tenemos una plantilla básica de como ir registrando cada característica, la idea es que sea bien resumida

-la lista es más resumida que los CU

(otro power)

Comprender contexto del sistema

Comprender el contexto del sistema Muchas de las personas implicadas en el desarrollo de software son especialistas en temas relativos al software. Sin embargo, para capturar los requisitos correctos y para construir el sistema correcto, los desarrolladores clave —en particular el arquitecto y algunos de los analistas senior— requieren un firme conocimiento del contexto en el que se emplaza el sistema.

Hay por lo menos dos aproximaciones para expresar el contexto de un sistema en una forma utilizable para desarrolladores de software: **modelado del dominio y modelado del negocio**. Un

COMPRENDER CONTEXTO DEL SISTEMA

- **Modelo del Dominio**
- comprender, describir y capturar las **clases más importantes** de objetos en el contexto del sistema.
- Tres formas típicas:
 - **Objetos del negocio** que representan cosas que se manipulan en el negocio, como pedidos, cuentas, contratos, etc.
 - **Objetos del mundo real y conceptos** de los que el sistema debe hacer seguimiento como aviación enemiga, misiles, trayectorias, etc.
 - **Sucesos** que ocurrirán o han ocurrido, como llegada de un avión, su salida, hora de la comida, etc.
- El MD se representa por diagramas de clases.
- Permite usar un vocabulario común

-El modelo de dominio es el que describe cuales son las clases conceptuales que nuestro dominio va a estar manejando, que generalmente va a ser contemplado por nuestro sistema. Puede pasar aunque es raro, que alguno no se contemple

-ej: auto, venta, caja de ahorros,

-es el fundamental que hablamos en donde va a funcionar el sistema

-el modelo de dominio por defecto se habla de la situación actual, previa a la informatización de ese dominio

-puede ser que que tengamos que modelar cómo quedaría ese sistema, porque hay veces que no tendríamos de otra

COMPRENDER CONTEXTO DEL SISTEMA

Modelado del Negocio

- técnica para comprender los **procesos de negocio** de la organización.
- soportado por dos tipos de modelos:
 - casos de uso de negocio y
 - objetos de negocio
- Un **Modelo de Casos de Uso del Negocio**
 - describe los **procesos de negocio** de una empresa en términos de
 - **casos de uso del negocio y actores del negocio**
- Cada realización de un caso de uso del negocio puede mostrarse en diagramas de interacción y diagramas de actividad.

COMPRENDER CONTEXTO DEL SISTEMA

Modelo del Negocio

- Una **entidad** del negocio representa algo que los trabajadores toman, manipulan, inspeccionan, producen o utilizan en un negocio.
- Una **unidad de trabajo** es un conjunto de esas entidades que conforma un todo reconocible para el usuario final.
- La técnica de modelado de negocio identifica **entidades y trabajadores** que participan en la realización de los casos de uso del negocio.
- Los trabajadores identificados en el modelo de negocio se utilizan como punto de partida para derivar un primer conjunto de **actores y casos de uso** del sistema.

- a veces no es necesario hacer este modelado, porque lleva mucho tiempo
- consiste en modelar el modelo de negocio y las entidades que se va a manejar
- CU de negocio y CU de sistema. La diferencia es que el de negocio modela procesos del mismo
- conjunto de entidades vinculadas
- Actor de negocio externo al negocio, se suele confundir con el trabajador
- Actor de sistema externo al sistema

El arquitecto y el jefe de proyecto juntos deciden si se prepara un modelo del dominio, si se recorre todo el camino y se prepara un modelo del negocio entero, o si no se hace ninguno de esos modelos.

FUNCIONALES - CAPTURAR REQUISITOS

CAPTURAR REQUISITOS FUNCIONALES

- Identificar requisitos de sistema a través de casos de uso
- Capturar los requisitos funcionales y no funcionales que son específicos de los casos de uso individuales

- si no aclara que son de negocio, normalmente son de sistema
- los CU de sistema estan pensado para capturar requisitos funcionales
- no es una herramienta tan útil para capturar requisitos no funcionales. Además pueden servir como puntos de concentración o agrupamientos

(otro power)

CAPTURAR REQUISITOS FUNCIONALES

Búsqueda de Casos de Uso a partir de un modelo del negocio

- Se identifica un actor por cada trabajador y por cada actor del negocio que vaya a ser usuario del sistema.
- Por cada rol de un trabajador / actor del negocio, asignamos un caso de uso candidato.

-te indica como derivado los CUS de los CUN

-la especificación de CU de que hace se realiza con el Diagrama de actividades

-los actores del sistema pueden venir de trabajadores de negocio y actores de negocio

NO FUNCIONALES - CAPTURA REQUISITOS

CAPTURAR REQUISITOS NO FUNC.

- Los requisitos no funcionales especifican las propiedades del sistema, como:
 - las restricciones ambientales y de implementación,
 - el rendimiento,
 - las dependencias de la plataforma,
 - la capacidad de mantenimiento,
 - la extensibilidad y
 - la confiabilidad
- Algunos requisitos no funcionales son más genéricos y no se pueden conectar a un caso de uso particular o a una clase particular del mundo real. **Lista de requisitos suplementarios**

-en la lista de requisitos suplementarios van la lista de requisitos no funcionales que no son específicos de un CU, sino que se aplican de forma general

Trabajo a realizar	Artefactos resultantes	
Enumerar requisitos candidatos	Lista de características	
Comprender el contexto del sistema	Modelo del dominio o del negocio	
Capturar los requisitos funcionales	Modelo de casos de uso	} Definen una especificación de requisitos tradicional
Capturar los requisitos no funcionales	Requisitos adicionales o casos de uso concretos (para requisitos específicos de un caso de uso)	

6.7. Requisitos adicionales

Los requisitos adicionales son fundamentalmente requisitos no funcionales que no pueden asociarse a ningún caso de uso en concreto —en cambio cada uno de estos requisitos tiene impacto en varios casos de uso o en ninguno—. Algunos ejemplos son el rendimiento, las interfaces y los requisitos de diseño físico, así como las restricciones arquitectónicas, de diseño y de implementación [1]. Los requisitos adicionales se capturan de forma muy parecida a como se hacía en la especificación de requisitos tradicional, es decir, como una lista de requisitos. Después se utilizan durante el análisis y el diseño junto al modelo de casos de uso.

Un *requisito de interfaz* especifica la interfaz con un elemento externo con el cual debe interactuar el sistema, o que establece restricciones condicionantes en formatos, tiempos, u otros factores de relevancia en esa interacción.

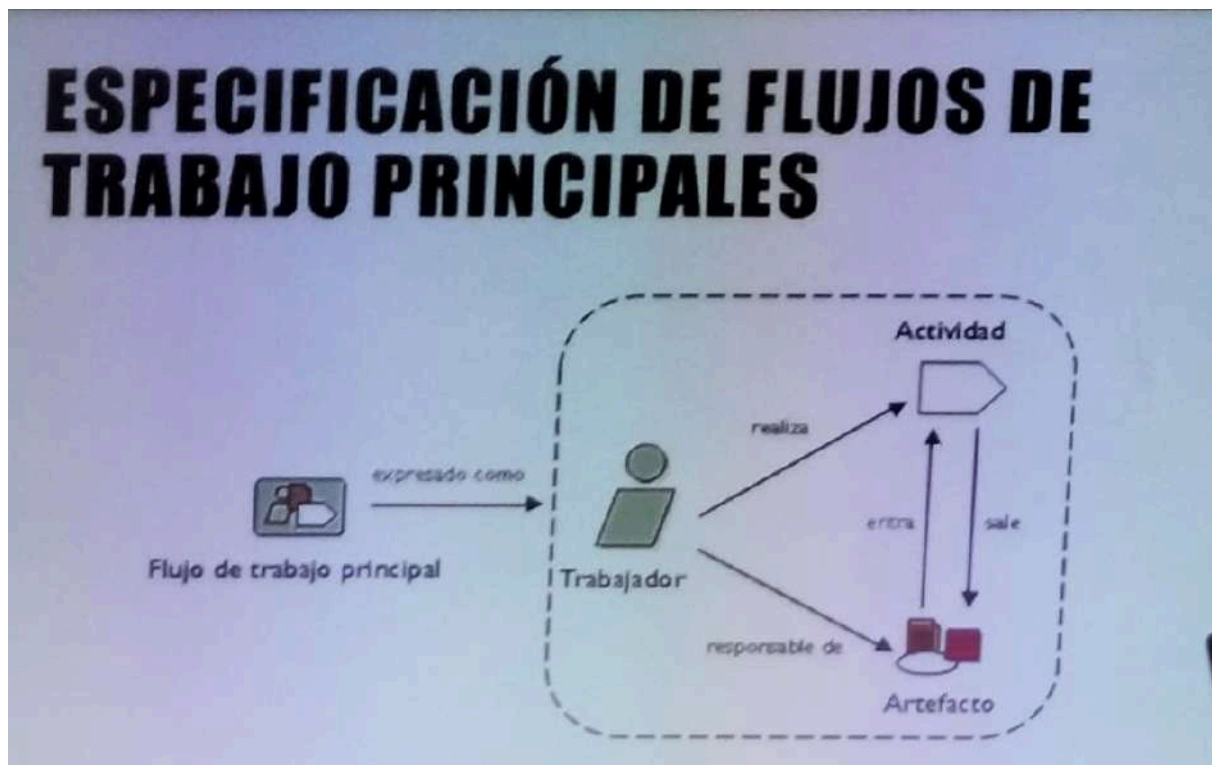
Un *requisito físico* especifica una característica física que debe poseer un sistema, como su material, forma, tamaño o peso. Por ejemplo, puede utilizarse este tipo de requisito para representar requisitos hardware como las configuraciones físicas de red requeridas.

Una *restricción de diseño* limita el diseño de un sistema, como lo hacen las restricciones de extensibilidad y mantenibilidad, o las restricciones relativas a la reutilización de sistemas heredados o partes esenciales de los mismos.

Una *restricción de implementación* especifica o limita la codificación o construcción de un sistema. Son ejemplos los estándares requeridos, las normas de codificación, los lenguajes de programación, políticas para la integridad de la base de datos, limitaciones de recurso, y entornos operativos.

Además, existen *otros requisitos*, como los requisitos legales y las normativas.

ESPECIFICACIÓN DE FLUJOS DE TRABAJOS PRINCIPALES



-este es una forma genérica de escribir cada flujo de trabajo

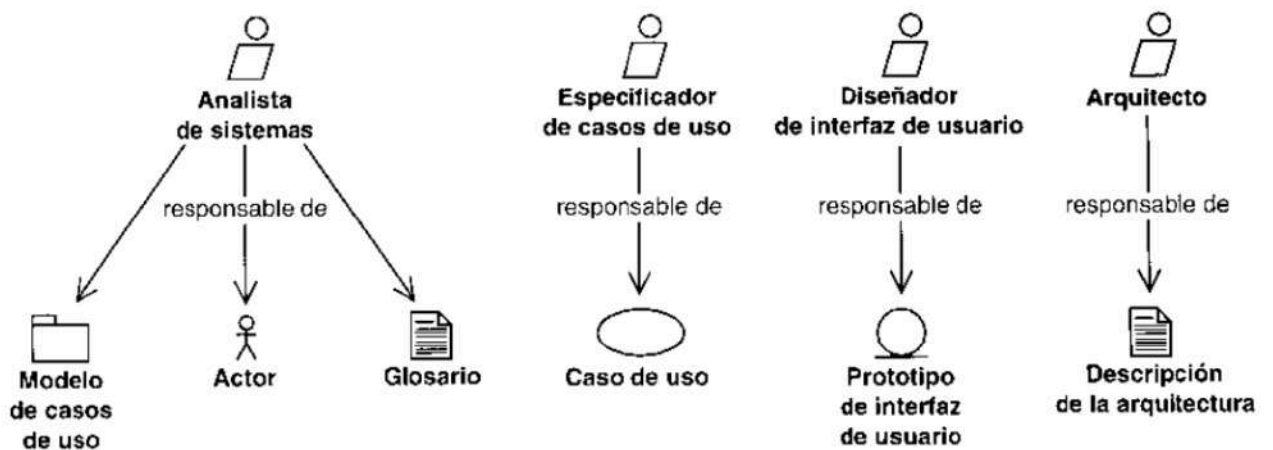
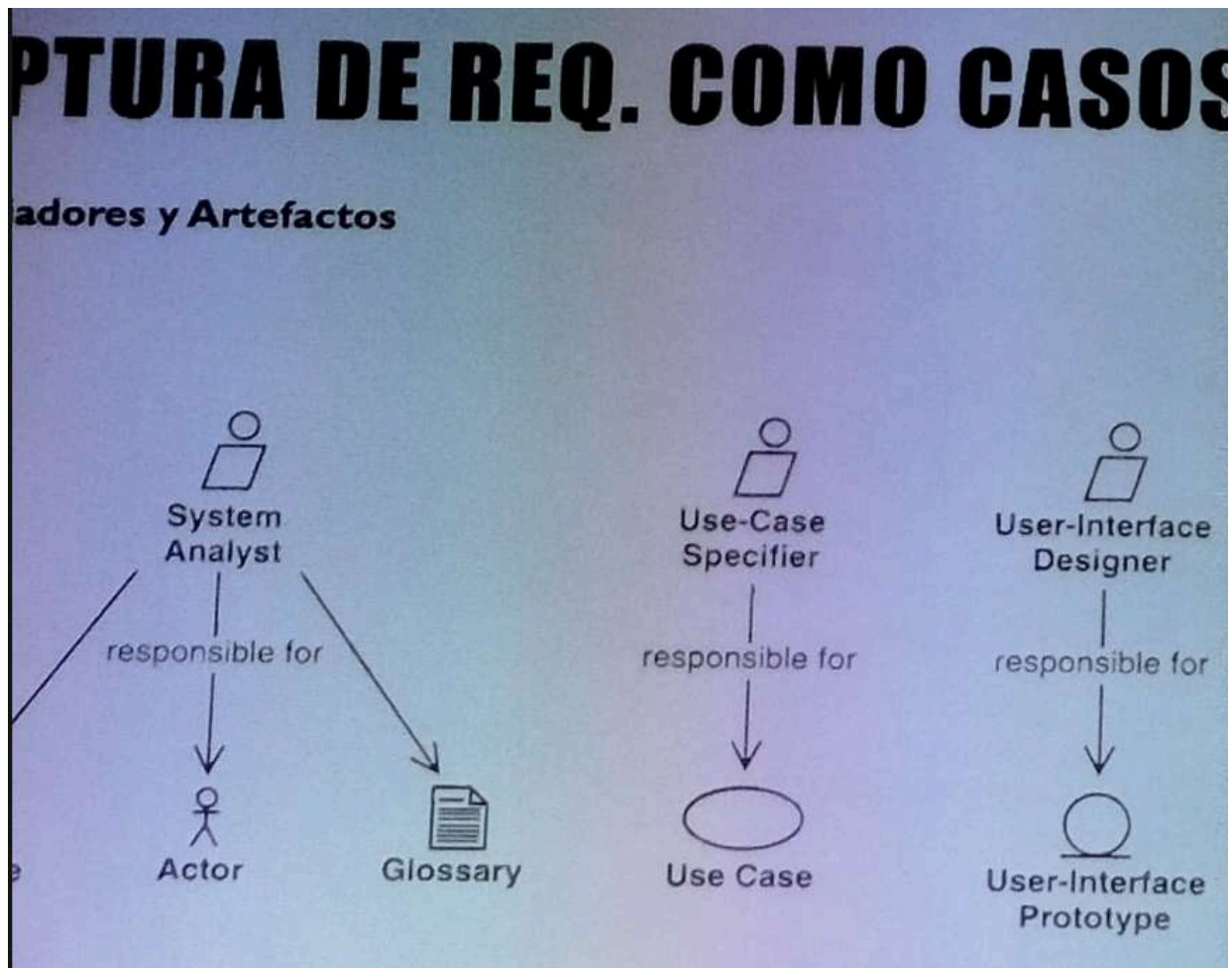
-vamos a tener tipos de trabajadores como arquitecto, analista de sistemas o especificador de casos de uso

-El flujo de trabajo donde los casos de uso son más maduros va a ser el de construcción o cercano a él ya que van a estar más depurados/trabajados

CAPTURA DE RE. COMO CASOS DE USO

- a través de CU me concentro en buscar las funcionalidades que aportan mayor valor para el usuario
- empezar por los actores

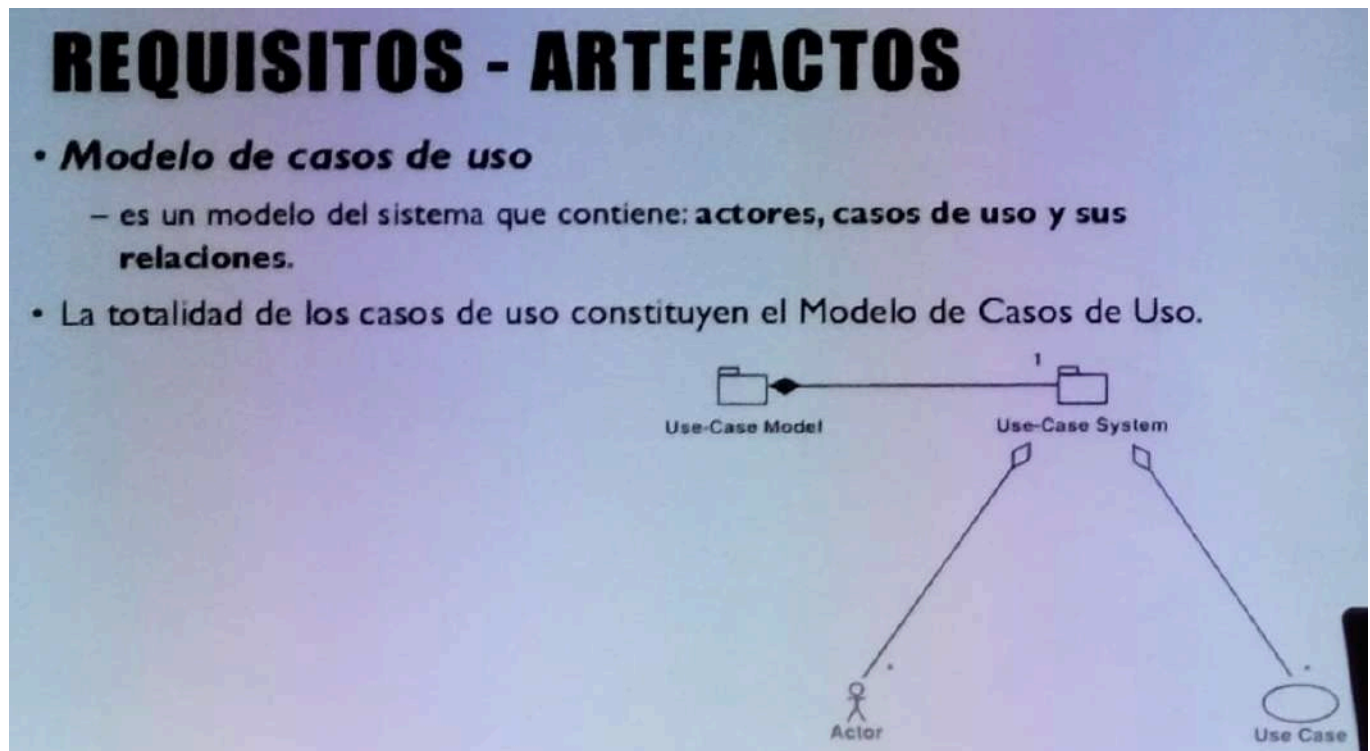
Diagrama - Trabajadores y Artefactos



- tipos de trabajadores que vamos a tener en la etapa de requisitos
- el arquitecto está presente en todos lados (hay otros que su participación es solo lo que indica la diapositiva)
- pueden instanciarse en un trabajador específico cuando ocupemos este proceso (Proyecto es la instancia de un Proceso)

REQUISITOS

ARTEFACTOS



- en la fase de inicio va a estar mucho más verde cuando estemos terminando la etapa de elaboración
- el modelo de CU esta compuesto por el CUS, esto es una formalidad del proceso unificado

(otro power)

REQUISITOS - ARTEFACTOS

- **Actor:** representa un tipo de usuario externo al sistema. Se corresponden con trabajadores o actores del negocio.
- **Caso de Uso (CU):** son fragmentos de funcionalidad que el sistema ofrece para aportar un resultado de valor para sus actores.
- **Descripción de la arquitectura (vista del modelo de casos de uso):** representa los CU significativos para la arquitectura. CU críticos.
- **Glosario:** para definir términos comunes importantes que los analistas utilizan para describir el sistema.
- **Prototipo de Interfaz de usuario:** ayudan a comprender y especificar las interacciones entre actores humanos y el sistema durante la captura de requisitos.

-son tan artefactos como lo puede ser un ejecutable, son todos productos/partes que forman parte de todo el sistema que vamos a estar desarrollando

-Vista del modelo de CU de la descripción de la arquitectura, va a ser lo más **importante**

-forma parte de la disciplina, los prototipos de interfaces. Diseño entrada/salida.

-recuerden que los prototipos pueden ser dibujado a mano alzada o productos navegables, dependiendo de que se quiere hacer

Sin embargo, reconocemos que ciertamente existen temas de interferencia entre los diferentes usos de un sistema. Estos temas no pueden resolverse en el modelado de casos de uso sino que se posponen al análisis y al diseño (descritos respectivamente en los Capítulos 8 y 9),

7.2.3.1. Flujo de sucesos El flujo de sucesos para cada caso de uso puede plasmarse como una descripción textual de la secuencia de acciones del caso de uso. Por tanto, el flujo de sucesos especifica lo que el sistema hace cuando se lleva a cabo el caso de uso especificado. El flujo de sucesos también especifica cómo interactúa el sistema con los actores cuando se lleva a cabo el caso de uso.

7.2.3.2. Requisitos especiales Llamamos requisitos especiales a una descripción textual que agrupa todos los requisitos del tipo de los requisitos no funcionales sobre el caso de uso. Son fundamentalmente requisitos no funcionales relacionados con el caso de uso y que deben tratarse en flujos de trabajo posteriores como análisis, diseño o implementación.

TRABAJADOR

REQUISITOS - TRABAJADORES

- **Analista de Sistemas**

Responsable de:

- Modelo de casos de uso
- Actores que contiene
- Glosario

- **Especificador de Casos de Uso**

Asisten al analista de sistema en la descripción detallada de cada caso de uso.

Responsable de:

- Caso de uso

- **Diseñador de Interfaz de Usuario**

Responsable de:

- Prototipo de interfaz de usuario

- **Arquitecto**

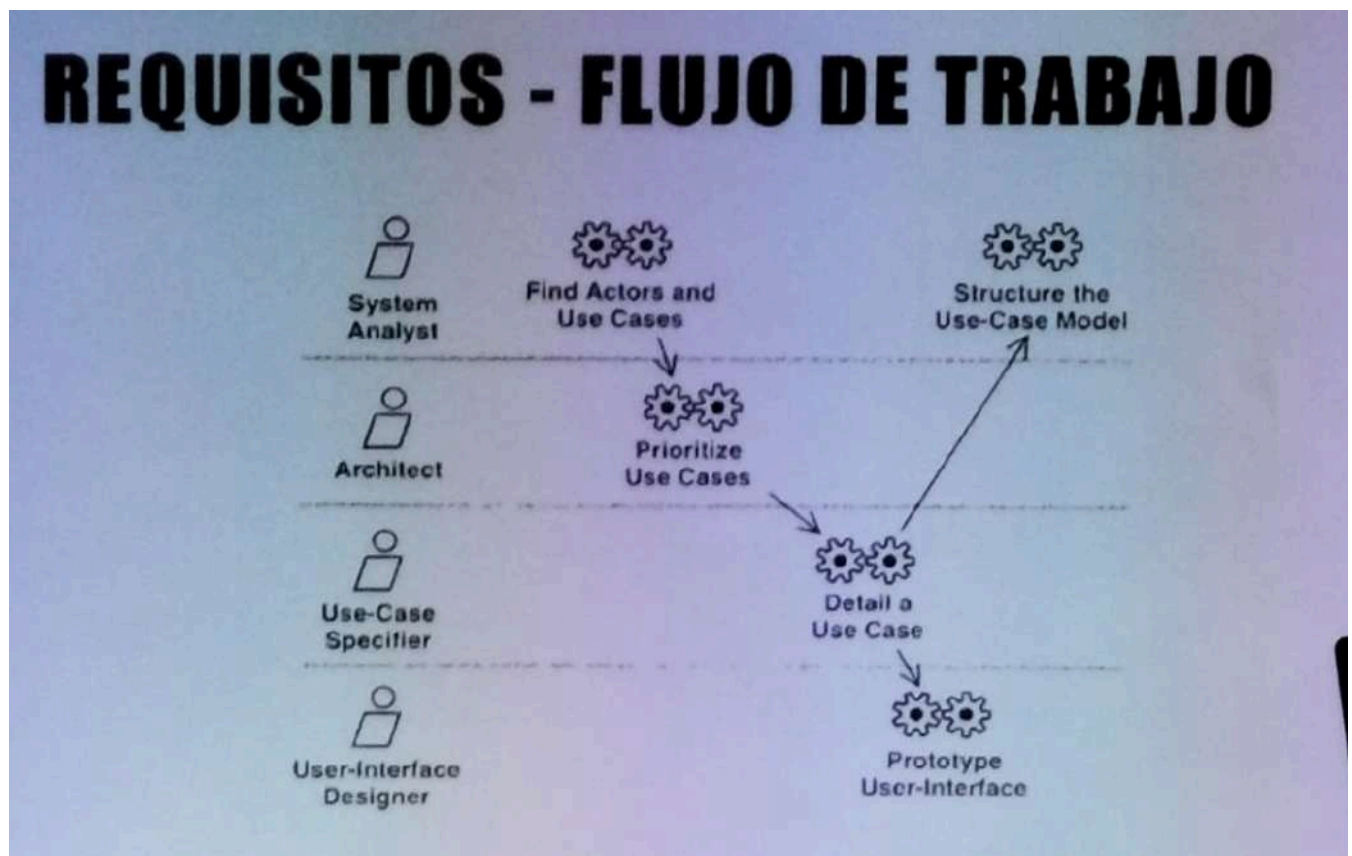
Responsable de:

- Descripción de la arquitectura (vista del modelo de casos de uso)

-concentrador de responsabilidades

Aunque el analista de sistemas es responsable del modelo de casos de uso y de los actores que contiene, no es responsable de cada caso de uso en particular. Esto es una responsabilidad aparte, que pertenece al trabajador especificador de casos de uso (Sección 7.3.2). El analista de sistemas es también el que dirige el modelado y el que coordina la captura de requisitos.

FLUJO DE TRABAJO - REQUISITOS

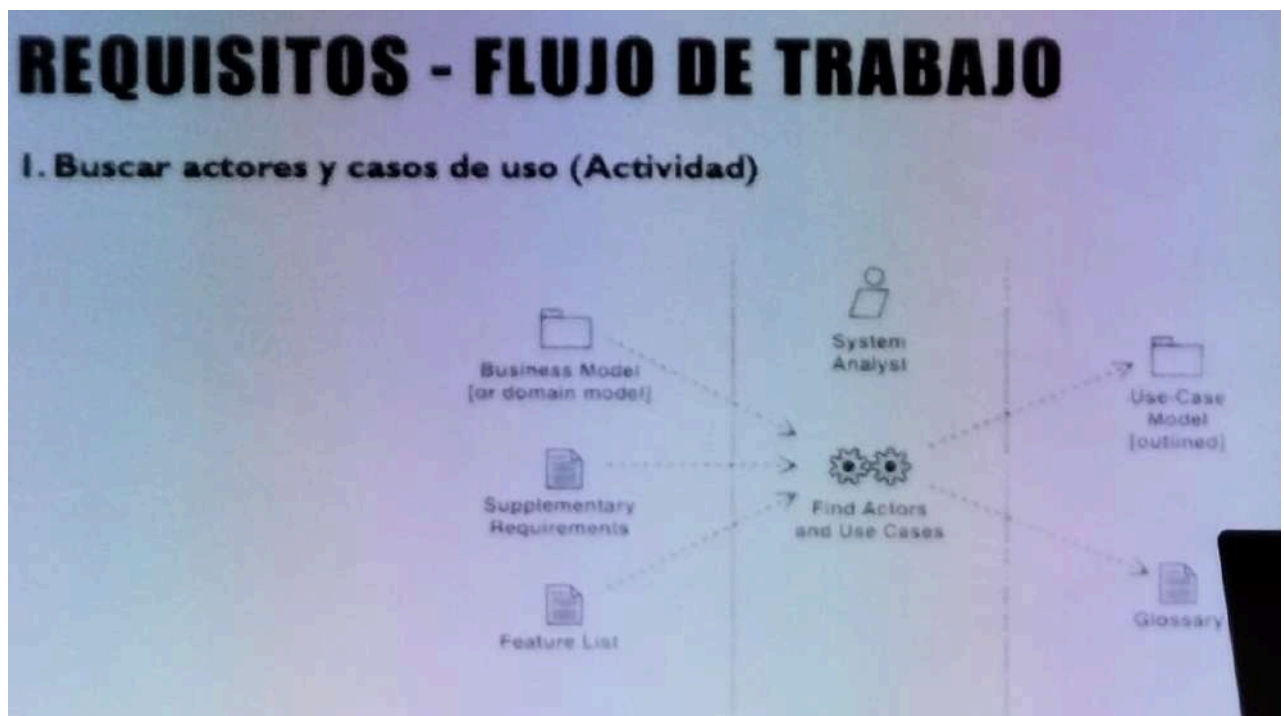


-el arquitecto es importante que prioriza los CU para armar la arquitectura

Los resultados de la primera iteración a través de este flujo de trabajo consisten en una primera versión del modelo de casos de uso, los casos de uso y cualquier prototipo de interfaz de usuario asociado. Los resultados de cualquier iteración subsiguiente consistirán entonces en nuevas versiones de estos artefactos. Hay que recordar que todos los artefactos se completan y mejoran incrementalmente a través de las iteraciones.

Las distintas actividades en el modelado de casos de uso adoptan formas diferentes en diferentes fases del proyecto (véase la Sección 6.4). Por ejemplo, cuando un analista de sistemas

1. Buscar actores y CU



-como resultado sale el glosario que nos va a servir para la parte de análisis

Esta actividad consta de cuatro pasos:

- Encontrar los actores.
- Encontrar los casos de uso.
- Describir brevemente cada caso de uso.
- Describir el modelo de casos de uso completo (este paso también incluye la preparación de un glosario de términos).

Estos pasos no tienen por qué ser ejecutados en ningún orden en particular y a menudo se hacen simultáneamente. Por ejemplo, el diagrama de casos de uso puede actualizarse tan

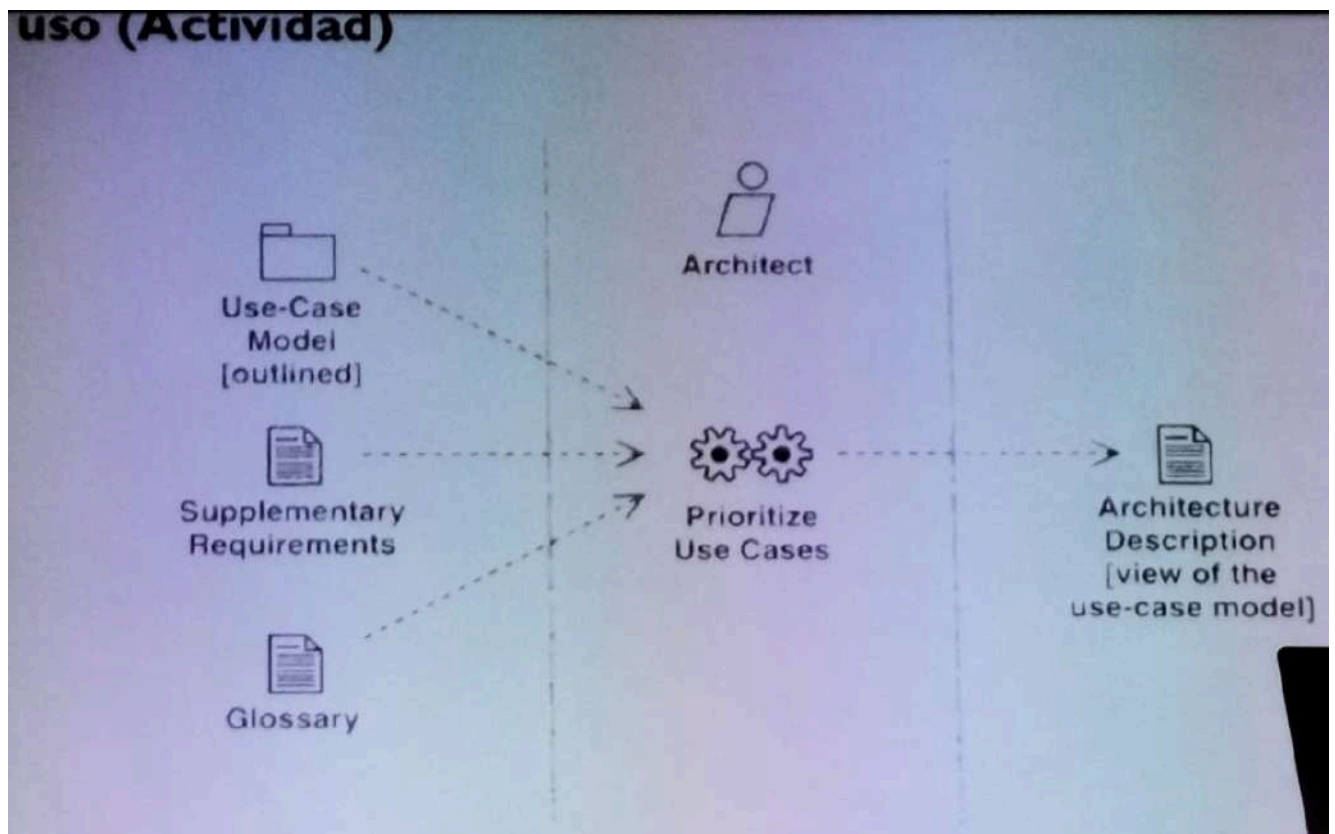
No hay que tener más de un actor con los mismos roles.

Soluciones: combinar o generalizar

Cuando la descripción del modelo de casos de uso esté preparada, dejamos que dé el visto bueno del modelo de casos de uso la gente que no forma parte del equipo de desarrollo (es decir, clientes y usuarios), convocando una revisión informal para determinar si:

- Se han capturado como casos de uso todos los requisitos funcionales necesarios.
- La secuencia de acciones es correcta, completa y comprensible para cada caso de uso.
- Se identifica algún caso de uso que no proporcione valor. Si es así, ese caso de uso debería reconsiderarse.

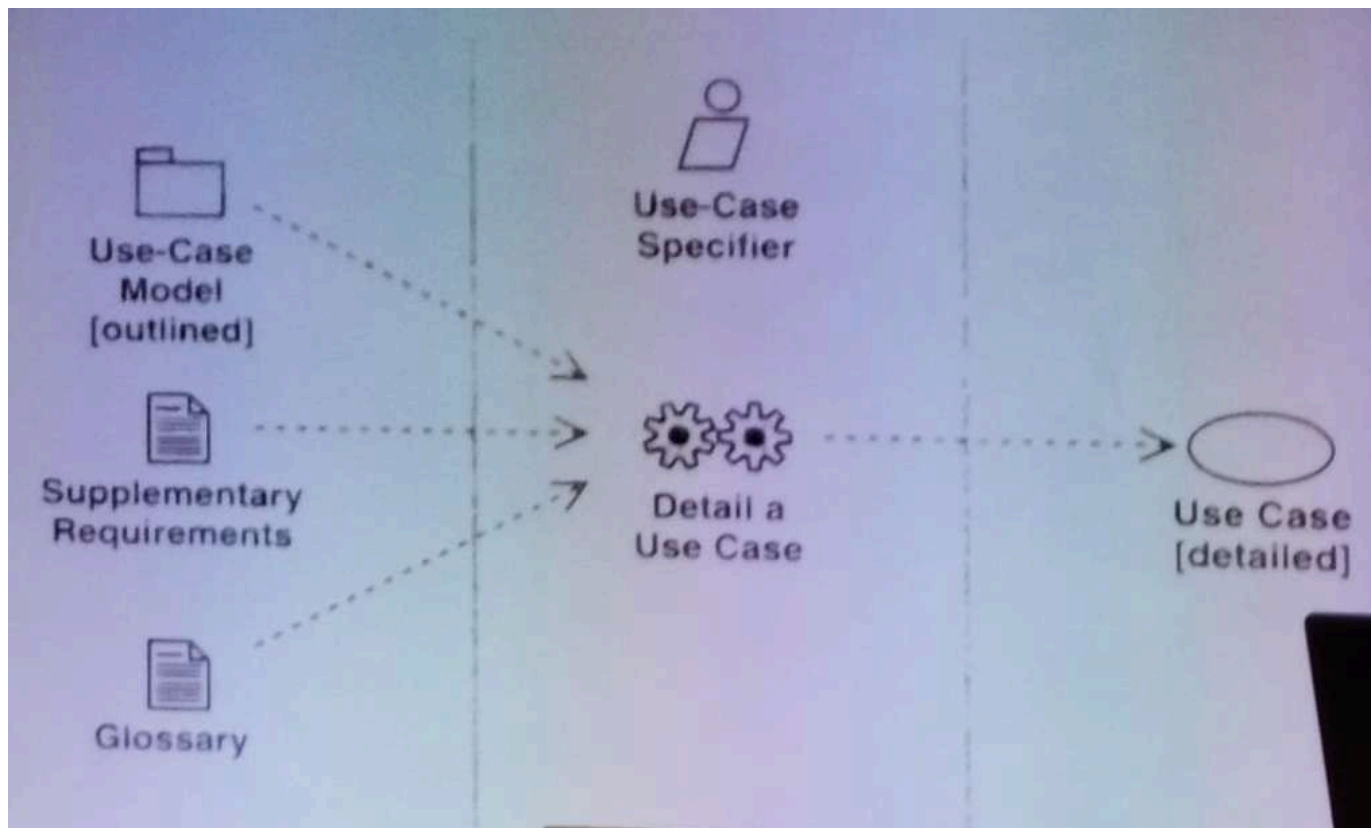
2. Priorizar CU



- recibe los artefactos: modelo de CU y glosario
- toma los requisitos no funcionales (requisitos suplementarios)
- se buscan los CU más significativos arquitecturalmente, se les va a dar prioridad a la hora de desarrollar
- va a salir la descripción de la arquitectura de la vista del modelo de CU

(Tiene como entrada los 2 de salida del anterior y se mantienen los requerimientos suplementarios)

3. Detallar CU



- en esencia se trata de describir cada CU
- es un trabajador que puede ser llevado a cabo por uno o más personas, puede ser que una persona tenga especificada un CU
- lo más importante de los CU son sus **descripciones textuales**
- sale el CU detallado
- salen los CU que aparecieron en esta iteración, son varios pero no necesariamente todos

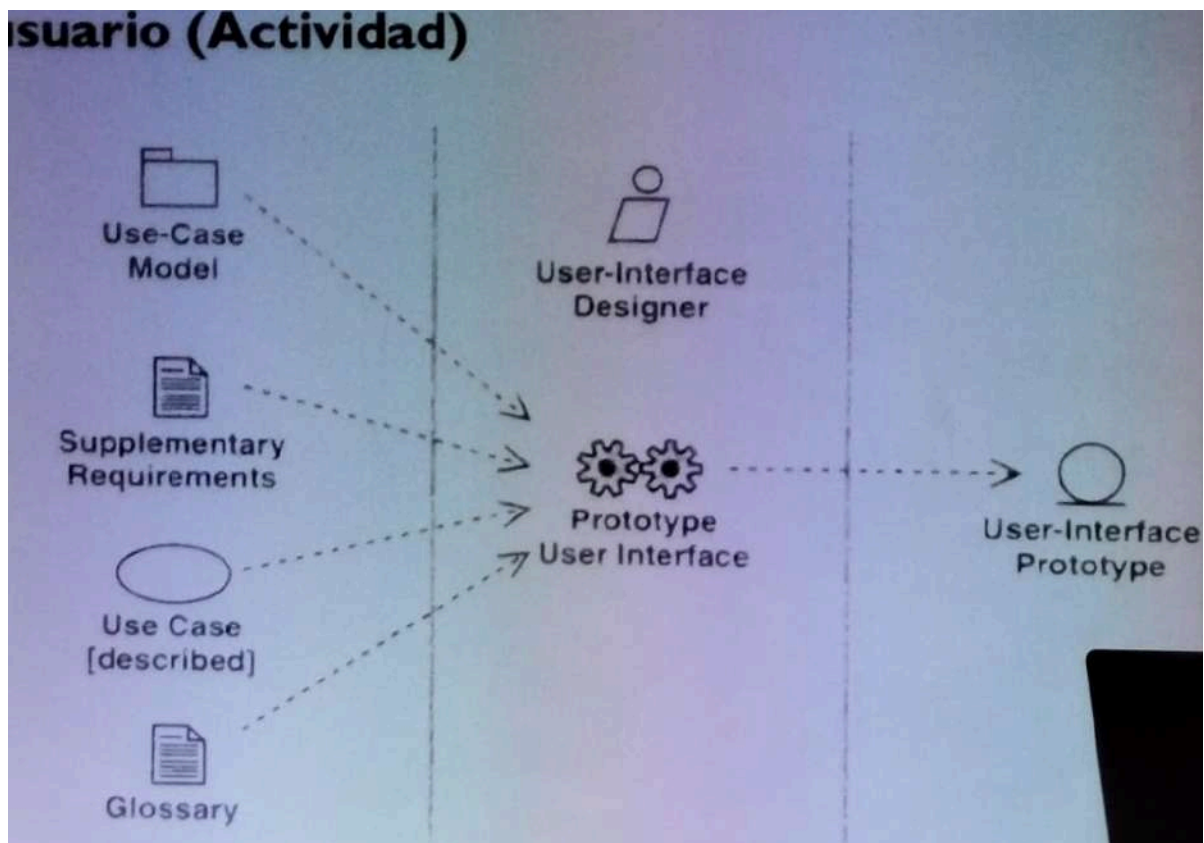
(Tiene como entrada los mismos artefactos que la anterior actividad “Priorizar CU”)

tilo utilizado en el ejemplo de Pagar Factura. Aun así, es una buena idea tener una máquina de estados en nuestras mentes cuando describimos un caso de uso, para asegurarnos de que cubre todas las posibles situaciones. No obstante, en algunas ocasiones, como cuando

Cuando el CU es tan complicado que la descripción textual se vuelve poco entendible es necesario formalizarla con alguno, o varios, de estos diagramas

- Pueden utilizarse los diagramas de estado
- Pueden utilizarse los diagramas de actividad
- Se pueden utilizar los diagramas de interacción

4. Prototipar UI



- me sirve para validar los CU que tengo hasta el momento, ver si se entendieron bien, y pueden aparecer nuevos
- la calidad de los interfaces de los prototipos depende de que tan importante es para nosotros la usabilidad
- si nosotros queremos probar cosas en profundidad, necesitaremos un prototipo navegable. Que claramente lleva más tiempo y costo

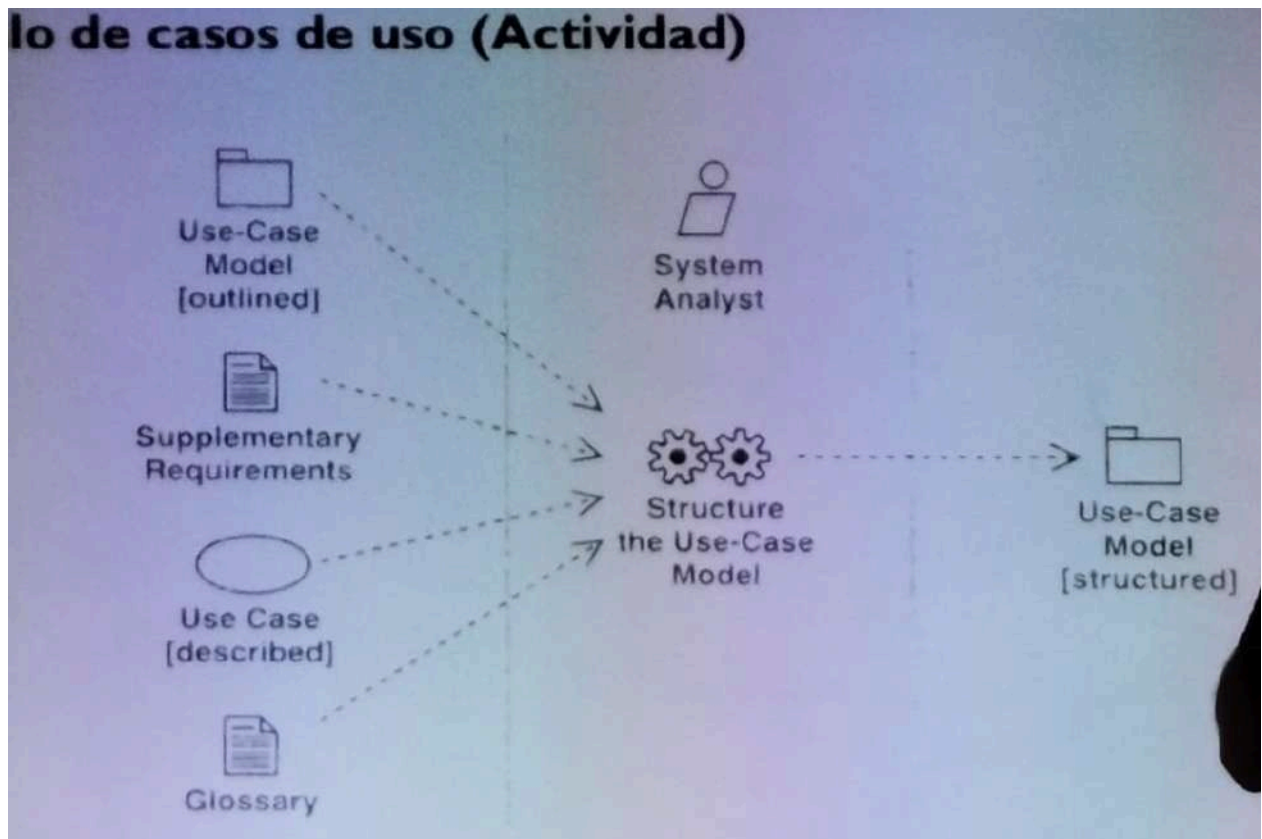
(Tiene como entrada los mismos artefactos que la anterior actividad “Detallar CU”. Pero se agrega la salida del anterior y el modelo de CU no importa si es esbozado o estructurado, utiliza el que esté en el momento)

El resultado final de esta actividad es un conjunto de esquemas de interfaces de usuario y prototipos de interfaces de usuario que especifican la apariencia de esas interfaces de cara a los actores más importantes.

Saber que los prototipos de las UI sirven principalmente para ayudar al usuario a comprender visualmente lo que se está desarrollando, así adquirir una retroalimentación mucho más valiosa

De esta forma, los diseñadores de interfaz de usuario aseguran que cada caso de uso es accesible a través de sus elementos de las interfaces de usuario. No obstante, también deben asegurarse de que el conjunto de casos de uso accesibles para el actor tiene una interfaz de usuario bien integrada, fácil de utilizar y consistente.

5. Estructura el modelo CU



- análisis de sistema es el que tiene la visión completa de todos los CU, los especificadores solamente ve el que le toca
- los diagramas de CU deben ser entendibles por los clientes
- una opción podría ser empaquetar, pero va más allá de eso.
- esto conlleva agregar include, extend, etc
- existen caminos comunes entre CU
- sale el modelo estructurado, terminado pero dependiendo (hay que tener en cuenta el factor tiempo)

El modelo de casos de uso se estructura para:

- Extraer descripciones de funcionalidad (de casos de uso) generales y compartidas que pueden ser utilizadas por descripciones (de casos de uso) más específicas.
- Extraer descripciones de funcionalidad (de casos de uso) adicionales u opcionales que pueden extender descripciones (de casos de uso) más específicas.

La generalización se emplea para simplificar la forma de trabajo y la comprensión del modelo de casos de uso y para reutilizar casos de uso “semifabricados” cuando reunimos casos de uso terminados requeridos por el usuario. Cada caso de uso terminado se denomina *caso de uso concreto*. Los inicia un actor, y sus instancias constituyen una secuencia de acciones completa ejecutada por el sistema. El caso de uso “semifabricado” existe solamente para que otros casos de uso lo reutilicen y se les llama *casos de uso abstractos*. Un caso de uso abstracto no puede instanciarse por sí mismo, pero una instancia de un caso de uso concreto también exhibe el comportamiento especificado por un caso de uso abstracto que lo (re)utiliza. Para entendernos, llamamos a esta instancia el caso de uso “real” que el actor(es) percibe cuando interactúa con el sistema.

Los casos de uso reales son aquellos que aportan valor a los usuarios, así, el criterio para obtener buenos casos de uso que se mencionó en la Sección 7.4.1, “Encontrar actores y casos de uso” (un caso de uso obtiene valores de resultados observables para un actor en particular) es relevante solamente para casos de uso reales. Así que debemos identificar criterios separados para realizar buenos casos de uso concretos, buenos casos de uso abstractos y buenas extensiones de casos de uso. Los casos de uso concretos no deben describir comportamientos (significativos) que son compartidos con otros casos de uso concretos. Un caso de uso abstracto hace más fácil de especificar los casos de uso concretos ofreciendo comportamiento reutilizable y compartido. Una extensión de un caso de uso especifica comportamiento adicional para otros casos de uso, independientemente de si su comportamiento es opcional u obligatorio.

PRÁCTICA

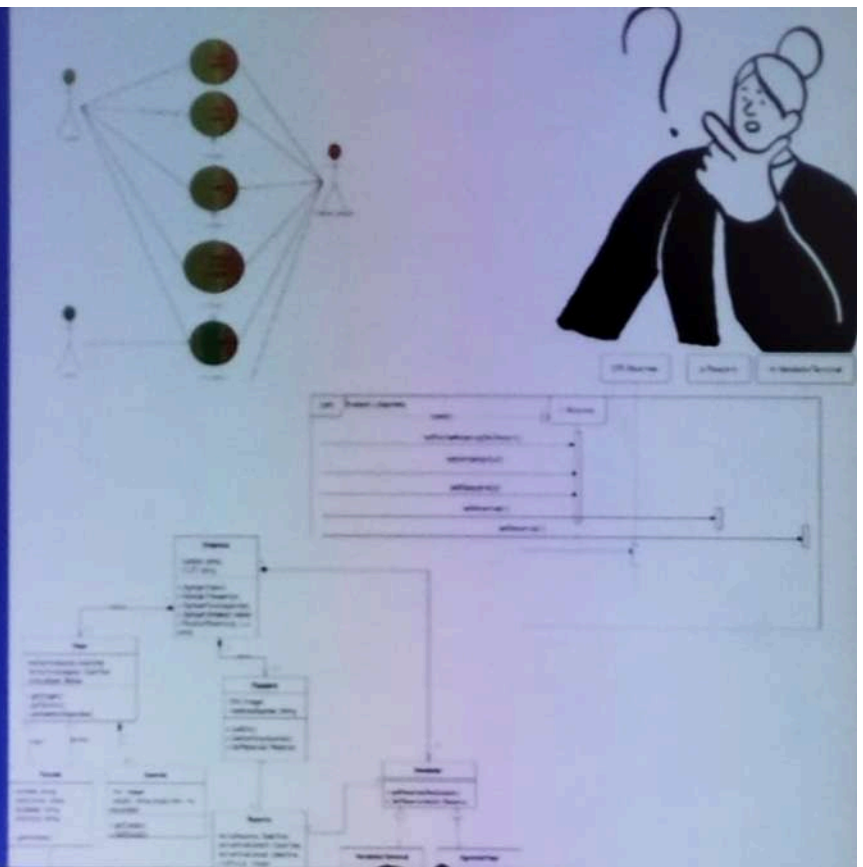
REALIZACIÓN DE CU

Diseño de Sistemas de Información

¿QUÉ ES UNA REALIZACIÓN DE CU?

Describe cómo se realiza un
CU particular en el modelo de
diseño, en función de los
OBJETOS QUE COLABORAN

Sirve para dejar
explícitamente expresado la
**relación entre los requisitos
enunciados en los CU y el
diseño de objetos que
satisface los requisitos**



REALIZACIÓN DE CU

2 Aspectos

ESTÁTICO

- Clases QUE INTERVIENEN en el CU → Diagrama de Clases
- Tarjetas CRC

DINÁMICO

Diagrama de secuencia

1 Describir el flujo de eventos del CU textualmente.

2 Crear una realización del CU

3 Extender la descripción del CU (si es necesario)

4 Encontrar clases de análisis a partir de la descripción del CU

5 Describir las responsabilidades de las clases

6 Establecer relaciones entre clases

7 Distribuir el comportamiento entre las clases

8 Describir atributos y asociaciones

9 Determinar mecanismos necesarios de las clases.

**PASOS PARA LA
REALIZACIÓN
DE CU**

¿QUÉ EFECTIVAMENTE DEBEMOS HACER?

1

Tarjetas CRC

2

Diagrama de
Clases

3

Diagrama de
Secuencia

PARTIMOS DE DESCRIPCION DE CU

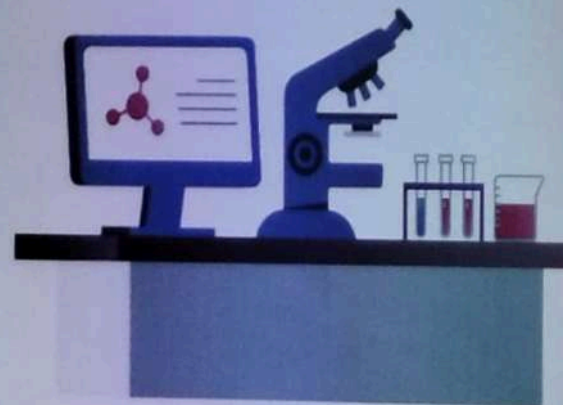
Descripción del CU "Agregar análisis"

Actor: Personal

Pre-condición: el personal de la clínica debe estar logueado

Pos-condición: análisis agregado al sistema

1. El personal elige la opción "Agregar análisis".
2. El sistema solicita ingrese DNI del paciente.
3. El personal ingresa DNI.
4. El sistema muestra el paciente solicitado.
5. El sistema solicita la matrícula del profesional que solicita el análisis.
6. El personal ingresa la matrícula solicitada.
7. El sistema muestra el profesional.
8. El sistema muestra los tipos de análisis.
9. El personal selecciona el tipo de análisis.
10. El sistema registra análisis y muestra fecha probable de entrega.
11. El personal confirma.
12. Fin del CU



ATENCIÓN

ANTES DE PASAR AL DIAGRAMA DE CLASES



Cada Realización de CU tiene una clase **CONTROLADOR** e **INTERFAZ**

EN GENERAL LOS NOMBRES UTILIZADOS TIENEN QUE VER CON EL CU QUE SE ESTÁ REALIZANDO

Para el CU "Agregar Análisis" agregaríamos la clase:

UIAgregarAnalysis

CTRLAgregarAnalysis

ENCONTRAR CLASES

Deberíamos identificar las clases candidatas que deben llevar a cabo el comportamiento descrito en el CU. Como una primera medida se deberían buscar **sustantivos** en la descripción textual del CU. Las clases identificadas a partir de los sustantivos formarán parte de una lista de clases candidatas

1

¿Está la clase candidata dentro de los límites del sistema?

2

¿Tiene esta clase candidata un comportamiento identificable para nuestro dominio de problema?

3

¿Tiene la clase candidata una estructura identificable? (se puede identificar una estructura de datos que maneje esa clase)

4

¿Tiene esta clase relaciones con otras clases?



RESPONSABILIDAD NO
ES LO MISMO QUE UN
MÉTODO

PERO LOS MÉTODOS SE
IMPELEMENTAN PARA
LLEVAR A CABO UNA
RESPONSABILIDAD

CLASE

C

RESPONSABILIDAD

R

COLABORACIÓN

C

LAS RESPONSABILIDADES A
TRAVÉS DE

TARJETAS CRC

Nombre de la clase

Propósito

Responsabilidad

Colaboración

Debe indicar cuál es fin para el cual esa
clase será utilizada en el CU que se está
realizando

TARJETAS CRC

Clase	CTRLAgregarAnálisis
Propósito	Representa a la clase encargada de conocer las acciones necesarias para agregar un nuevo análisis.

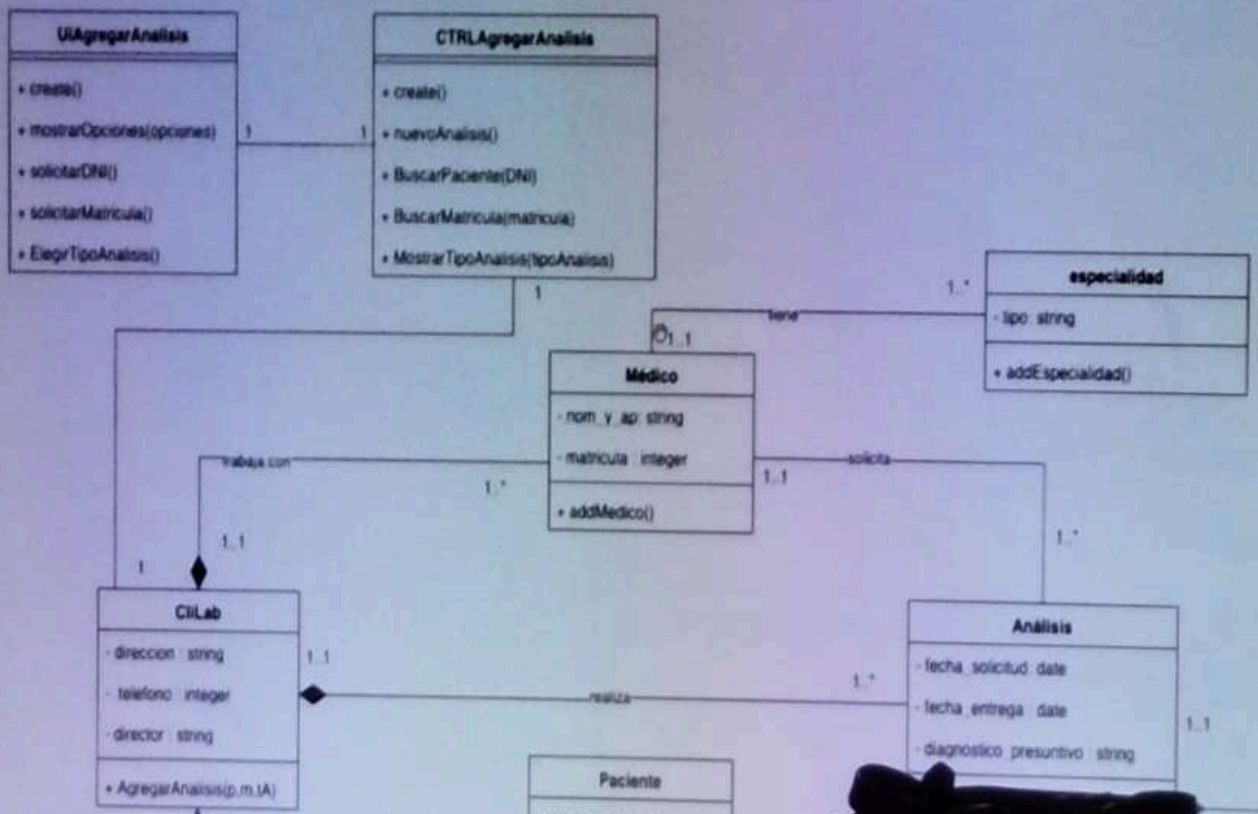
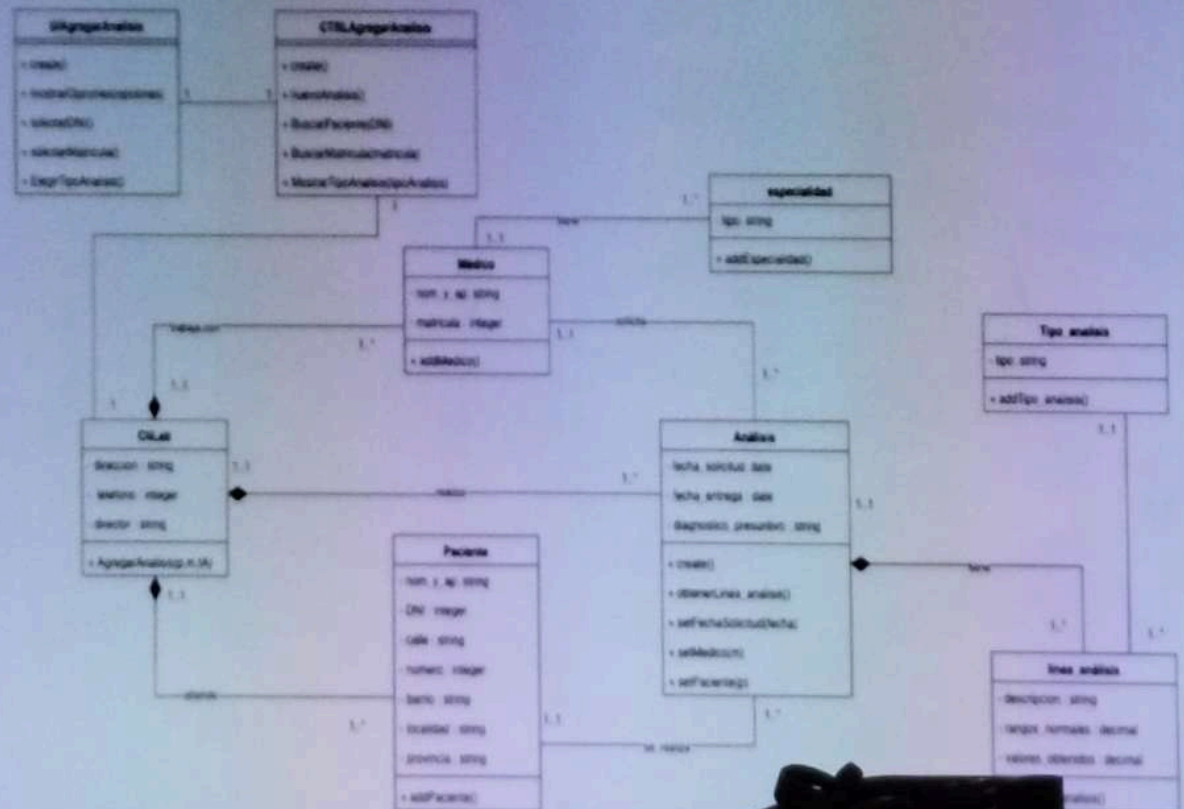
Responsabilidad	Colaboración
Crear la UI Enviar mensajes al sistema sobre las acciones que fueron seleccionados por el actor. Recibir los resultados de las operaciones realizadas por el sistema	UIAgregarAnálisis Sistema

TARJETAS CRC

Clase	UIAgregarAnálisis
Propósito	Representa a la clase interfaz que posibilita la interacción entre el actor y el Sistema para agregar un nuevo análisis

Responsabilidad	Colaboración
Recibir mensajes del nuevo análisis solicitado por el paciente. Recibir y enviar mensajes de confirmación. Mostrar en el formato adecuado los datos necesarios para agregar un nuevo análisis.	CTRLAgregarAnálisis

DIAGRAMA DE CLASES





Acuerdos de la cátedra

- Cuando debemos devolver una lista completa (como en el ejemplo de los tipos de análisis), SIN FILTRO, es posible que directamente lo pase SISTEMA, ya que conoce todas las clases.
- Búsquedas indexadas, por ejemplo por DNI, lo podemos hacer directamente con un método GET(lo que quiero buscar) con el valor como parámetro. SIN NECESIDAD DE RECORRER TODA LA LISTA
- CTRL puede consultar directamente a las clases SIN PASAR por la CLASE SISTEMA. Solo debe hacerlo SI O SI cuando se realiza un ABM

Anotación practica diseño

- Poner solo las clases que necesito para realizar ese caso de uso (muy importante)
- Tiene parte estatica y dinamica
- Diagrama de clase completo
- Mecanismos de clase (no se realiza porque tiene que ver más con las clases como se pasan a una base de datos)
- Tarjetas CRC
- Diagrama de secuencia
- Por cada realización de CU vamos a tener un controlador y una interfaz (los nombres de los mismos vienen dado por el CU)
- Preguntas guías para saber si la clase la agregó al Diagrama
 1. Está dentro del dominio?
 2. Tiene un método que perjudica
 3.)?
 4. Tiene relaciones o no
- La responsabilidad está dada por el conocer y hacer
- Puede haber métodos sin responsabilidad asociada
- La clase Interfaz colabora solo con el controlador

Diagrama de secuencia

- Solamente si no requiere ningún filtro al devolver la lista la catedra toma una convención para que pueda enviar un mensaje con toda la lista

- Siempre realizamos el camino feliz, no los alternativos
- Los actores secundarios se ponen del lado derecho al final, tienen el mismo formato que el actor que aparece a la izquierda. Se manda la solicitud lo que sea que quiera verificar o necesite. Si una person, no requiere que tenga una interfaz ni controlador, solamente que verifique lo que se necesite
- Si me devuelve uno solo no hace falta recorrer toda la lista, se hace el get
- Si yo quisiera realizar una consulta simplemente, puede no pasar necesariamente por el sistema e ir a la clase indicada
- Estas son convenciones triviales para que nos entendemos todos 😊

CLASE - 08/10

ANÁLISIS

ANÁLISIS

- Se analizan los requisitos refinándolos y estructurándolos.
- Problemas sin resolver.
- El lenguaje que utilizamos en el análisis se basa en un modelo de objetos conceptual, el modelo de análisis.
- Partes internas del sistema.

¿A USTED LE PARECE, CON LO QUE LE PAGO, LLEVARME LA CONTRARIA?

PROPÓSITOS - ANÁLISIS

ANÁLISIS - PROPÓSITOS

- Especificación más precisa que en captura de requisitos
- Se describe en el lenguaje de los desarrolladores
 - mayor formalismo.
- Estructura los requisitos entonces facilita:
 - la comprensión de requisitos
 - su preparación, su modificación, y en general, su mantenimiento.
- Es una primera aproximación al modelo de diseño.
- Abstracciones de alto nivel y evita resolver problemas vinculados con el diseño o la implementación

-los CU podrían llegar a tener cierta redundancia

-el análisis me permite a mi hacer una descripción

COMPARATIVA - ANÁLISIS

ANÁLISIS - COMPARATIVA	
Modelo de casos de uso	Modelo de Análisis
Descrito en el lenguaje del cliente .	Descrito en el lenguaje del desarrollador .
Vista externa del sistema.	Vista interna del sistema.
Estructurado por casos de uso , proporciona la estructura a la vista externa.	Estructurado por clases y paquetes estereotipados; proporciona la estructura a la vista interna.
Utilizado fundamentalmente como contrato entre el cliente y los desarrolladores sobre qué debería y qué no debería hacer el sistema.	Utilizado fundamentalmente por los desarrolladores para comprender cómo deberá darse forma al sistema, es decir, cómo debería ser diseñado e implementado.
Puede contener redundancias e inconsistencias entre requisitos.	No debería contener redundancias ni inconsistencias entre requisitos.
Captura la funcionalidad del sistema, incluida la funcionalidad significativa para la arquitectura.	Esboza cómo llevar a cabo la funcionalidad dentro del sistema, incluida la funcionalidad significativa para la arquitectura, sirve como una primera aproximación al diseño .
Define casos de uso que se analizarán con más profundidad en el modelo de análisis.	Define realizaciones de caso de uso, y cada una de ellas representan el análisis de un caso de uso del modelo de casos de uso.

Modelo de casos de uso	Modelo de análisis
Descrito con el lenguaje del cliente.	Descrito con el lenguaje del desarrollador.
Vista externa del sistema.	Vista interna del sistema.
Estructurado por los casos de uso; proporciona la estructura a la vista externa.	Estructurado por clases y paquetes estereotipados; proporciona la estructura a la vista interna.
Utilizado fundamentalmente como contrato entre el cliente y los desarrolladores sobre qué debería y qué no debería hacer el sistema.	Utilizado fundamentalmente por los desarrolladores para comprender cómo debería darse forma al sistema, es decir, cómo debería ser diseñado e implementado.
Puede contener redundancias, inconsistencias, etc., entre requisitos.	No debería contener redundancias, inconsistencias, etc., entre requisitos.
Captura la funcionalidad del sistema, incluida la funcionalidad significativa para la arquitectura.	Esboza cómo llevar a cabo la funcionalidad dentro del sistema, incluida la funcionalidad significativa para la arquitectura; sirve como una primera aproximación al diseño.
Define casos de uso que se analizarán con más profundidad en el modelo de análisis.	Define realizaciones de casos de uso, y cada una de ellas representa el análisis de un caso de uso del modelo de casos de uso.

- en el modelo de análisis se trata de llevar esos requisitos al ámbito del desarrollador
- los CU solo se concentran cómo interactúa el sistema con el exterior
- las clases que vamos a ver fundamentalmente, son clases conceptuales
- empiezan aparecer clases y paquetes. Tiene la particularidad que el PU propone ciertos estereotipos
- en los CU decimos el **QUE** y en el análisis y diseño el **CÓMO**
- aparecen los CU como guía del proceso
- cada CU nos va a servir para como hacer una Realización de CU a nivel de análisis

Y EL CICLO DE VIDA - ANÁLISIS

ANÁLISIS - Y EL CICLO DE VIDA

- Es el foco de las primeras iteraciones de la elaboración
- Contribuye a desarrollar la arquitectura y entender mejor los requisitos
- Variantes en diferentes proyectos (costo/beneficio):
 - Usar el modelo de análisis y mantenerlo
 - Usar el modelo de análisis y no mantenerlo
 - No usar el modelo de análisis

-empezamos a desarrollar la arquitectura de nuestro sistema

-el análisis también contribuye a ir armando la arquitectura y como los requisitos empiezan a encajar en esa arquitectura

-el modelo de análisis es de una manera opcional

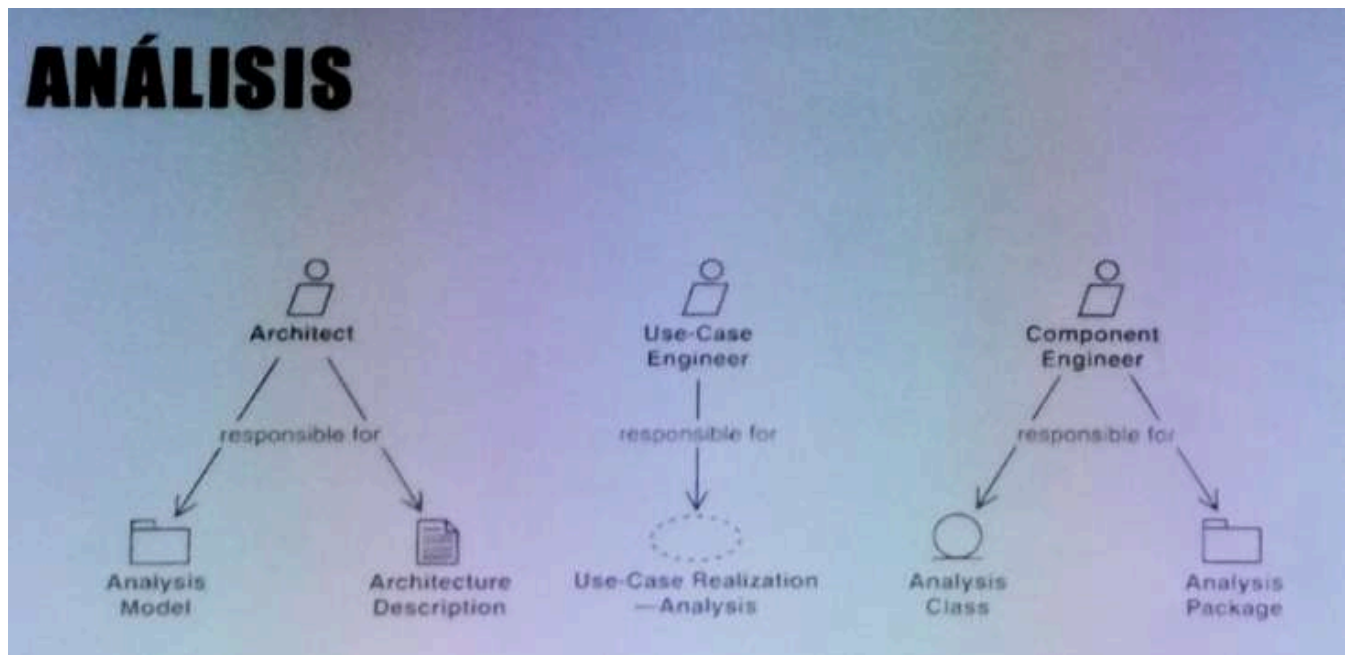
-hay 3 formas de encararlo, no siempre se busca hacer el modelo de análisis pero de alguna forma hay que reemplazarlo

2) hacemos un bosquejo que nos sirve como puente para llegar al diseño

3) no usar el modelo de análisis para nada. Hacer mucho más específico el modelo de CU

Al elegir entre las dos primeras variantes, debemos sopesar las ventajas de mantener el modelo de análisis con el coste de mantenerlo durante varias iteraciones y generaciones. Por tanto, tenemos que realizar un análisis coste/beneficio correcto, y decidir si deberíamos dejar de actualizar el modelo de análisis —quizá tan pronto como al final de la fase de elaboración— o si deberíamos conservarlo y mantenerlo durante el resto de la vida del sistema.

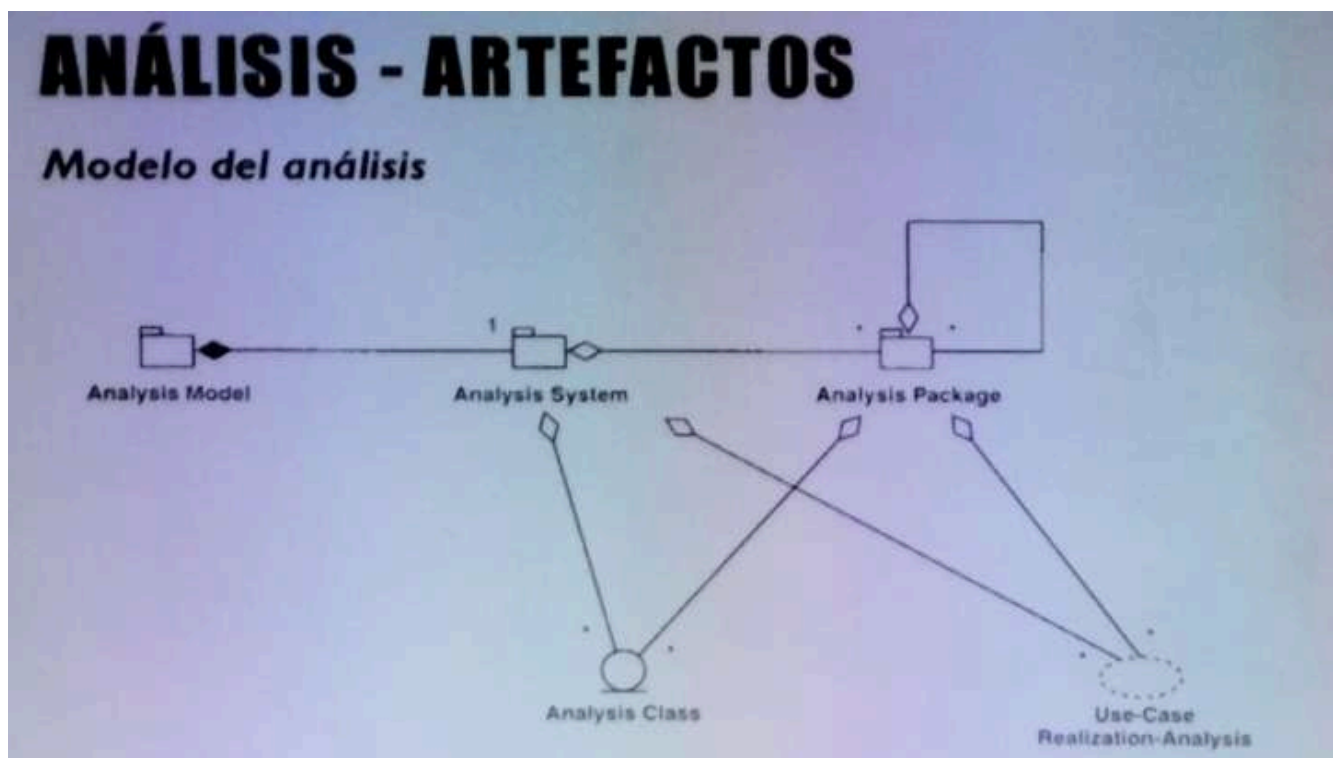
Diagrama - ANÁLISIS



-fíjense que acá tenemos 3 actores. El **arquitecto** ya se mencionaba antes

ARTEFACTOS - ANÁLISIS

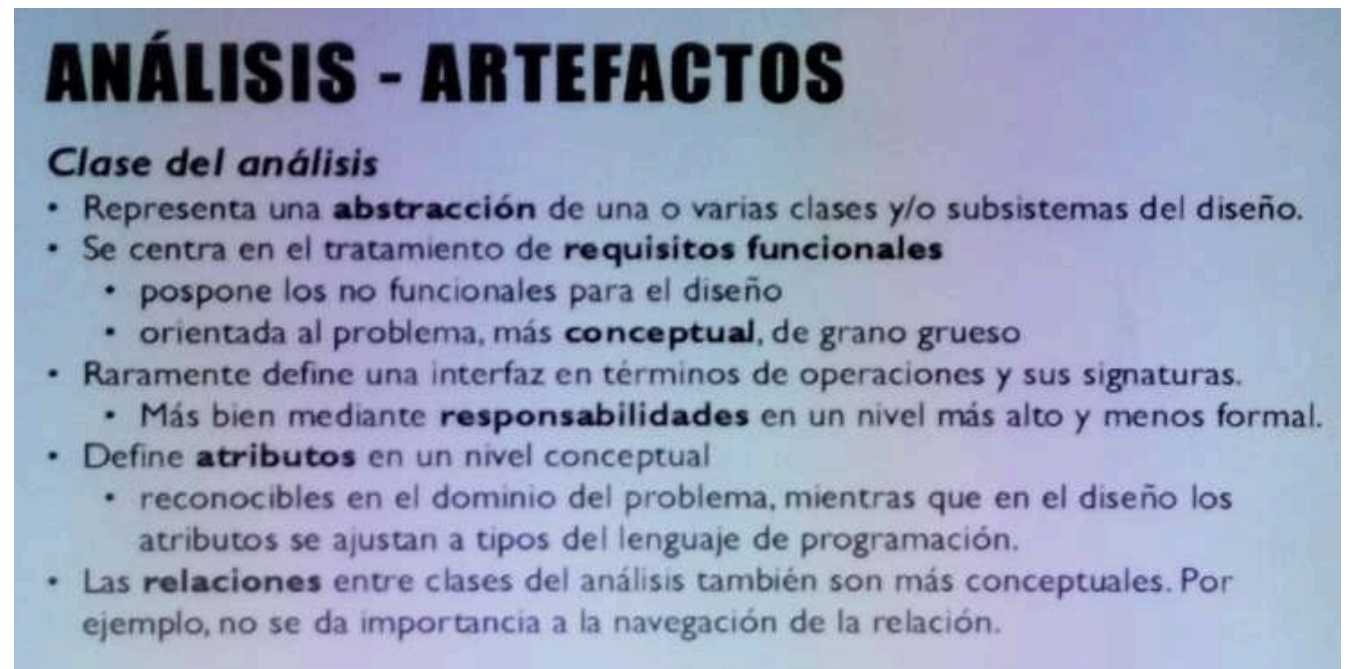
Modelo de análisis



-el modelo de análisis está compuesto por un análisis o un sistema de análisis. Dentro del sistema de análisis podemos llegar a tener un montón de paquetes

- la mejor manera sería representarlo con una composición, no una agregación como se muestra
- los paquetes pueden tener otros paquetes

Clase de análisis



ANÁLISIS - ARTEFACTOS

Clase del análisis

- Representa una **abstracción** de una o varias clases y/o subsistemas del diseño.
- Se centra en el tratamiento de **requisitos funcionales**
 - pospone los no funcionales para el diseño
 - orientada al problema, más **conceptual**, de grano grueso
- Raramente define una interfaz en términos de operaciones y sus signatures.
 - Más bien mediante **responsabilidades** en un nivel más alto y menos formal.
- Define **atributos** en un nivel conceptual
 - reconocibles en el dominio del problema, mientras que en el diseño los atributos se ajustan a tipos del lenguaje de programación.
- Las **relaciones** entre clases del análisis también son más conceptuales. Por ejemplo, no se da importancia a la navegación de la relación.

-primero hay que pensar como hoy dijimos que la disciplinas de análisis se encarga de abordar en mayor detalle los requisitos. Las clases que vamos a tener acá van a estar descrita en mayor abstracción

-lo que se hace con una clase es definir sus atributos y sus responsabilidades. No se definen métodos

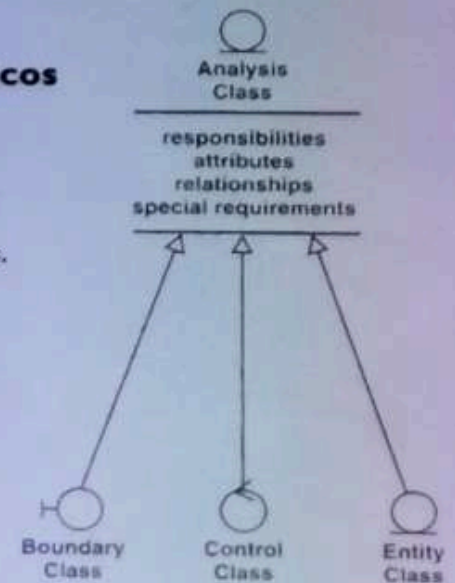
-los atributos se definen a nivel conceptual, diciendo es string o entero

(otro power)

ANÁLISIS - ARTEFACTOS

Clase del análisis

- siempre encajan en alguno de los **estereotipos básicos**
 - **Entidad**
se derivan directamente de las clases entidad del negocio (o dominio).
 - **Control**
encapsulan el control de casos de uso, o cálculos complejos.
 - **Frontera**
modelan la interacción entre actores y el sistema.



-una clase de análisis en definitiva debería verse de esta manera, con este tipo de estereotipos

-vamos a tener clases, que ya van a tener ciertas características

-una clase frontera interactúa con el entorno del sistema, puede ser un actor humano o pueden ser otros sistemas

-todo lo que sea un actor debería llevar en el medio una clase de tipo frontera

-la clase control se utilizan para los CU. Vamos a aplicar el patrón que veíamos en GRASP (Controlador). Saben a quién llamar para que haga las cosas. Sabe cómo llevar a cabo un CU o un cálculo complejo

-la clase control termina ese CU y se destruye

-las entidades se encargan generalmente de conocer los datos o información.

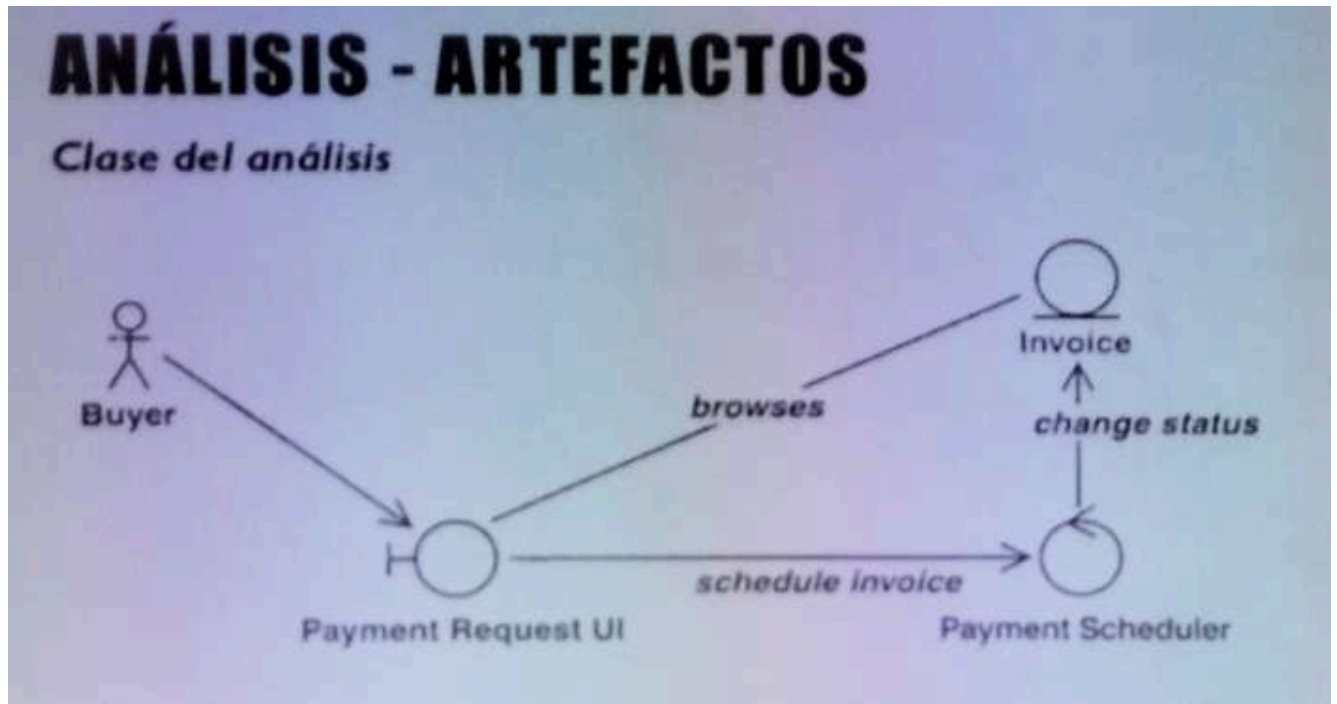
-cuando no tenemos definidas clase de control particulares, la usamos como clase de control

-la clase de Sistema ya debería estar despejada y deberíamos tener un controlador por cada CU

Cada clase de interfaz debería asociarse con al menos un actor, y viceversa.

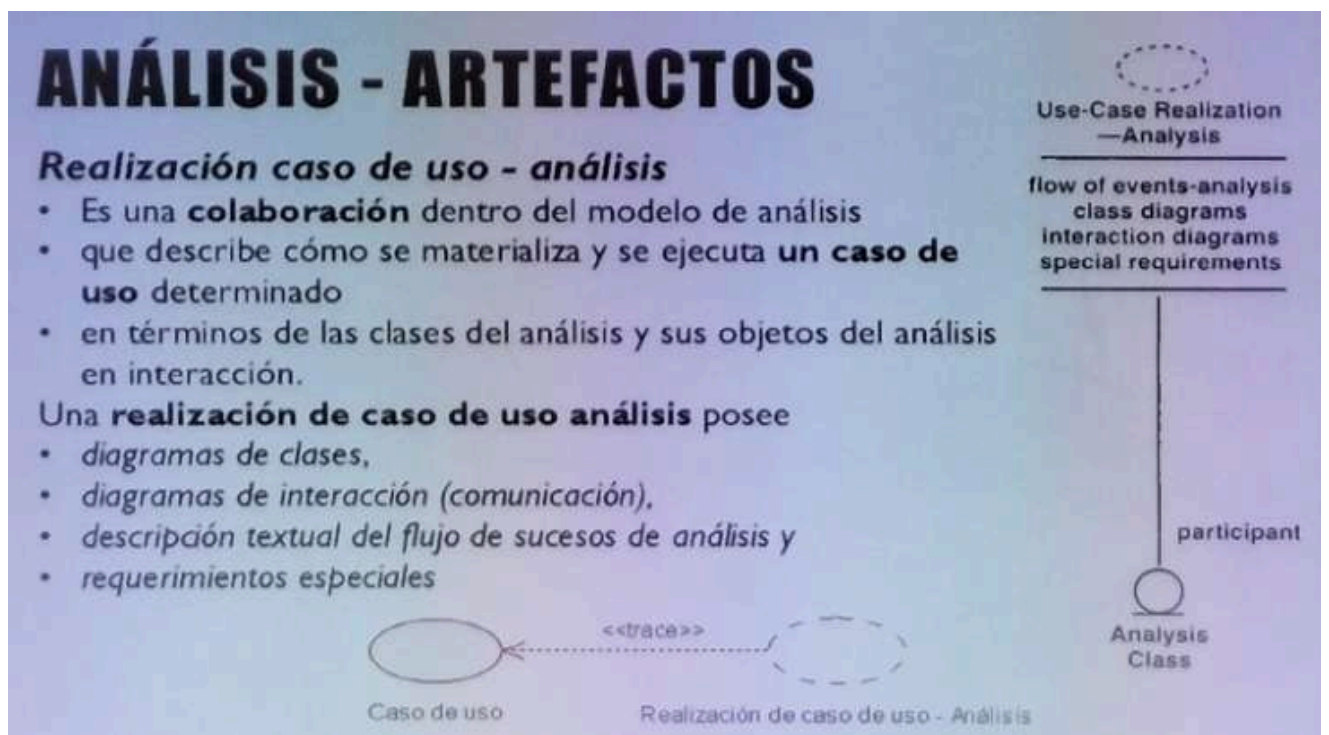
Los aspectos dinámicos del sistema se modelan con clases de control, debido a que ellas manejan y coordinan las acciones y los flujos de control principales, y delegan trabajo a otros objetos (es decir, objetos de interfaz y de entidad).

(otro power)



- una clase factura que se encarga de...
- este es un planificador de pagos, que lo que hace es acceder a las peticiones que vienen desde la clase frontera
- este diagrama de clases es muy básico en comparación con lo que venimos haciendo pero es un primer diagrama dentro de la disciplina de análisis

Realización caso de uso - análisis

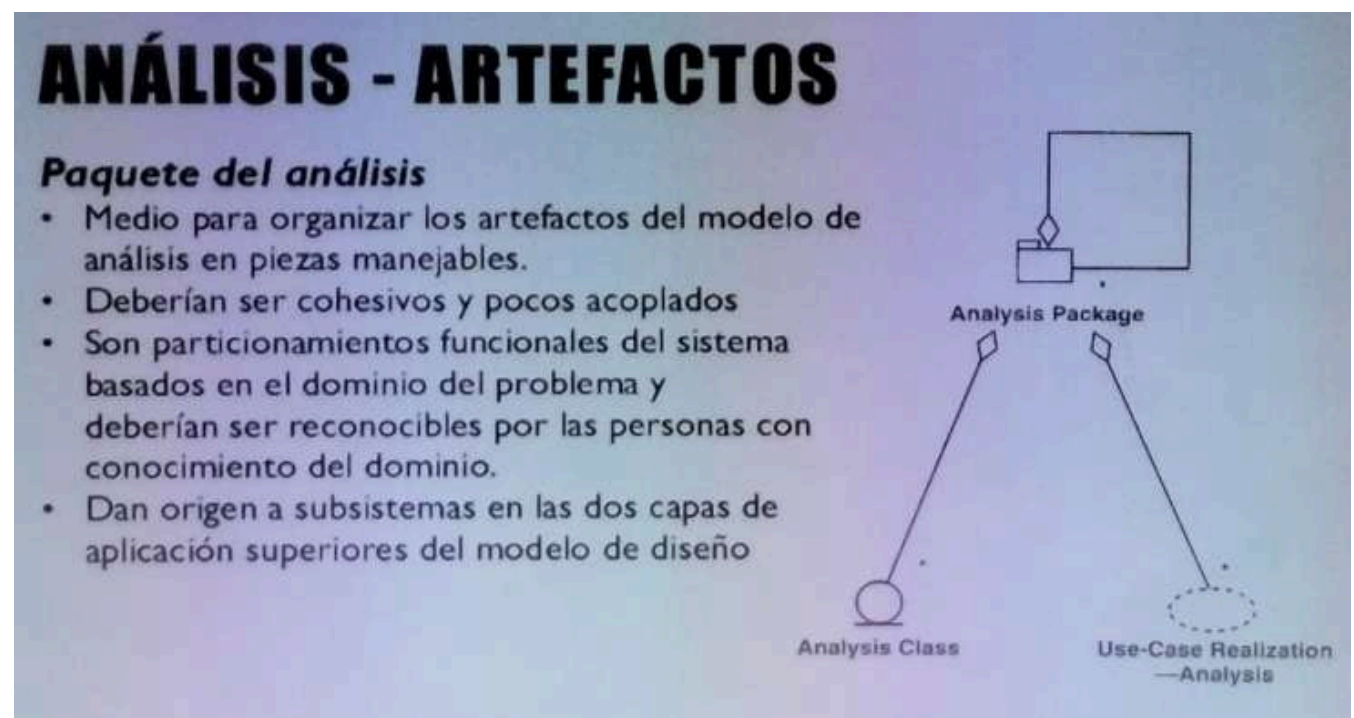


- en cada realización lo que va a tener un conjunto de clases para llevarlo a cabo y las interacciones necesarias
- el PU lo divide en dos partes: a nivel de **análisis** y a nivel de **diseño**
- son las interacciones a nivel de clase de análisis
- esas Realizaciones se dan en el contexto de que vienen modeladas a través de **Diagramas de Comunicación**
- el Diagrama de comunicación es una forma más sencilla del Diagrama de interacción. Y se suele usar más en esta etapa
- si un CU sufre una modificación esto indica que todas las clases se ven afectadas
- (arriba a la derecha del power) requerimientos especiales son sinónimos de requisitos no funcionales. (Dijo algo que no lo vemos)
- el **flujo de evento de análisis** es una descripción textual para los diagramas de comunicación o secuencia (puede confundirse con la Descripción textual de un CU). **Incluye objetos**, esto no está permitido en la Descripción textual

esto con diagramas de colaboración ya que nuestro objetivo fundamental es identificar requisitos y responsabilidades sobre los objetos, y no identificar secuencias de interacción detalladas y ordenadas cronológicamente (es ese caso, utilizaríamos en cambio diagramas de secuencia).

8.4.3.3. Flujo de sucesos-análisis Los diagramas —especialmente los de colaboración— de una realización de caso de uso pueden ser difíciles de leer por sí mismos, de modo que puede ser útil un **texto adicional que los explique**. Este texto debe escribirse en términos de objetos, particularmente objetos de control que interactúan para llevar a cabo el caso de uso. Sin embargo, el texto no debería mencionar ninguno de los atributos, responsabilidades, y asociaciones del objeto, debido a que cambian con bastante frecuencia. Sería difícil mantenerlos.

Paquete del Análisis



- en general en cada paquete, el criterio de agrupamiento a seguir es que vamos a tratar de vincular los de un CU, o casos de uso muy vinculados
- esta capa de paquete sería la que estaría más arriba de todos

Paquete de servicio

ANÁLISIS - ARTEFACTOS

Paquete de servicio

- Un **servicio** representa un conjunto coherente de acciones relacionadas funcionalmente —un paquete de funcionalidad— que se utiliza en varios casos de uso.
- Un **servicio** es indivisible en el sentido de que el sistema necesita ofrecerlo o todo entero o nada en absoluto.
- Un caso de uso requiere acciones de varios servicios.
- Los **paquetes de servicio** se utilizan en el nivel más bajo de la jerarquía de paquetes de análisis para estructurar el sistema de acuerdo con los servicios que proporciona.

(no es tan importante)

- forma un núcleo donde esa funcionalidad puede ser ocupada por uno o más CU
- esta asociada a la Arquitectura asociada a servicios y al concepto de **Componente**
- La idea de paquetes de servicios es que puedan ser reemplazados sin problema

ANÁLISIS - ARTEFACTOS

Paquete de servicio

- contiene un conjunto de clases relacionadas funcionalmente
- es indivisible.
- para llevar a cabo un caso de uso puede que participen más de un paquete de servicios.
- puede depender de otro paquete de servicios.
- puede gestionarse como una unidad de distribución independiente. Puede representar una funcionalidad "adicional" del sistema.
- pueden ser mutuamente excluyentes, o pueden representar diferentes variantes del mismo servicio.
- constituyen la entrada fundamental para las actividades de diseño e implementación subsiguientes, dado que ayudarán a estructurar los modelos de diseño e implementación en términos de subsistemas de servicio.

Es una "caja cerrada" de funcionalidad lógica que usas como pieza de construcción para armar tus casos de uso, y que está lista para convertirse en un componente de software real.

Descripción de la arquitectura (vista del modelo del análisis)

ANÁLISIS - ARTEFACTOS

Descripción de la arquitectura (vista del modelo de análisis)

Artefactos del modelo de análisis significativos para la arquitectura:

- Descomposición del modelo de análisis en **paquetes de análisis y sus dependencias**. Esta descomposición suele tener su efecto en los subsistemas de las capas superiores durante el diseño e implementación.
- Las **clases fundamentales** del análisis.
- **Realizaciones de casos de uso** que describen funcionalidades importantes y críticas, correspondientes a los casos de uso que aparecen en la vista de la arquitectura del modelo de casos de uso.

-vamos a tomar los paquetes y clases más importantes

-desde el punto de vista del modelo de análisis solo va una parte, la que influye más en la arquitectura

- Las **clases fundamentales del análisis** como las clases de entidad que encapsulan un fenómeno importante del dominio del problema; las clases de interfaz que encapsulan interfaces de comunicación importantes y mecanismos de interfaz de usuario; las clases de control que encapsulan importantes secuencias con una amplia cobertura (es decir, aquellas que coordinan realizaciones de casos de uso significativas); **y clases del análisis que son generales, centrales, y que tienen muchas relaciones con otras clases del análisis**. Suele ser suficiente con considerar significativa para la arquitectura una clase abstracta pero no sus subclases.

TRABAJADORES - ANÁLISIS

ANÁLISIS - TRABAJADORES

Arquitecto

- Resp. de la integridad del modelo de análisis,
 - garantizando que sea correcto, consistente, y legible como un todo.
- El arquitecto es responsable de:
 - la descripción de la arquitectura
 - el modelo del análisis

Ingeniero de casos de uso

- Resp. de la integridad de una o varias realizaciones de casos de uso – análisis

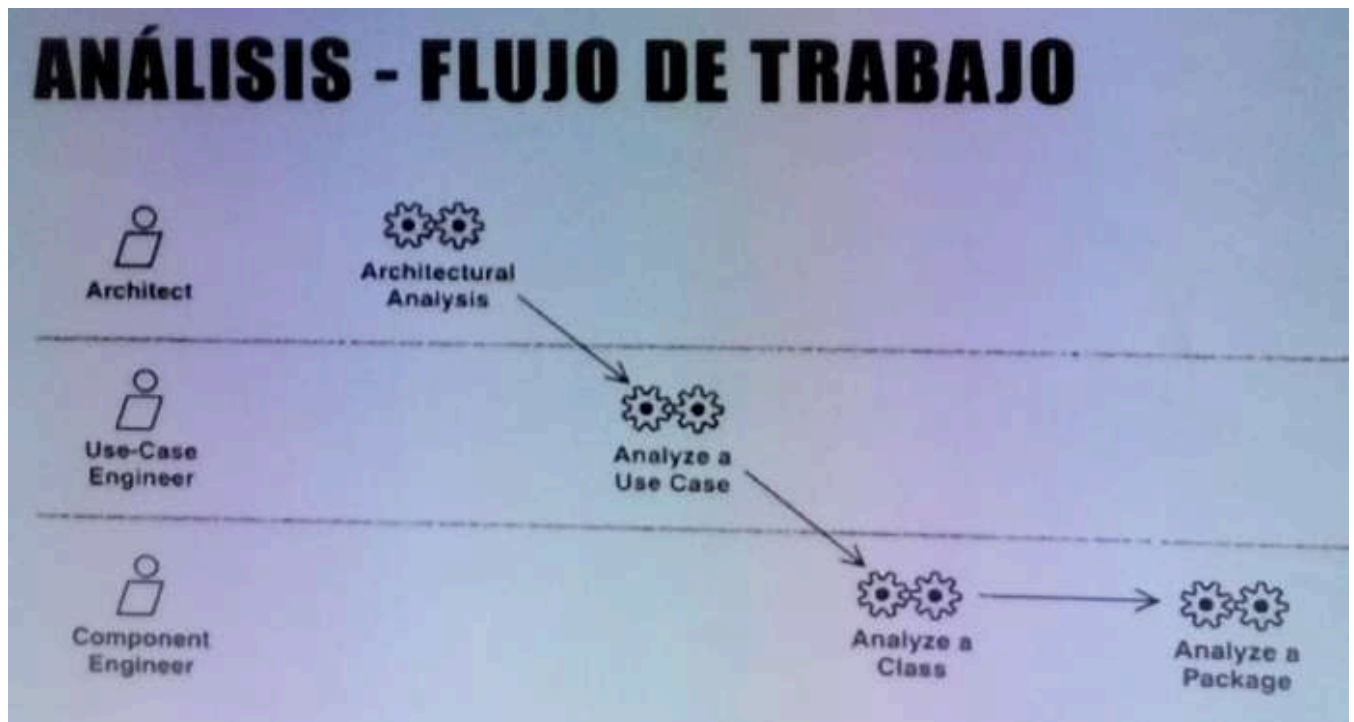
Ingeniero de componentes

- Define y mantiene las responsabilidades, atributos, relaciones, y requisitos especiales de una o varias clases del análisis asegurándose de que cada clase del análisis cumple los requisitos que se esperan de ella de acuerdo a las realizaciones de caso de uso en que participan.
- También mantiene la integridad de uno o varios paquetes del análisis.



Figura 8.1. Los trabajadores y artefactos implicados en el análisis.

FLUJO DE TRABAJO - ANÁLISIS



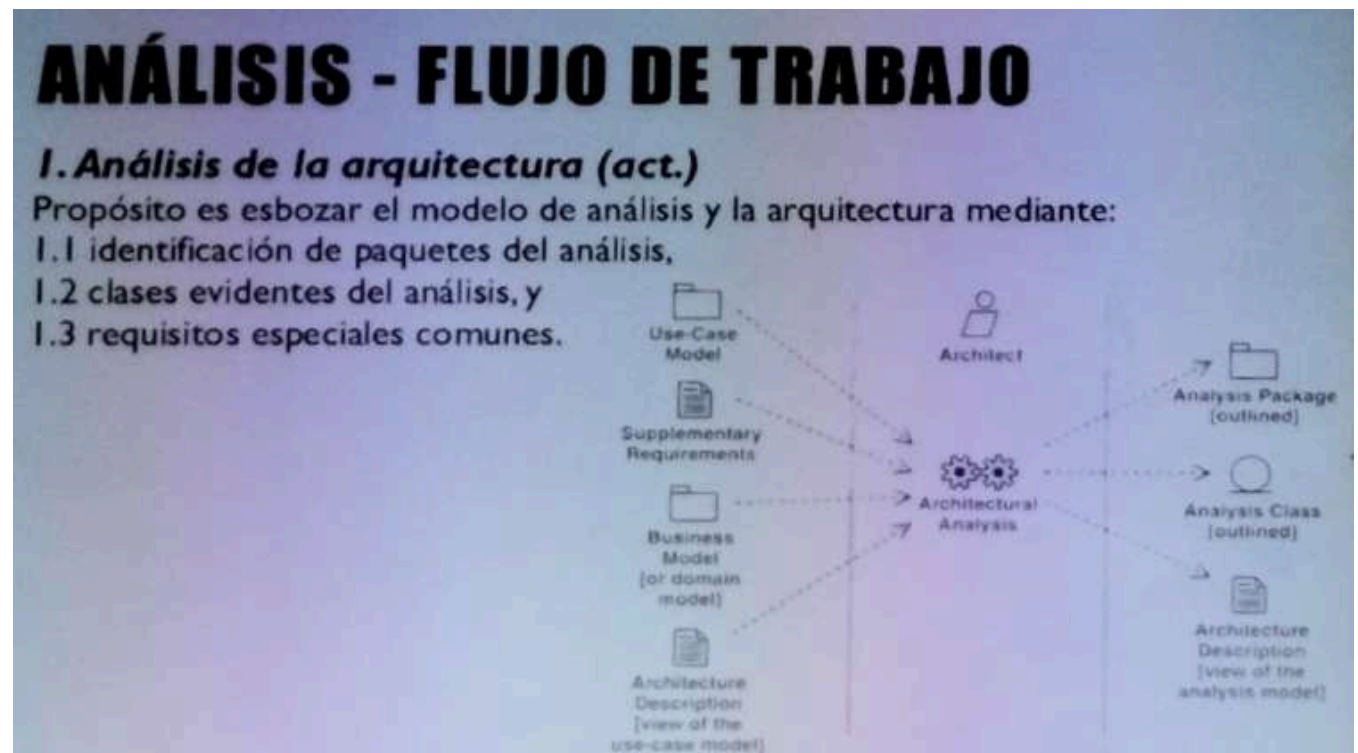
-no es necesario que se siga esta **secuencia** de manera estricta

-se va mostrando desde diferentes perspectivas

-empezamos un análisis de la arquitectura a nivel general, en la segunda etapa es donde realmente se hace la realización de CU y finalmente es donde se encarga de los detalles de cada cosa

Los arquitectos comienzan la creación del modelo de análisis (como hemos definido anteriormente en este capítulo), identificando los paquetes de análisis principales, las clases de entidad evidentes, y los requisitos comunes. Después, los ingenieros de casos de uso realizan cada caso de uso en términos de las clases del análisis participantes exponiendo los requisitos de comportamiento de cada clase. Los ingenieros de componentes especifican posteriormente estos requisitos y los integran dentro de cada clase creando responsabilidades, atributos y relaciones consistentes para cada clase. Durante el análisis, el arquitecto identifica de manera continua nuevos paquetes del análisis, clases, y requisitos comunes a medida que el modelo de análisis evoluciona, y los ingenieros de componentes responsables de los paquetes del análisis concretos continuamente los refinan y mantienen.

1. Análisis de la arquitectura



-los artefactos vienen de la captura de requisitos

-el arquitecto se encarga de identificarlo en un alto nivel

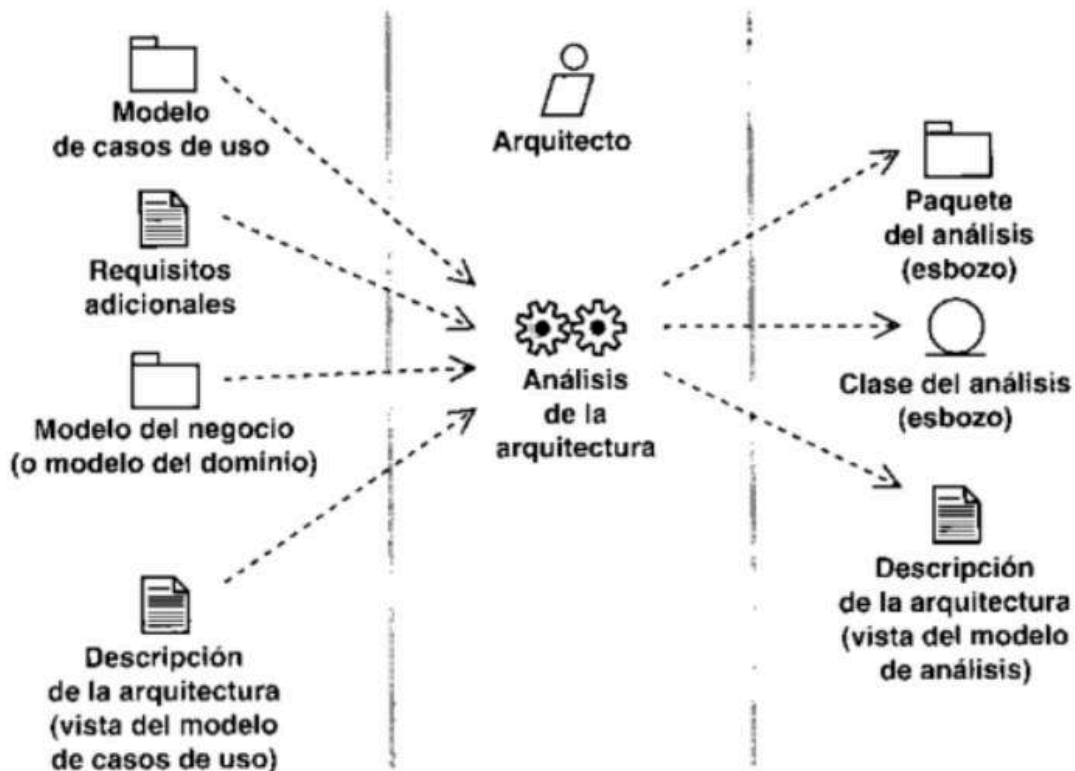


Figura 8.19. Las entradas y los resultados del análisis arquitectónico.

1.1 Identificación de paquetes de análisis

ANÁLISIS - FLUJO DE TRABAJO

1.1 Identificación de paquetes de análisis

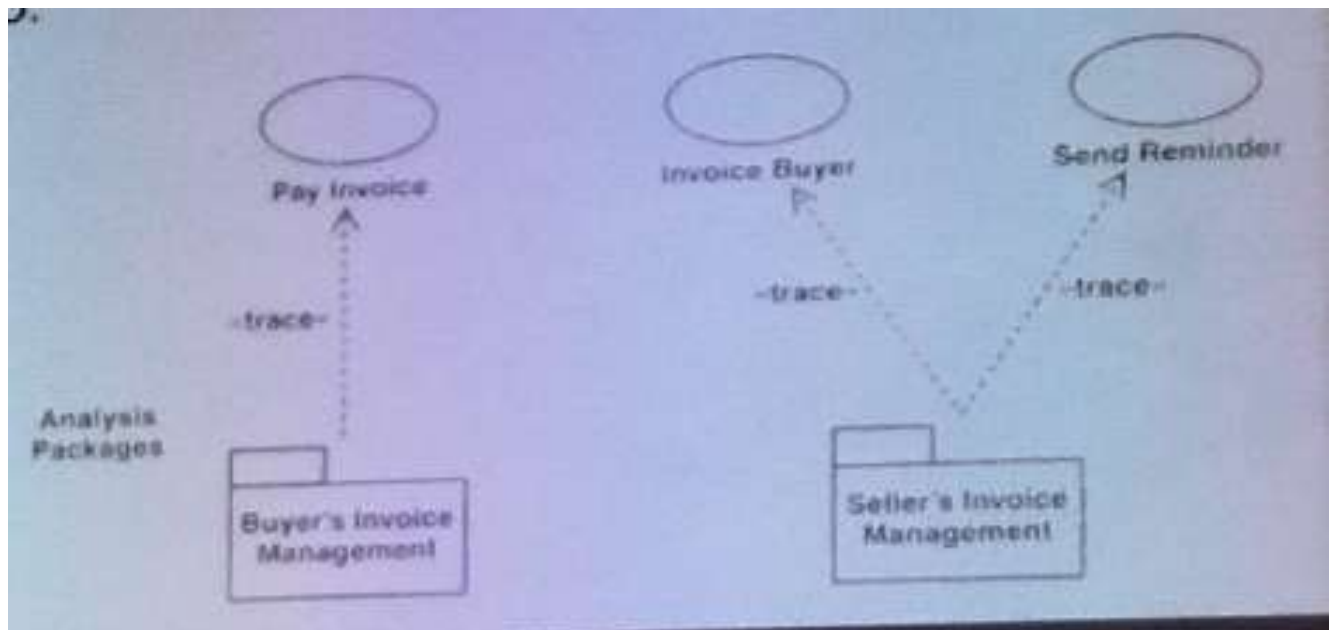
Pueden identificarse inicialmente como forma de dividir el análisis.
Agrupación basada en los requisitos funcionales, agrupando un cierto número de casos de uso en un paquete concreto, y realizando la funcionalidad correspondiente dentro de dicho paquete.

Criterios para agrupar casos de uso:

- Aquellos que den soporte a un **proceso de negocio**.
- Aquellos que den soporte a un **actor del sistema**.
- Casos de uso relacionados mediante **generalización o extensión**.

El diagrama de flujo de trabajo muestra dos paquetes de análisis. El paquete 'Buyer's Invoice Management' (representado por un rectángulo) tiene una flecha de 'trace' que apunta al caso de uso 'Pay Invoice' (círculo). El paquete 'Seller's Invoice Management' (representado por un rectángulo) tiene una flecha de 'trace' que apunta al caso de uso 'Invoice Buyer' (círculo). Además, hay una flecha de 'trace' que apunta desde 'Seller's Invoice Management' al caso de uso 'Send Reminder' (círculo). Las flechas de 'trace' indican que los paquetes de análisis soportan o rastrean la funcionalidad de los casos de uso.

(literalmente lo paso, dijo q dps veamos)



1.2 Identificación de clases entidad obvias

1.3 Identificación de requisitos especiales

ANÁLISIS - FLUJO DE TRABAJO

1.2 Identificación de clases de entidad obvias

Pueden identificarse una lista de clases entidad candidatas (10-20) basado en las clases del dominio o las entidades del negocio.

Sin embargo, la mayoría de las clases se identificarán al crear las realizaciones de casos de uso.

En esta etapa es conveniente un esbozo inicial de las clases significativas para la arquitectura.

1.3 Identificación de requisitos especiales

Como ejemplos podemos citar restricciones sobre:

- persistencia
- distribución y concurrencia
- características de seguridad
- tolerancia a fallos
- gestión de transacciones

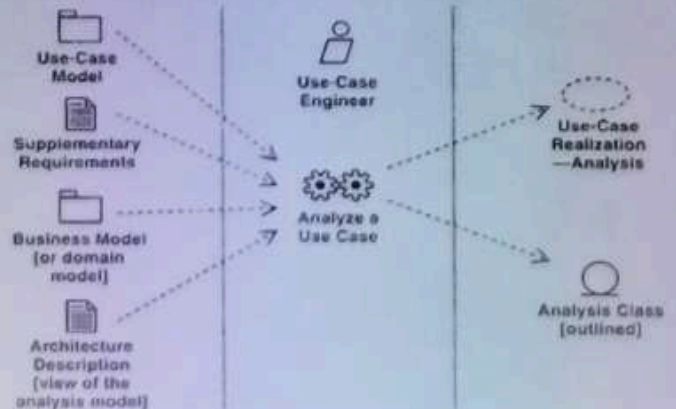
2. Analizar un caso de uso

ANÁLISIS - FLUJO DE TRABAJO

2. Analizar un caso de uso (act.)

Analizamos un caso de uso para:

1. Identificar las clases del análisis necesarias para llevar a cabo el flujo de sucesos del caso de uso.
2. Distribuir el comportamiento del caso de uso entre objetos del análisis que interactúan.
3. Capturar requisitos especiales sobre la realización del caso de uso.



-vamos a ver que clase se ocupan ahí

-vamos a tratar de distribuir el comportamiento que vamos identificando en los CU

-sigue siendo una clase bosquejada porque no tiene los detalles, vemos el comportamiento esperado. Lo que debemos hacer es distribuir ese comportamiento entre los objetos

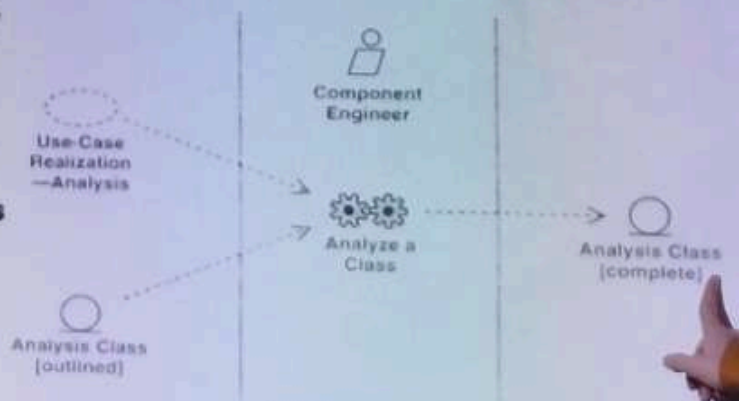
-el **único artefacto** que viene de la disciplina de análisis es la **Descripción de la arquitectura** (el resto de **artefactos de entrada** se mantiene)

3. Analizar una clase

ANÁLISIS - FLUJO DE TRABAJO

3. Analizar una clase (act.)

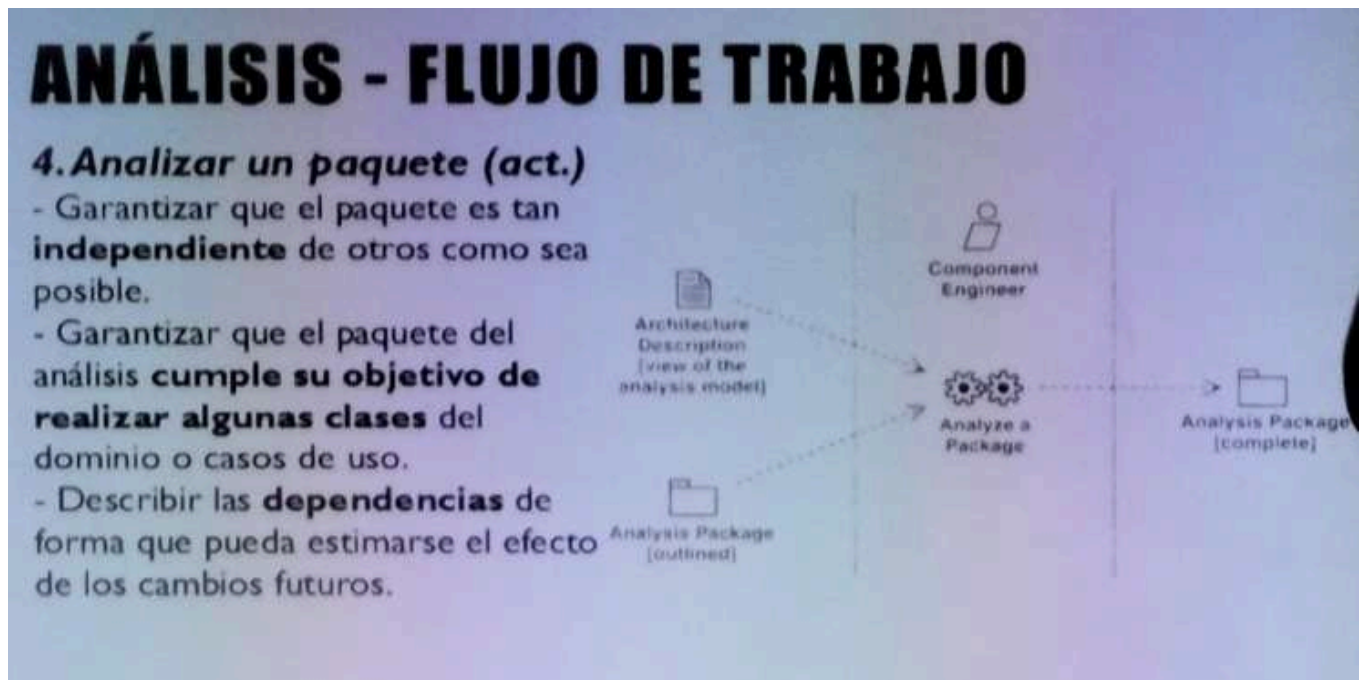
1. Identificar y mantener las **responsabilidades** de la clase, basadas en su papel en las realizaciones de casos de uso.
2. Identificar **atributos**.
3. Identificar **asociaciones y agregaciones**.
4. Identificar **generalizaciones**.
5. Capturar **requisitos especiales** sobre la realización de la clase.



- la idea es tener una clase terminada una vez que salgo de aca
- una vez que tenemos claras las interacciones, nos vamos a meter a analizar la clase
- se marcan lo más importante, en la etapa de diseño entramos más en detalle

(Tiene como entrada todos lo de salida del anterior)

4. Analizar un paquete



-es muy importante el tema de las dependencias que van a existir entre paquetes

(Tiene como entrada la salida de “Análisis de la arquitectura”)

- Definir y mantener las dependencias del paquete con otros paquetes cuyas clases contenidas estén asociadas con él.
- Asegurarnos de que el paquete contiene las clases correctas. Intentar hacer cohesivo el paquete incluyendo sólo objetos relacionados funcionalmente.

DISEÑO

DISEÑO

- Da **forma** al sistema para que soporte todos los requisitos
- Crea modelos que se enfocan en los req. **no funcionales** y el **dominio de la solución**.
- Modelos de **diseño y de despliegue**
- Una entrada esencial al diseño es el **modelo de análisis**, si fue desarrollado.



PROPÓSITOS - DISEÑO

DISEÑO - PROPÓSITOS

- Lograr una comprensión profunda de los problemas relacionados con los **requisitos no funcionales**
 - lenguajes de programación, los sistemas operativos, etc.
- Crear un **punto de partida** para las actividades de **implementación** posteriores
 - mediante la captura de requisitos en subsistemas, interfaces y clases individuales.
- **Descomponer** el trabajo de implementación en piezas más manejables
 - por diferentes equipos de desarrollo
 - útil cuando dicha descomposición no se puede hacer en función del modelo de casos de uso o el de análisis

(otro power)

DISEÑO - PROPÓSITOS

- Capturar las principales **interfaces entre subsistemas**
 - razonamiento sobre la arquitectura
 - uso de interfaces para sincronizar diferentes equipos
- Visualizar y razonar sobre el diseño mediante el uso de una **notación común**
- Crear una **abstracción sin costuras** de la implementación del sistema
 - la implementación es un refinamiento directo del diseño al rellenar la "carne" pero no cambiar la estructura
 - Permite el uso de técnicas como la generación de código y la ingeniería de ida y vuelta entre el diseño y la implementación

- en la disciplina de diseño empieza aparecer explícitamente el concepto de interfaces, ya sea entre clases y sobre todo entre subsistemas
- las interfaces como clasificador, no de tipo frontera
- el diseño es un modelo que nosotros hacemos con el suficiente nivel de detalle....
- trabajamos todo con el lenguaje que vamos a utilizar, pero no se llega a hacer todo

COMPARATIVA

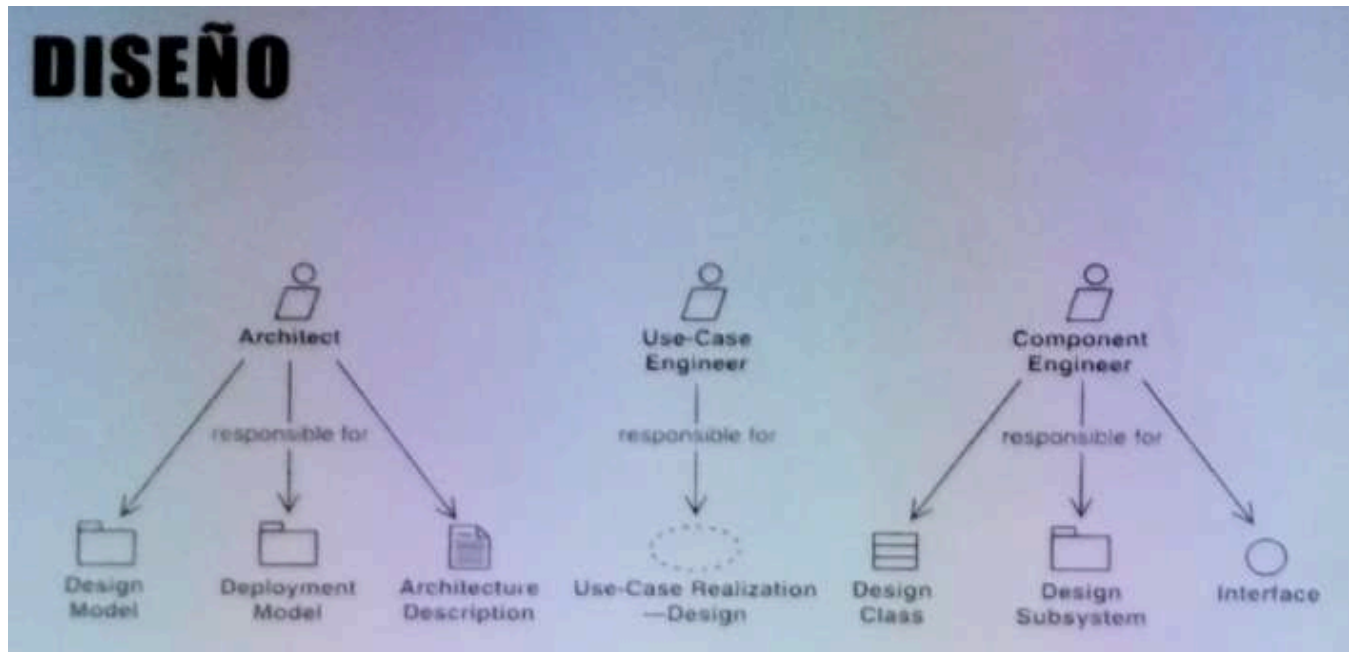
DISEÑO - COMPARATIVA

Modelo de Análisis	Modelo de Diseño
Modelo conceptual.	Modelo físico (implementación)
Genérico (aplicable a varios diseños)	Específico para una implementación
Tres estereotipos: entidad, control, frontera.	Cualquier nro. de estereotipos físicos.
Menos formal.	Más formal.
Menos caro de desarrollar	Más caro.
Menos capas.	Más capas.
Dinámico (no muy centrado en la secuencia)	Dinámico (muy centrado en la secuencia)
Creado principalmente como trabajo manual	Uso de "programación visual" en ing. de ida y vuelta.
Puede no mantenerse todo el ciclo de vida.	Debe ser mantenido todo el ciclo de vida.

- nivel bajo de abstracción para que se entienda la flexibilidad de uso
- una característica es que el modelo de análisis viene a ser genérico desde el punto de vista del dominio
- el modelo de diseño es más costoso, obviamente va a serlo por tener mayor detalle

- este modelo de diseño, debería mantenerse a lo largo de todo el proyecto. Luego se analizan los cambios que se le aplican de acuerdo a los costos que conlleva cada uno, pero el modelo en si se debe mantener.
- en la etapa de diseño es donde llegamos a un nivel de detalle máximo, entonces a partir de ese modelo de diseño se puede llegar a obtener el de implementación
- mientras más formal hay **menos ambigüedad** en lo que estoy realizando

Diagrama - DISEÑO



- se mantienen los 3 trabajadores
- cambian los artefactos en su mayoría
- el arquitecto es el responsable de mantener la integridad de todo el sistema
- el ing de CU, ahora tiene la realización de CU pero de Diseño.
- los subsistemas de diseño son los paquetes con mayor nivel de detalle
- (este es un estereotipo de interfaz que se podría usar en UML también)



Figura 9.1. Trabajadores y artefactos involucrados en el diseño.

Y EL CICLO DE VIDA - DISEÑO

DISEÑO - Y EL CICLO DE VIDA

- Importante al **final de la fase de elaboración** y al **inicio de la fase de construcción**
- Se pretende **refinar y estabilizar la arquitectura**
- **Crear un plano detallado** para el modelo de implementación.
- Una vez que la arquitectura está estable y los requisitos bien comprendidos, el foco cambia **hacia la implementación**
- Debe **mantenerse y actualizarse** durante todo el ciclo de vida

-al estar al final de la fase de elaboración, el diseño se encarga de formar la arquitectura estable

ARTEFACTOS - DISEÑO

Modelo del Diseño

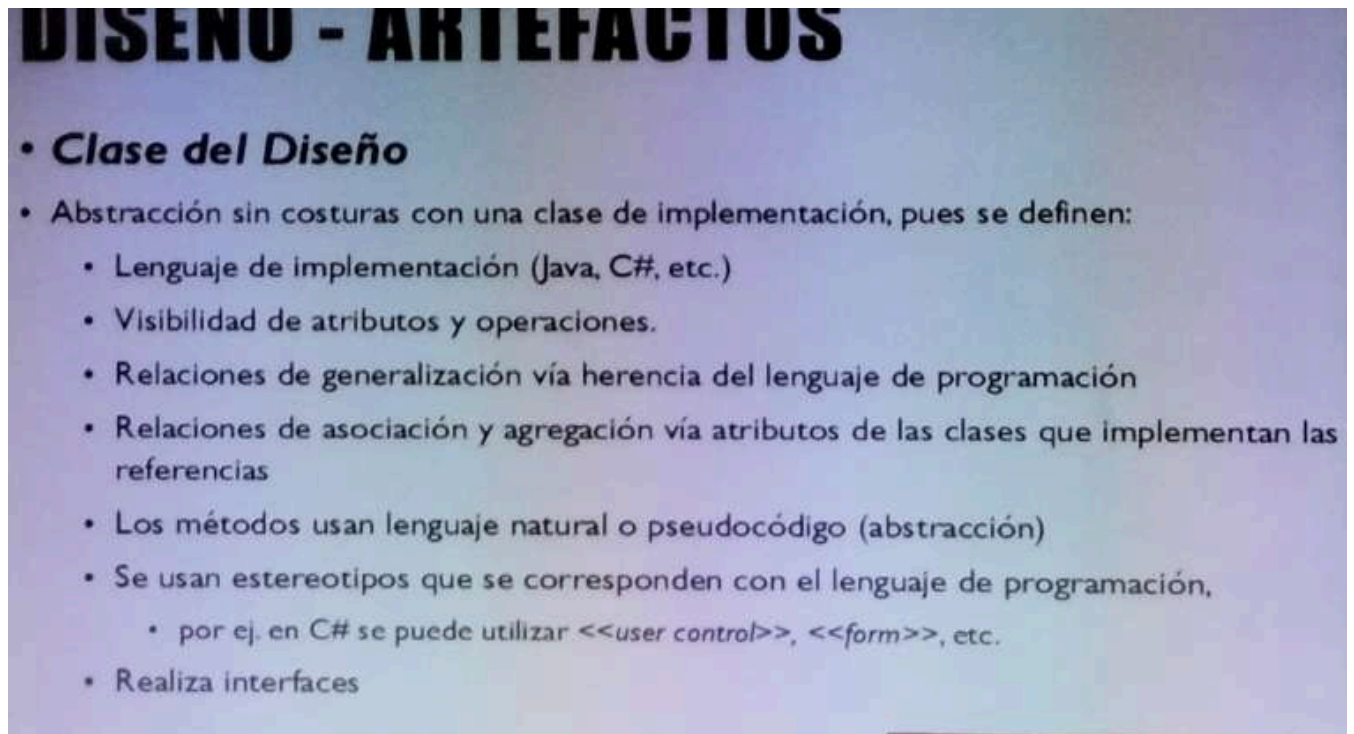
DISEÑO - ARTEFACTOS

- **Modelo del Diseño**

The diagram illustrates the structure of the Design Model artifact. It consists of the following components and relationships:

- Design Model**: Represented by a folder icon, it is the root artifact.
- Design System**: Represented by a folder icon, it is contained within the Design Model (indicated by a solid line with a filled diamond at the Design Model end and a multiplicity of 1).
- Design Subsystem**: Represented by a folder icon, it is contained within the Design System (indicated by a solid line with a filled diamond at the Design System end and a multiplicity of *).
- Design Class**: Represented by a class icon, it is contained within the Design Subsystem (indicated by a solid line with a filled diamond at the Design Subsystem end and a multiplicity of *).
- Use-Case Realization—Design**: Represented by a dashed circle icon, it is contained within the Design Subsystem (indicated by a solid line with a filled diamond at the Design Subsystem end and a multiplicity of *).
- Interface**: Represented by a circle icon, it is contained within the Design Subsystem (indicated by a solid line with a filled diamond at the Design Subsystem end and a multiplicity of *).

- Modelo de objetos que describe la **realización física** de los casos de uso.
- Las abstracciones de este modelo tienen una **correspondencia directa** con los elementos físicos del ambiente de **implementación**.
- Debe mantenerse durante el ciclo de vida



-los atributos van a estar vinculados ya con el lenguaje de implementación

-acá también se agrega la visibilidad, que no exige la etapa de análisis

-en la generalización puede pasar que el lenguaje no contempla herencia múltiple

-Puede ser conveniente que todas las relaciones las referencia en forma de atributos a ciertas clases. (En la práctica hay que ver como se maneja esto)

-no se llegan a implementar los métodos

-hay que saber diferenciar una clase de análisis de una clase de diseño

DISEÑO - ARTEFACTOS

- **Realización de caso de uso-diseño**
- Es una colaboración en términos de clases de diseño que describe como se realiza un caso de uso específico.
- Tiene una traza directa a la correspondiente realización del caso de uso análisis.
 - O con la del modelo de caso de uso en su defecto
- Se describe utilizando:
 - Diagramas de clases
 - Diagramas de interacción
 - Flujos de sucesos-diseño

-los diagramas de interacción se van a basar más en los **diagramas de secuencia**

-el PU recomienda que a nivel de ... se utilizan diagramas de secuencia para mostrar la interacción

Una realización de caso de uso-diseño tiene una descripción de flujo de eventos textual, diagramas de clases que muestra sus clases de diseño participantes, y diagramas de interacción que muestran la realización de un flujo o escenario concreto de un caso de uso en términos de interacción entre objetos del diseño (véase Figura 9.7). Si fuera necesario, los diagramas pueden mostrar también los subsistemas e interfaces implicados en la realización de casos de uso (es decir, los subsistemas que contienen las clases participantes del diseño).

Diagrama de clases

Diagramas de Interacción

Flujos de sucesos-diseño

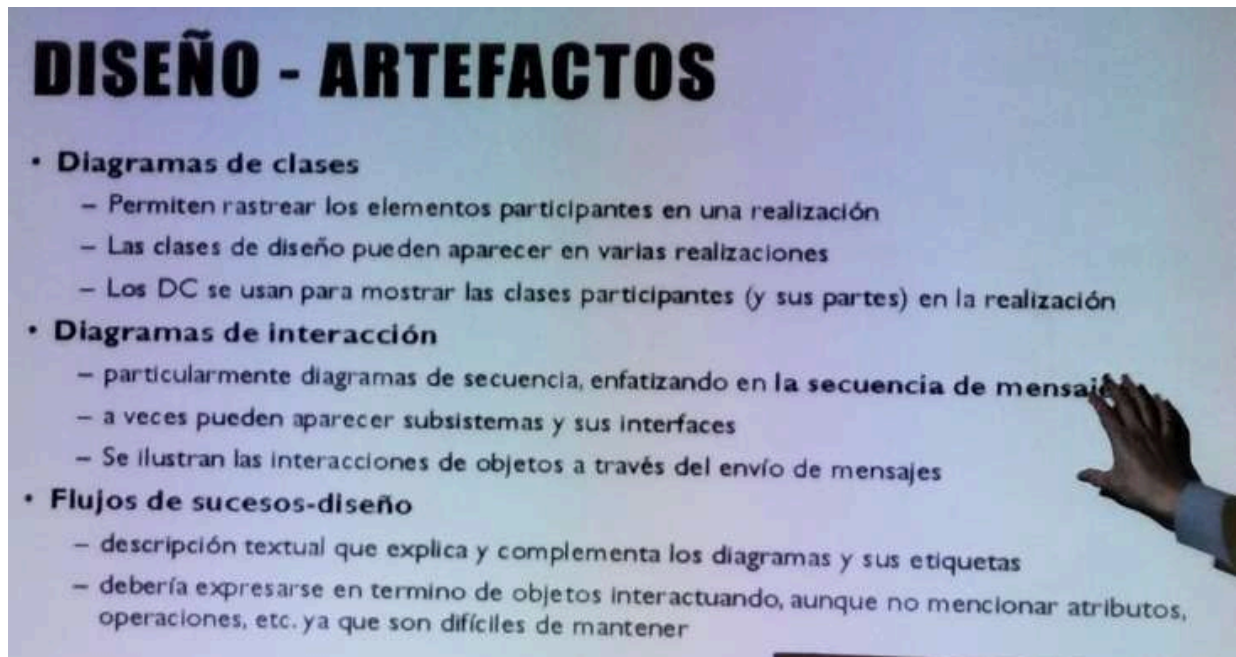


Diagrama de clases

-al final al cabo lo que vamos a tener muchas realizaciones de CU que parecen que no se conectan, cuando son todo lo contrario

-a medida que vamos agregando realizaciones, las clases van tomando mayor relevancia. Ya que van agrupando atributos, métodos y responsabilidades. Puede pasar que al final debamos refactorizar la clase, agregar generalizaciones

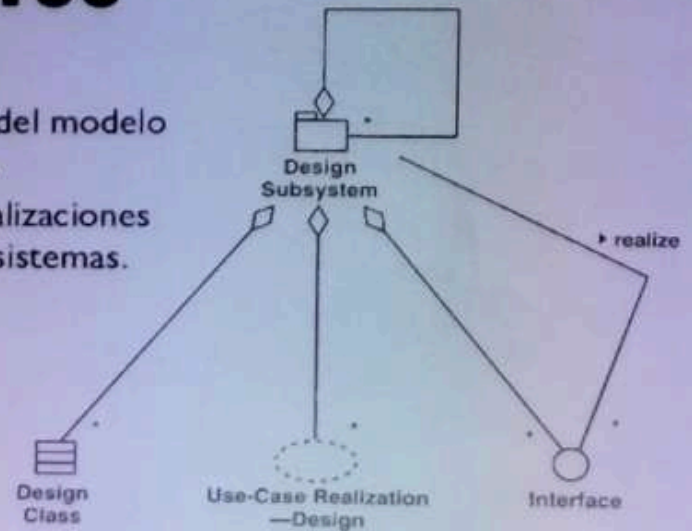
-por cada CU hay al menos un diagrama de clases, y se puede plantear uno gigante (hay que ver si esto sirve)

9.3.3.3. Flujo de sucesos-diseño Los diagramas de una realización de caso de uso, y especialmente los diagramas de interacción, son difíciles de interpretar por sí solos. Por esto, puede ser útil el artefacto flujo-de-sucesos-diseño, que es una descripción textual que explica y complementa a los diagramas y a sus etiquetas. El texto debería redactarse en términos de objetos que interactúan para llevar a cabo el caso de uso, o bien en términos de los subsistemas que participan en él. Sin embargo, la descripción no debería hacer mención de ninguno de los atributos, operaciones y asociaciones de los objetos, ya que ello haría que la descripción fuese difícil de mantener debido a que los atributos, operaciones y asociaciones de las clases de diseño cambian con frecuencia. Además, en la descripción no debería aparecer ninguna operación de interfaz, en caso de que utilizemos interfaces en los diagramas. Mediante esta técnica, minimizamos la necesidad de tener que actualizar las

9.3.3.4. Requisitos de la implementación Los requisitos de la implementación son una descripción textual que recoge requisitos, tales como los requisitos no funcionales, sobre una realización de caso de uso. Nos referimos a requisitos que se capturan sólo en la fase de diseño, pero que es mejor tratar en la implementación. Algunos de estos requisitos pueden haber sido identificados en flujos de trabajo anteriores y por tanto sólo se cambian a una realización de caso de uso. Sin embargo, puede que algunos de ellos sean requisitos nuevos o derivados, que se identifican a medida que avanza el trabajo del diseño.

DISEÑO - ARTEFACTOS

- **Subsistema de diseño**
- **Forma de organizar** los artefactos del modelo de diseño en piezas más **manejables**.
- Puede constar de: clases de diseño, realizaciones de caso de uso, interfaces, y otros subsistemas.
- Cohesivo y débilmente acoplado.



-lo importante es para cada subsistema sea débilmente acoplado y altamente cohesivo

(otro power)

DISEÑO - ARTEFACTOS

- **Subsistema de diseño (continua)**
- Proporcionan **interfaces** que representan la funcionalidad que exportan en término de operaciones.
- Pueden representar separación de un sistema grande en subsistemas que pueden desarrollarse por equipos separados.
- Las dos capas de aplicación de nivel más alto suelen tener trazas directas hacia paquetes y/o clases del análisis.
- Pueden representar productos software reutilizados.
- Pueden representar **sistemas heredados** encapsulándolos.

Subsistema de Servicio

DISEÑO - ARTEFACTOS

- **Subsistemas de Servicio**
 - Se usan en un nivel inferior de la jerarquía de subsistemas.
 - La identificación de subsistemas de servicio se basa en los **paquetes de servicio** del modelo de análisis, y normalmente existe una traza uno a uno.
 - Ofrece servicios en término de interfaces y operaciones.
 - Contiene clases de diseño
 - Suele dar lugar a un componente ejecutable en la implementación.

Interfaz

DISEÑO - ARTEFACTOS

- **Interfaz**
 - Se utilizan para especificar las operaciones que proporcionan las clases y los subsistemas del diseño
 - Se dice que una clase de diseño o un subsistema de diseño "realizan" o implementan una interfaz
 - Separan la especificación de las implementaciones
 - Fácil substitución de implementaciones
 - **Casi todas las interfaces entre subsistemas son arquitecturalmente importantes**

```
graph LR; Interface((Interface)) -- "realize" --> DesignClass[Design Class]; Interface -- "realize" --> DesignSubsystem[Design Subsystem];
```

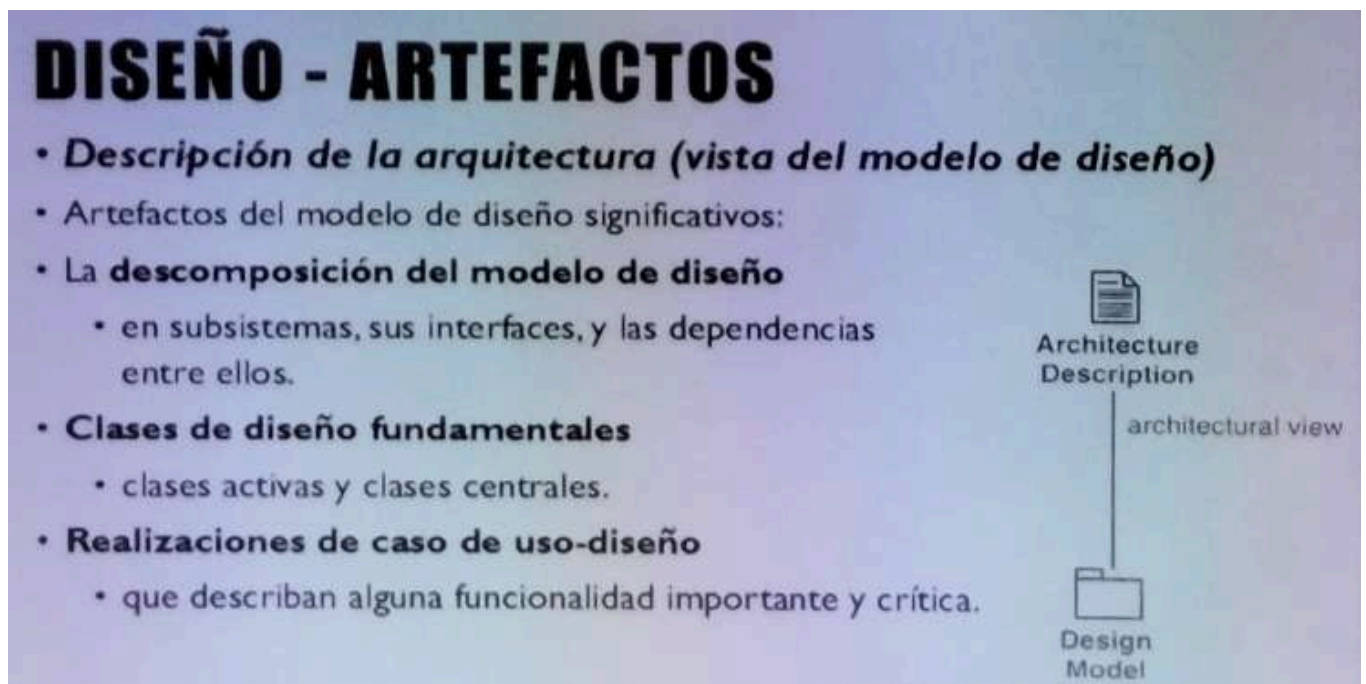
The diagram shows an 'Interface' represented by a circle with a horizontal line. Two arrows point from it to the right. The top arrow points to a 'Design Class' represented by a rectangle with three horizontal lines, and the bottom arrow points to a 'Design Subsystem' represented by a rectangle with a folded top-right corner. Both arrows are labeled 'realize' with a small arrowhead pointing towards the interface.

-es la que pone límites

-hace que un subsistema sea reemplazado por otro que implementa la misma interfaz

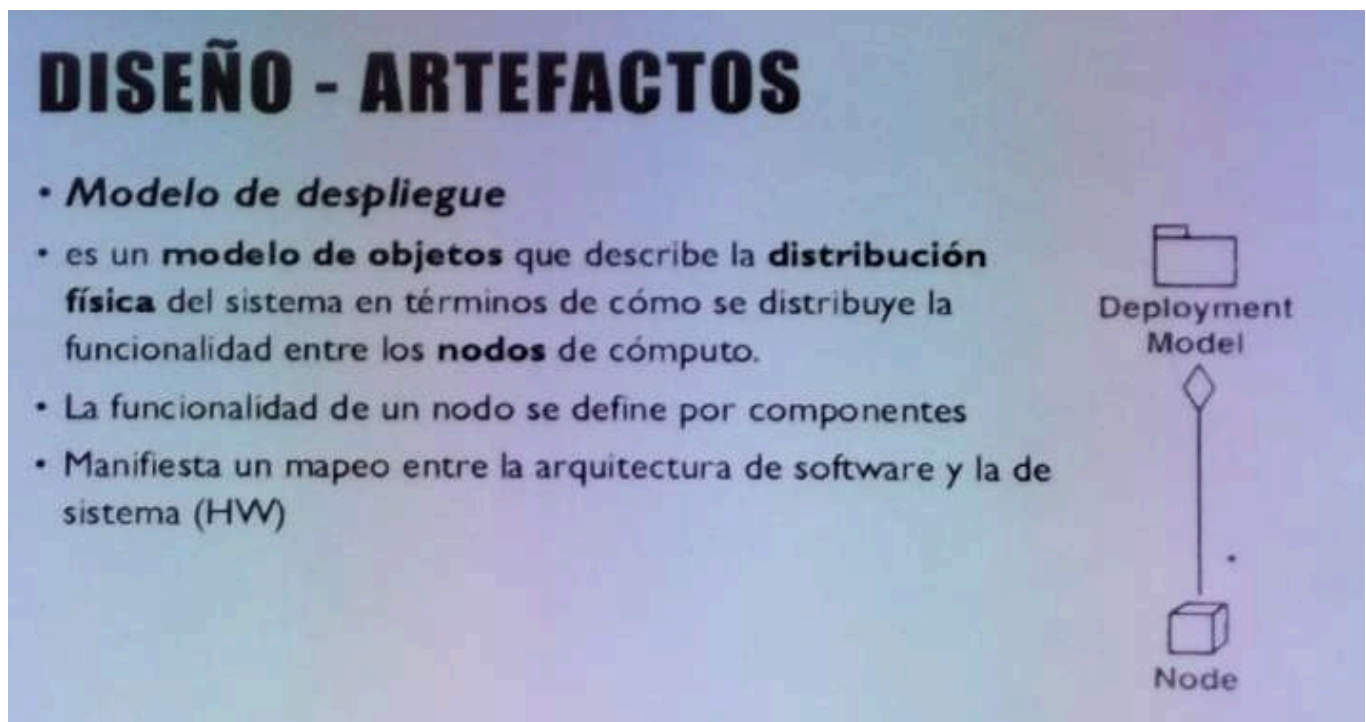
-(último punto) en la vista de diseño casi siempre van aparecer todas las interfaces

Descripción de la arquitectura (vista del modelo de diseño)



(la paso de largo nomas a la diapo)

Modelo de despliegue



-se utilizan diagramas de despliegue

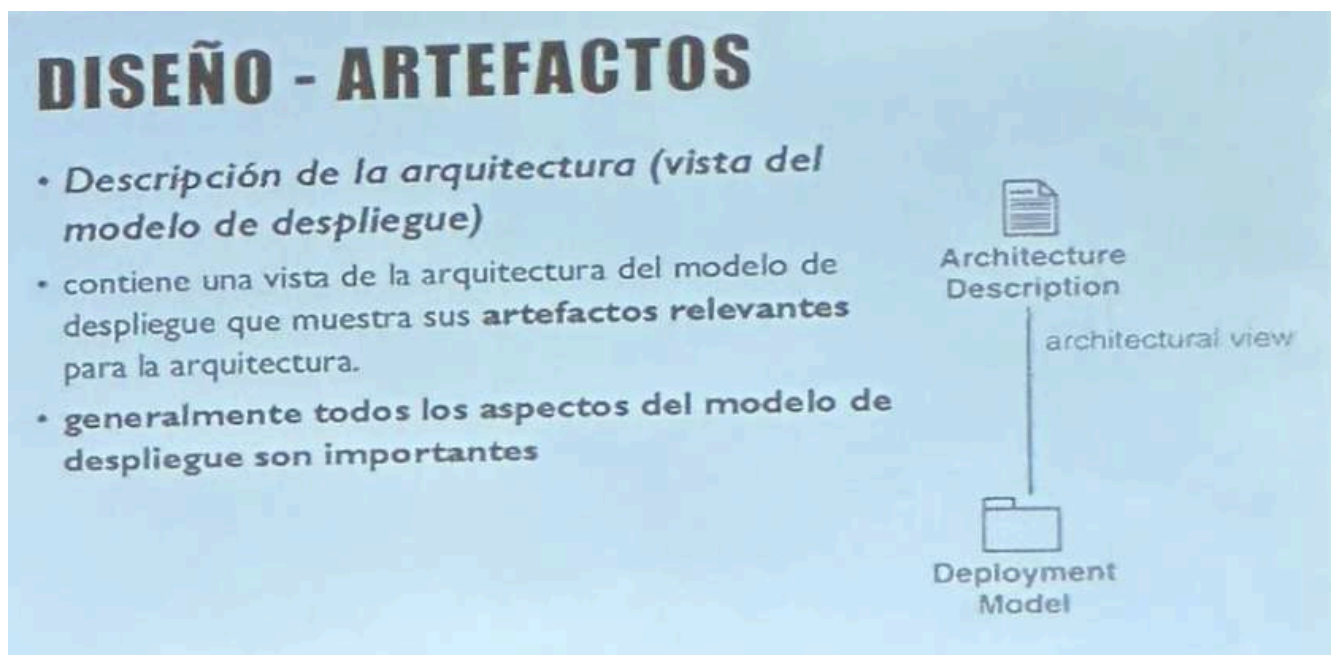
-suele ser difícil definir qué es lo importante desde el punto de vista de la arquitectura

-lo que también va casi completo es el modelo de despliegue, es importante arquitecturalmente

Podemos observar lo siguiente sobre el modelo de despliegue:

- Cada nodo representa un recurso de cómputo, normalmente un procesador o un dispositivo hardware similar.
- Los nodos poseen relaciones que representan medios de comunicación entre ellos, tales como *Internet*, *intranet*, *bus*, y similares.
- El modelo de despliegue puede describir diferentes configuraciones de red, incluidas las configuraciones para pruebas y para simulación.
- La funcionalidad (los procesos) de un nodo se definen por los componentes que se distribuyen sobre ese nodo.
- El modelo de despliegue en sí mismo representa una correspondencia entre la arquitectura *software* y la arquitectura *del sistema* (el hardware).

Descripción de la arquitectura (vista del modelo de despliegue)



Debido a su importancia, deberían mostrarse todos los aspectos del modelo de despliegue en la vista arquitectónica, incluyendo la correspondencia de los componentes sobre los nodos tal como se identificó durante la implementación.

DISEÑO - TRABAJADORES

- **Arquitecto**

- Responsable del modelo de diseño, modelo de despliegue, y descripción de la arquitectura.
- Integridad de los modelos completos, no sus componentes.

- **Ingeniero de casos de uso**

- Responsable de que:
 - la realización de caso de uso – diseño cumpla con los requerimientos,
 - las descripciones textuales y sus diagramas sean legibles y acordes con su objetivo.

- **Ingeniero de componentes**

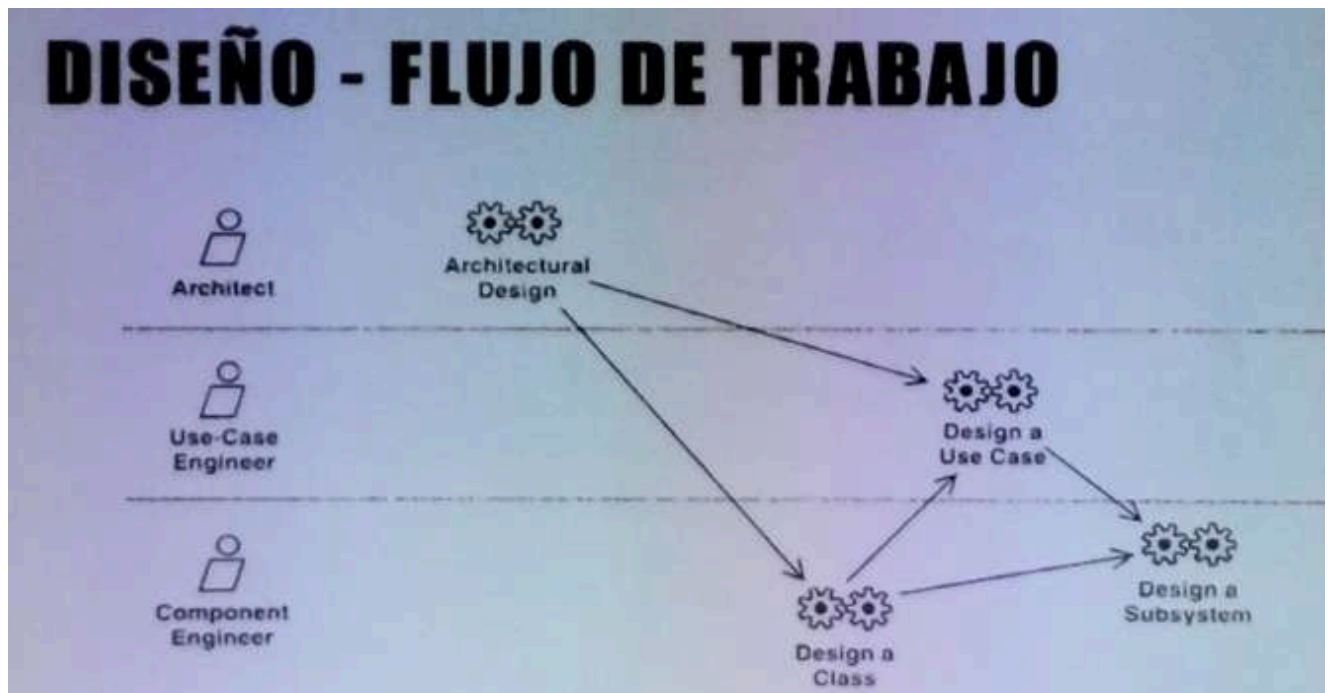
- Responsable de:
 - La mantención y definición clases de diseño, subsistema de diseño, interfaz.
 - Recomendable que el mismo ingeniero se encargue de la implementación

Los modelos son correctos cuando realizan la funcionalidad, y sólo la funcionalidad, descrita en el modelo de casos de uso, en los requisitos adicionales, y en el modelo de análisis.

El ingeniero de componentes define y mantiene las operaciones, métodos, atributos, relaciones y requisitos de implementación de una o más clases del diseño, garantizando que cada clase del diseño cumple los requisitos que se esperan de ella según las realizaciones de caso de uso en las que participa (véase la Figura 9.15).

El ingeniero de componentes puede mantener también la integridad de uno o más subsistemas. Esto incluye garantizar que sus contenidos (clases y sus relaciones) son correctos, que las dependencias de otros subsistemas y/o interfaces son correctas y mínimas, y que realizan correctamente las interfaces que ofrecen.

FLUJO DE TRABAJO - DISEÑO



-es muy parecido a lo de análisis, solo que es mucho más específico

Los arquitectos inician la creación de los modelos de diseño y de despliegue (tal y como se han definido anteriormente en este capítulo). Ellos esbozan los nodos del modelo de despliegue, los subsistemas principales y sus interfaces, las clases del diseño importantes como las activas, y los mecanismos genéricos de diseño del modelo de diseño. Después, los ingenieros de casos de uso realizan cada caso de uso en términos de clases y/o subsistemas del diseño participantes y sus interfaces. Las realizaciones de caso de uso resultantes establecen los requisitos de comportamiento para cada clase o subsistema que participe en alguna realización de caso de uso. Los ingenieros de componentes especifican a continuación los requisitos, y los integran dentro de cada clase, bien mediante la creación de operaciones, atributos y relaciones consistentes sobre cada clase, o bien mediante la creación de operaciones consistentes en cada interfaz que proporcione el subsistema. A lo largo del flujo de trabajo del diseño, los desarrolladores identificarán, a medida que evolucione el diseño, nuevos candidatos para ser subsistemas, interfaces, clases y mecanismos de diseño genéricos, y los ingenieros de componentes responsables de los subsistemas individuales deberán refinarlos y mantenerlos.

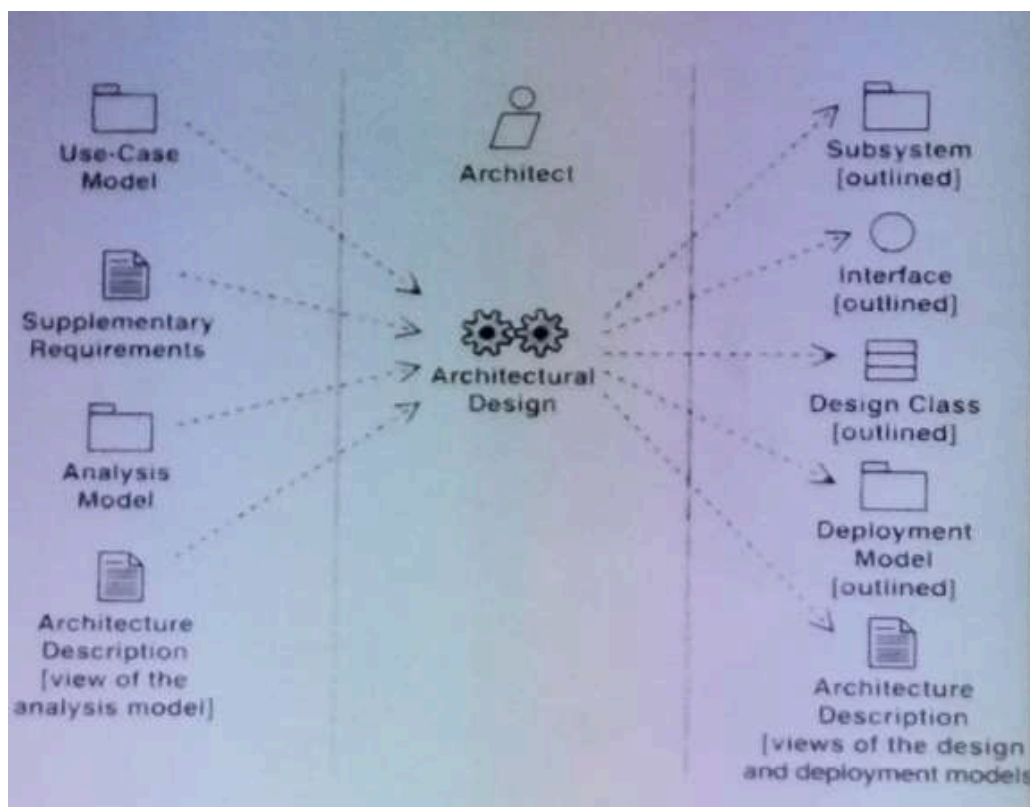
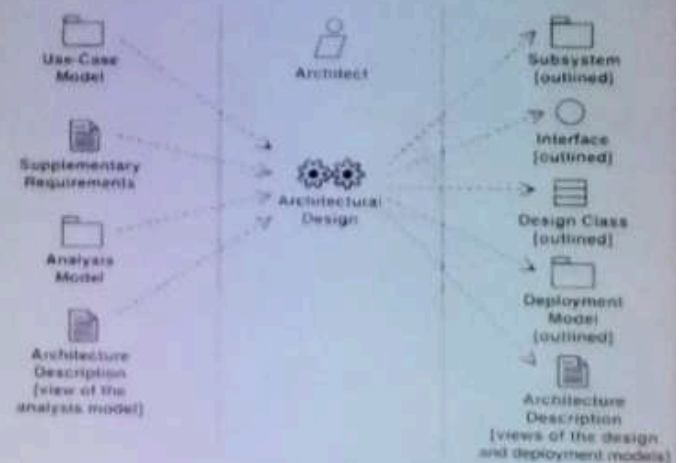
1. Diseño de la arquitectura

DISEÑO - FLUJO DE TRABAJO

1. Diseño de la arquitectura (act.)

- Esbozar los modelos de diseño y despliegue **identificando** (subactividades):

1. Nodos y configuraciones de red
2. Subsistemas y sus interfaces
3. Clases de diseño significativas
4. Mecanismos de diseño genéricos



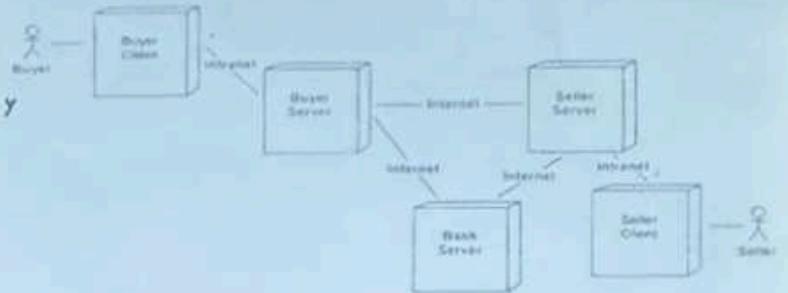
- todo lo que sale acá es bosquejado, nada es a detalle. Es un bosquejo de alto nivel
- subactividades dentro de la actividad principal
- mecanismos de diseño genérico, es algo que no se tiene en cuenta. Que tiene que ver con cuestiones tan importantes como es la persistencia
- podemos decir que es Desacuerdo de la independencia, cuando estoy tratando de guardar lo que esta OO en una base de datos por ejemplo
- diseño de la arquitectura parte **fundamental**

1.1 Identificar nodos y configuraciones de red

DISEÑO - FLUJO DE TRABAJO

1.1 Identificar nodos y configuraciones de red

- La configuración física tiene gran impacto en la arquitectura de SVV (3 capas, cliente/servidor)
- Incluye:
 - Nodos involucrados y su capacidad
 - Tipos de conexiones, sus características y protocolos
 - Procesamiento redundante, backup, etc



DISEÑO - FLUJO DE TRABAJO

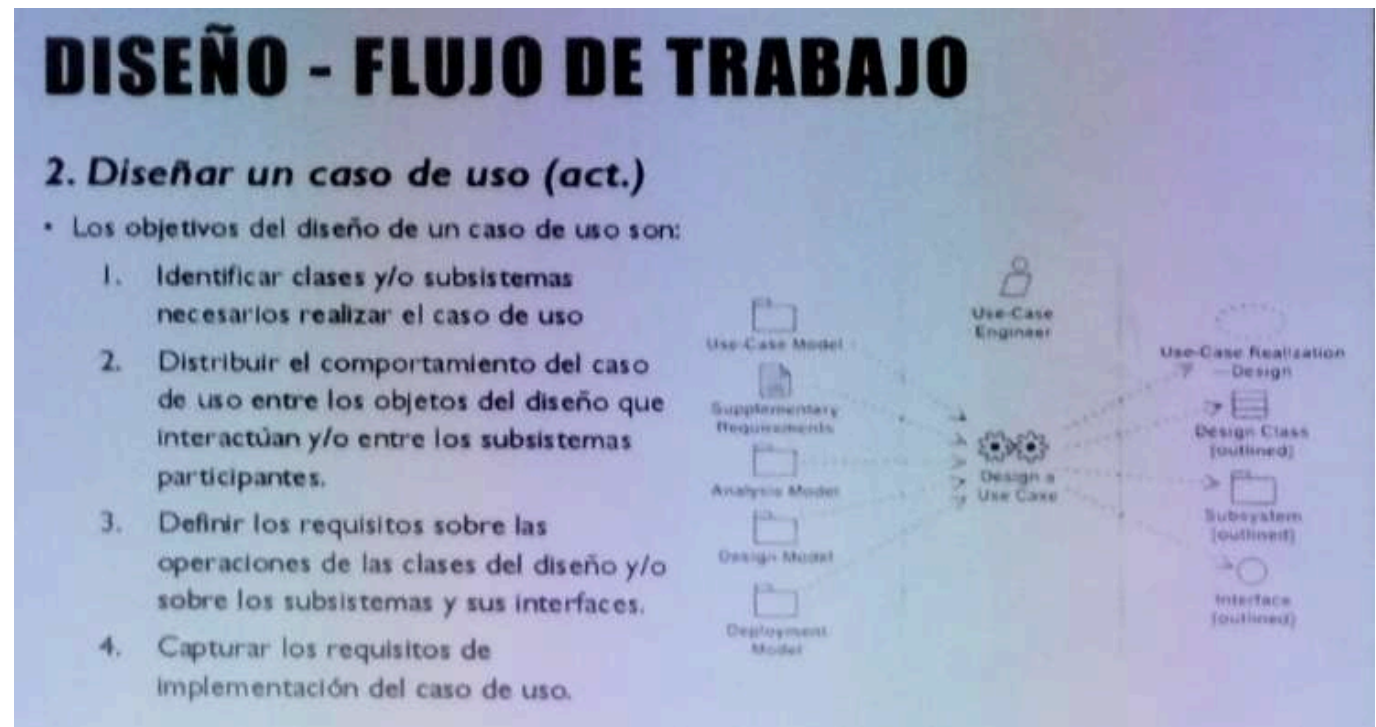
1.2.3 Definir las dependencias de subsistemas

- Si sus contenidos están relacionados
- Si existen interfaces las dependencias deberían ser contra ellas y no con los subsistemas

1.2.4 Identificar interfaces de subsistemas

- Las interfaces definen operaciones que son accesibles desde fuera del subsistema
- Inicialmente se consideran las dependencias como el punto anterior, probablemente necesite una interfaz
- También se deben identificar las operaciones

2. Diseñar un caso de uso



-se hace esa realización de manera análoga pero con otro nivel de detalle

(Se le agrega como entrada modelo de diseño y despliegue, se saca la descripción de la arquitectura)

2.1 Identificación de clases del diseño participantes En este paso

2.2 Descripción de las interacciones entre objetos del diseño

2.3 Identificación de subsistemas e interfaces participantes

2.4. Descripción de interacciones entre subsistemas

2.5. Captura de requisitos de implementación

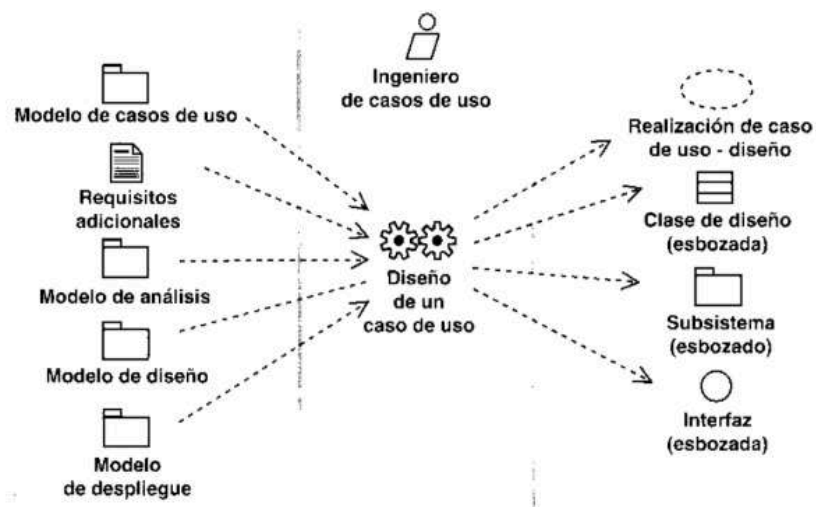
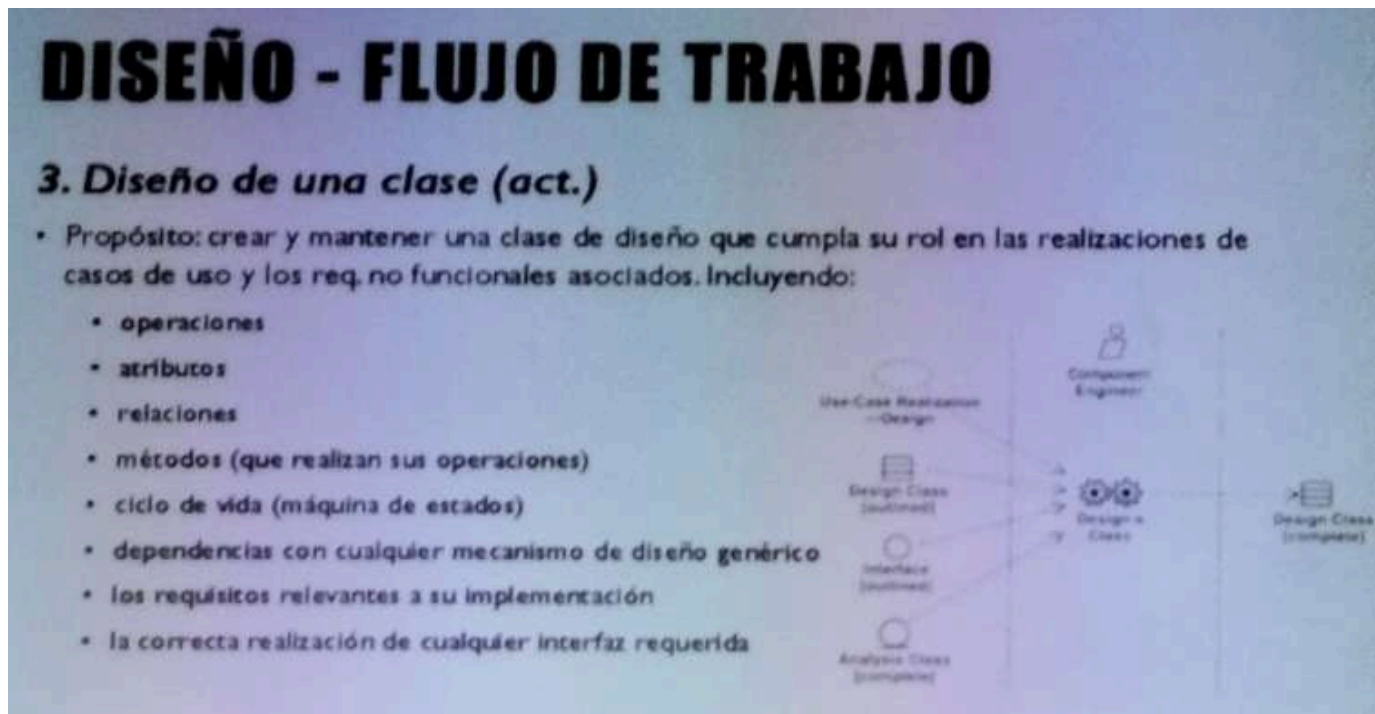


Figura 9.34. Las entradas y los resultados del diseño de un caso de uso. La correspondiente realización de caso de uso-análisis es una entrada fundamental para esta actividad.

3. Diseño de una clase



- cuando hablamos de análisis no nos interesaban las operaciones
- cuando hablamos de diseño todas las operaciones deben estar definidas
- forma parte principal del diseño, el modelado del ciclo de vida. Podríamos a cada clase hacer el Diagrama de máquina de estados
- si o si debe aparecer la especificación de las interfaces. Si dijimos que una clase implementa una interfaz, corroborar que implemente todos los métodos que propone esa interfaz
- tenemos muchas entradas pero la salida que tenemos aca, es la clase por completo

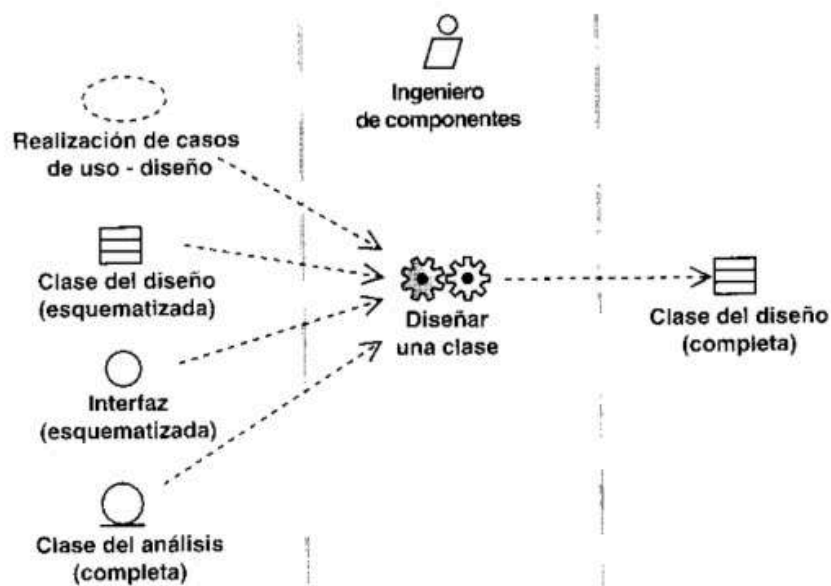


Figura 9.39. La entrada y los resultados del diseño de una clase.

3.1 Elaborar las clases de diseño

DISEÑO - FLUJO DE TRABAJO

3.1 Elaborar las clases de diseño

- Desde una interfaz: se asigna una clase que la implemente.
- Desde una o varias clases de análisis, depende del estereotipo:
 - **Frontend**: depende de la tecnología de interfaces, por ej. «form» para Windows Forms o «applet» para applets.
 - **Entidad información** que debe almacenarse. Por ej. mapeo de clases de diseño a tablas. DBMS específicos.
 - **Control** maneja secuencias, coordinación de objetos y lógica de negocio, considerando aspectos de distribución, rendimiento y transacciones.
 - Distribución: reparte clases o le asigna a diferentes nodos.
 - Rendimiento: puede ser necesario trabajar de junto con una clase frontend o de entidad.
 - Transacciones: requiere la tecnología correspondiente.

3.2 Identificar operaciones

DISEÑO - FLUJO DE TRABAJO

3.2 Identificar operaciones

- Identificar operaciones que requieran ser programadas usando un lenguaje de programación.
- Se incluye entidad.
- Entidades importantes:
 - Responsabilidades de clases de análisis, vistas, operaciones.
 - Requisitos especiales.
 - Interfaces a proveer.
 - Realizaciones de uso de una clase en las que participa.
- Debe reportar todos los roles jugados en las realizaciones.

3.3 Identificar atributos

DISEÑO - FLUJO DE TRABAJO

3.3 Identificar atributos

- Identificar atributos que necesitan ser provistos usando un lenguaje de programación
- Considerar atributos de clases de análisis, pueden convertirse en varios
- Tipos según lenguaje
- No existen atributos compartidos (posible clase)
- Si la clase es compleja, dividirla.
- Podrían usarse diagramas de clase separados para mostrar solo los atributos

3.4 Identificar asociaciones y agregaciones

DISEÑO - FLUJO DE TRABAJO

3.4 Identificar asociaciones y agregaciones

- El ingeniero de componentes debería revisar las interacciones
- Pueden usarse para mantener referencias a otros objetos
- Mezclar relaciones, usar las necesarias para las realizaciones (no mundo real)
- Considerar performance (rutas óptimas de búsqueda)
- Considerar asociaciones de análisis (podría ser uno a muchos)
- Refinar multiplicidades, roles, clases de asociaciones de acuerdo con el lenguaje de programación (las clases de asociación podrían convertirse en clase comunes, cambiar los roles a atributos)
- Refinar navegabilidad de acuerdo con las interacciones (envío de mensajes)

3.5 Identificar generalizaciones

DISEÑO - FLUJO DE TRABAJO

3.5 Identificar generalizaciones

- Acordadas de acuerdo con el lenguaje
- Se puede cambiar incluso a asociaciones o agregaciones

3.6 Describir métodos

DISEÑO - FLUJO DE TRABAJO

3.6 Describir métodos

- Cómo se realizan las operaciones
- Pseudocódigo o lenguaje natural
- Mencionar algún tipo de algoritmo específico
- Generalmente no se especifican en el diseño
- Si hay alguna herramienta de generación de código podría hacerse en diseño pero se considera de implementación

3.7 Describir estados

DISEÑO - FLUJO DE TRABAJO

3.7 Describir estados

- Algunos objetos son estado-dependiente (o estado controlado)
- Su comportamiento se define por el estado en el que se encuentra al recibir el mensaje
- Usar un diagrama de estados

```
graph TD; Start(( )) --> Crear[Crear]; Crear --> Procesar[Procesar]; Procesar --> Validar[Validar]; Validar --> Ejecutar[Ejecutar]; Ejecutar --> Terminar[Terminar]; Terminar --> End((( ))); Validar -- "Mensaje de retroalimentación" --> Ejecutar;
```

3.8 Manejar requerimientos especiales

DISEÑO - FLUJO DE TRABAJO

3.8 Manejar requerimientos especiales

- Se consideran requerimientos no tenidos en cuenta en los pasos anteriores
- No funcionales
- Usar los mecanismos propuestos por el arquitecto



FIGURE 5.43: Objects of the model class must be substituted. This is done by making them subtypes of the model class.

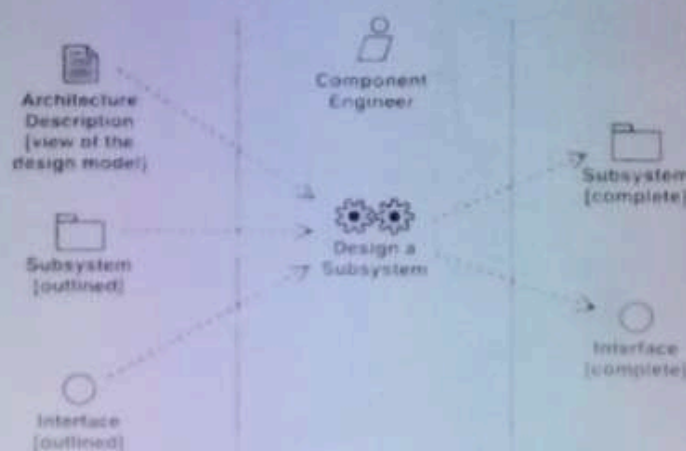
4. Diseño de un subsistema

DISEÑO - FLUJO DE TRABAJO

4. Diseño de un subsistema (act.)

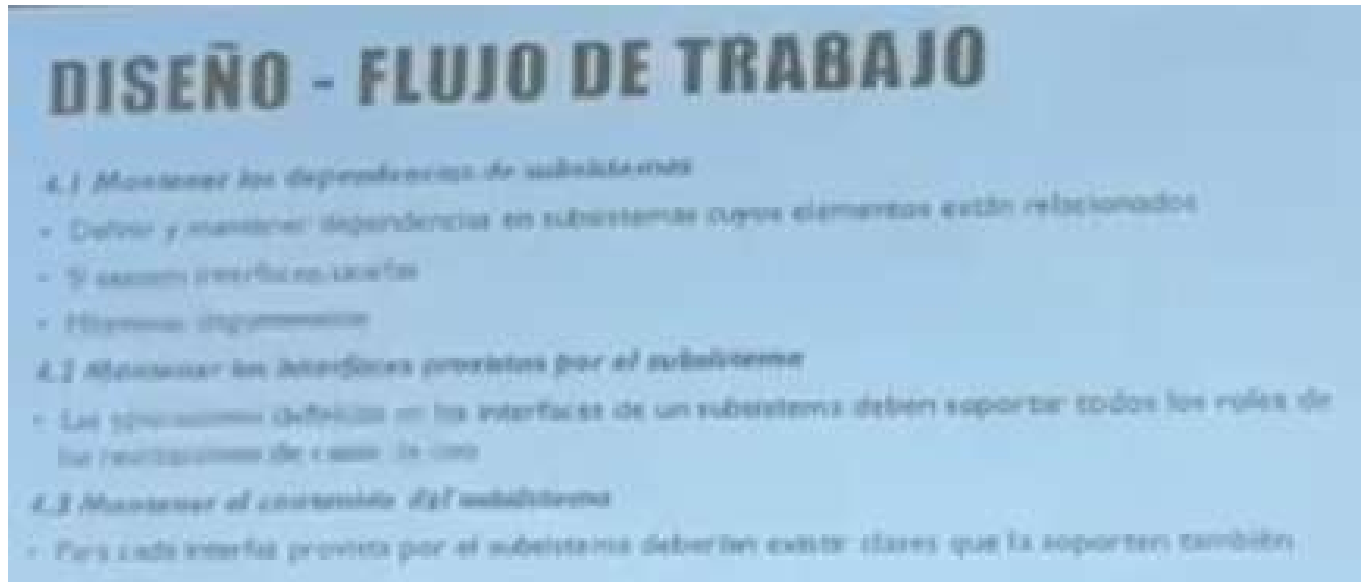
Garantizar que el subsistema:

- es lo más independiente de los otros subsistemas.
- proporciona las interfaces correctas.
- cumple su propósito de ofrecer una realización correcta de las operaciones que se definen en las interfaces.



-no es el subsistema directamente quien lo implementa sino las clases

- yo vengo con un conjunto de paquetes del análisis, yo debo pensar en lo siguiente. Como físicamente yo voy a implementar esos subsistemas, por alguna razón esos subsistemas pueden dividirse en varios
- debo ver bien que tenga las clases adecuadas, que además esas clases esten implementando correctamente esas interfaces
- es un paso más de control



(pretendemos que sepan cada paso, no cada subpaso que tienen estos)

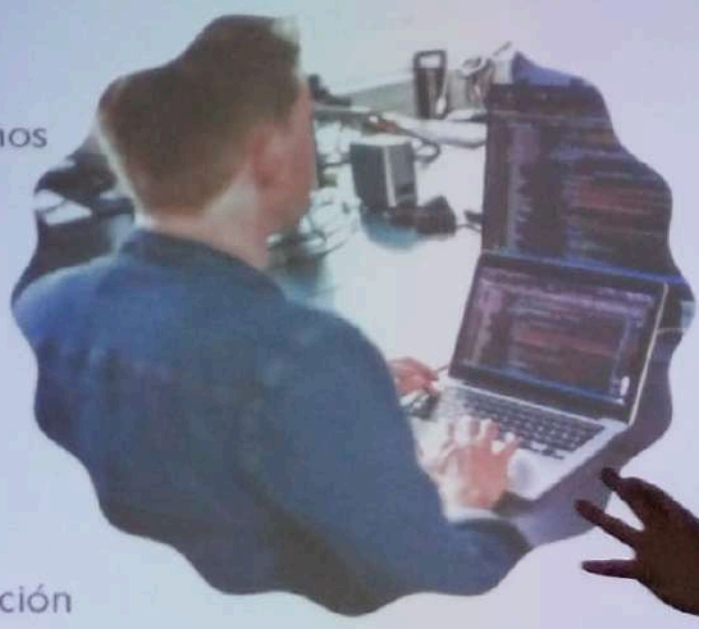
- 4.1. Mantenimiento de las dependencias entre subsistemas
- 4.2. Mantenimiento de interfaces proporcionadas por el subsistema
- 4.3. Mantenimiento de los contenidos de los subsistemas

CLASE - 15/10

IMPLEMENTACIÓN

IMPLEMENTACIÓN

- Implementa el sistema en términos de **componentes/artefactos**:
- ficheros de código fuente, scripts, ficheros de código binario, ejecutables, y similares.
- “Agrega carne al esqueleto” de la arquitectura
- Foco de atención en la construcción

A photograph of a person from behind, sitting at a desk and working on a laptop. The laptop screen shows code. To the right of the laptop, there is a large, curved screen or monitor displaying more code. The person is wearing a blue shirt. The background is a light blue wall.

PROPÓSITOS - IMPLEMENTACIÓN

IMPLEMENTACIÓN - PROPÓSITOS

- Planear las **integraciones** requeridas en cada iteración
- Distribuir el sistema mapeando **componentes ejecutables a nodos**
- **Implementar** las clases y subsistemas encontrados en el diseño
- Realizar **pruebas unitarias** e integrar

-Etapa de construcción

-Plan de integración: decidir qué funcionalidad se va a hacer ejecutable en ese momento

-Componentes físicos o artefactos

-En la etapa implementación nos centramos en la codificación

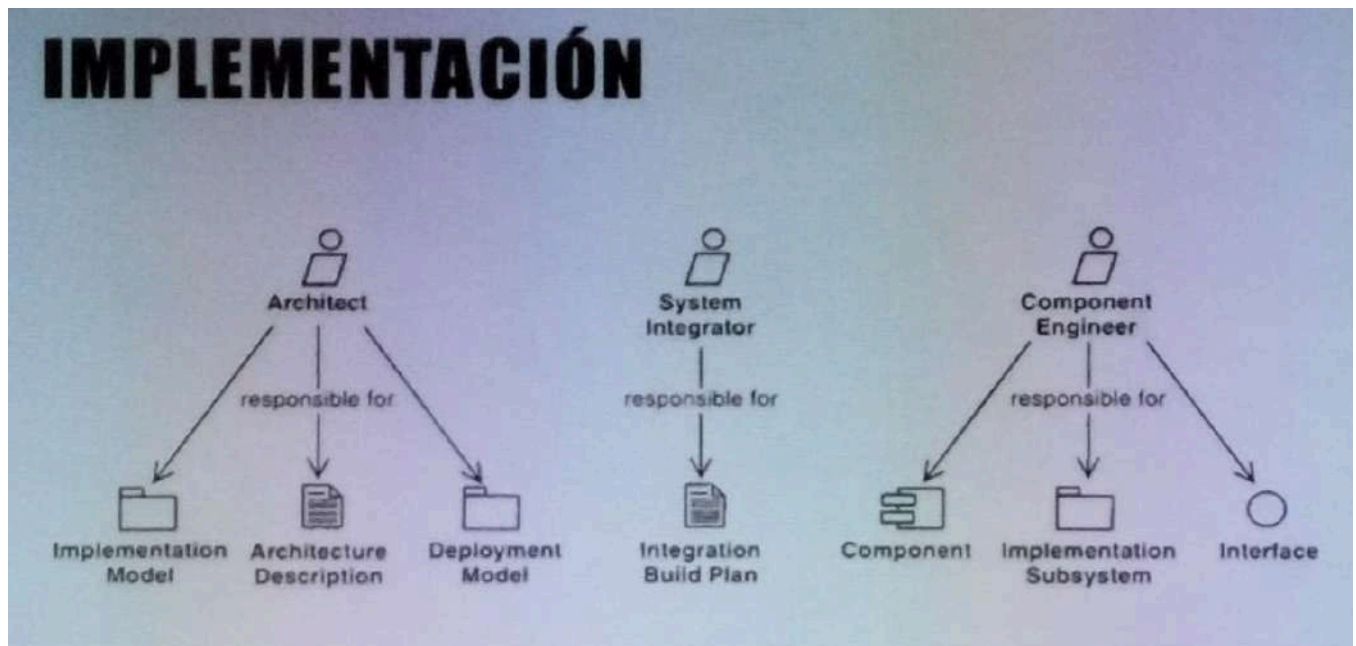
-En esta disciplina se realizan las **pruebas unitarias**, no en la etapa de prueba (evitar esa confusión)

Afortunadamente, la mayor parte de la arquitectura del sistema es capturada durante el diseño, siendo el propósito principal de la implementación el desarrollar la arquitectura y el sistema como un todo. De forma más específica, los propósitos de la implementación son:

- Planificar las integraciones de sistema necesarias en cada iteración. Seguimos para ello un enfoque incremental, lo que da lugar a un sistema que se implementa en una sucesión de pasos pequeños y manejables.
- Distribuir el sistema asignando componentes ejecutables a nodos en el diagrama de despliegue. Esto se basa fundamentalmente en las clases activas encontradas durante el diseño.
- Implementar las clases y subsistemas encontrados durante el diseño. En particular, las clases se implementan como componentes de fichero que contienen código fuente.
- Probar los componentes individualmente, y a continuación integrarlos compilándolos y enlazándolos en uno o más ejecutables, antes de ser enviados para ser integrados y llevar a cabo las comprobaciones de sistema.

La implementación es el centro durante las iteraciones de construcción, aunque también se lleva a cabo trabajo de implementación durante la fase de elaboración, para crear la línea base ejecutable de la arquitectura, y durante la fase de transición, para tratar defectos tardíos como los encontrados con distribuciones beta del sistema (como se indica en la Figura 10.2 con un pico en la columna de transición).

Diagrama

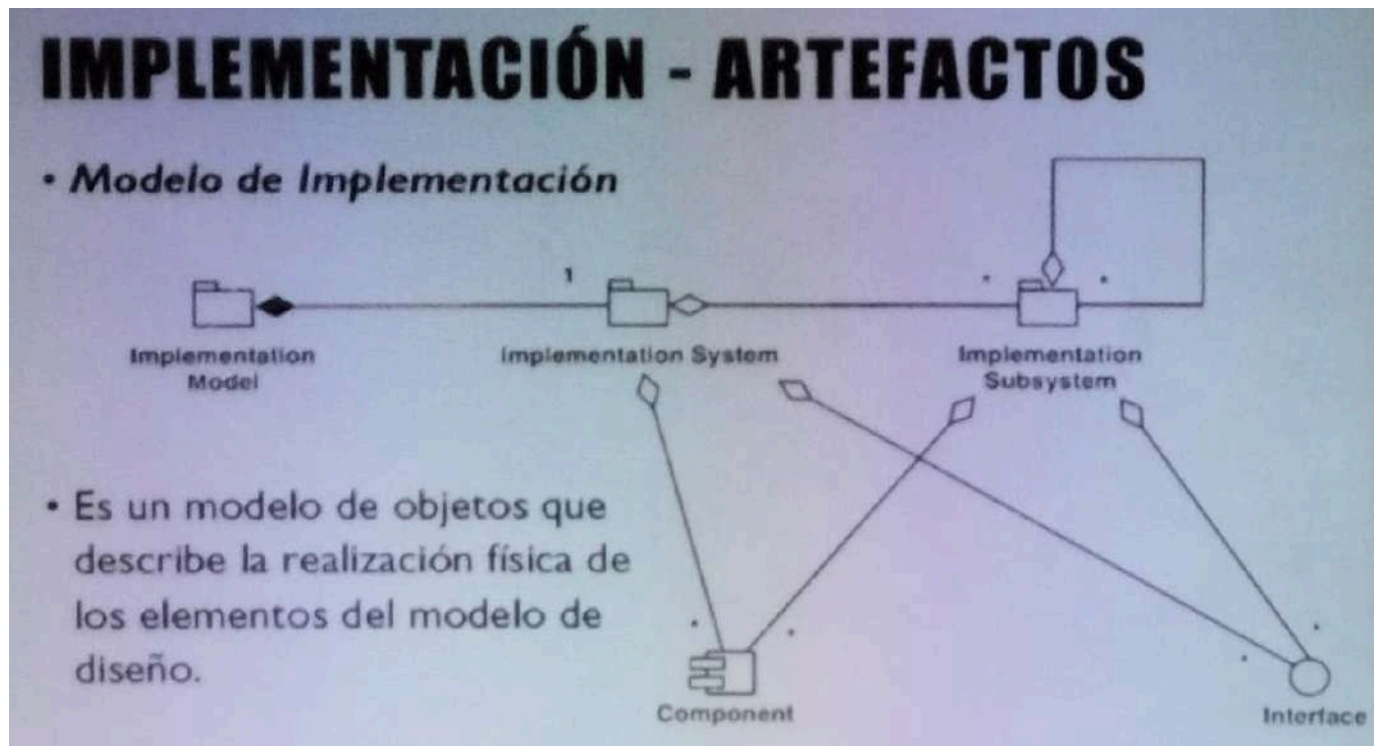


-aparece el trabajador **integrador de Sistemas**

-Tanto en diseño como en implementación aparece el modelo de despliegue, en el primero se plantea y en el segundo se refina.

ARTEFACTOS - IMPLEMENTACIÓN

Modelo de Implementación



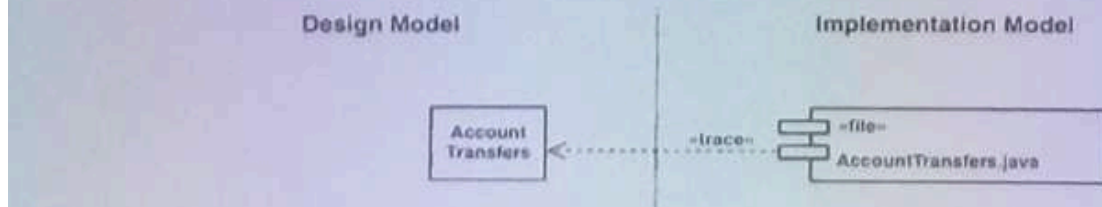
-(hay otros autores que dicen que el modelo de diseño e implementación son el mismo)

El modelo de implementación describe cómo los elementos del modelo de diseño, como las clases, se implementan en términos de componentes, como ficheros de código fuente, ejecutables, etc. El modelo de implementación describe también cómo se organizan los componentes de acuerdo con los mecanismos de estructuración y modularización disponibles en el entorno de implementación y en el lenguaje o lenguajes de programación utilizados, y cómo dependen los componentes unos de otros.

IMPLEMENTACIÓN - ARTEFACTOS

• **Componente (1.X) o Artefacto (2.X)**

- Un componente es el empaquetamiento físico de los elementos de un modelo
 - «executable» programa ejecutable en un nodo
 - «file» fichero que contiene código fuente o datos
 - «library» librería estática o dinámica
 - «table» es una tabla de base de datos
 - «document» es un documento
- Los componentes tienen relaciones de traza con aquello que implementan
- Proveen las mismas interfaces que aquello que implementan



-el concepto de Componente es el que veíamos antes como Artefacto

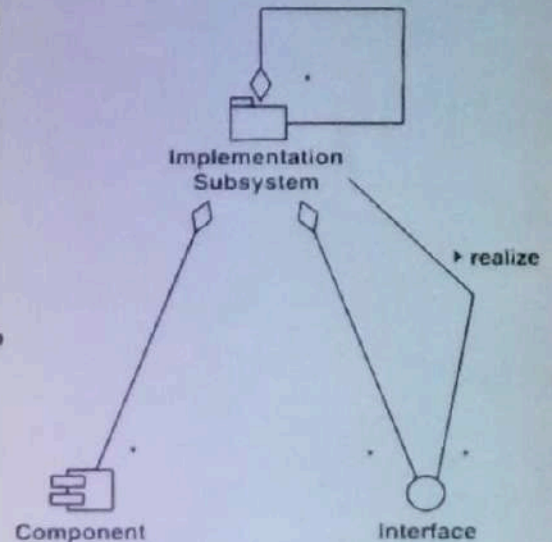
- Es normal que un componente implemente varios elementos, por ejemplo, varias clases; sin embargo, la forma exacta en que se crea esta traza depende de cómo van a ser estructurados y modularizados los ficheros de código fuente, dado el lenguaje de programación que se esté usando.
- Puede haber dependencias de compilación entre componentes, denotando qué componentes son necesarios para compilar un componente determinado.

10.3.2.1. Stubs Un *stub*² es un componente con una implementación esquelética o de propósito especial que puede ser utilizada para desarrollar o probar otro componente que depende del *stub*; una definición similar aparece en [1]. Los *stubs* pueden ser utilizados para minimizar el número de componentes nuevos necesarios en cada nueva versión (intermedia) del sistema, simplificando así los problemas de integración y las pruebas de integración (véase Sección 10.5.2, “Actividad: integrar el sistema”).

IMPLEMENTACIÓN - ARTEFACTOS

• *Subsistema de Implementación*

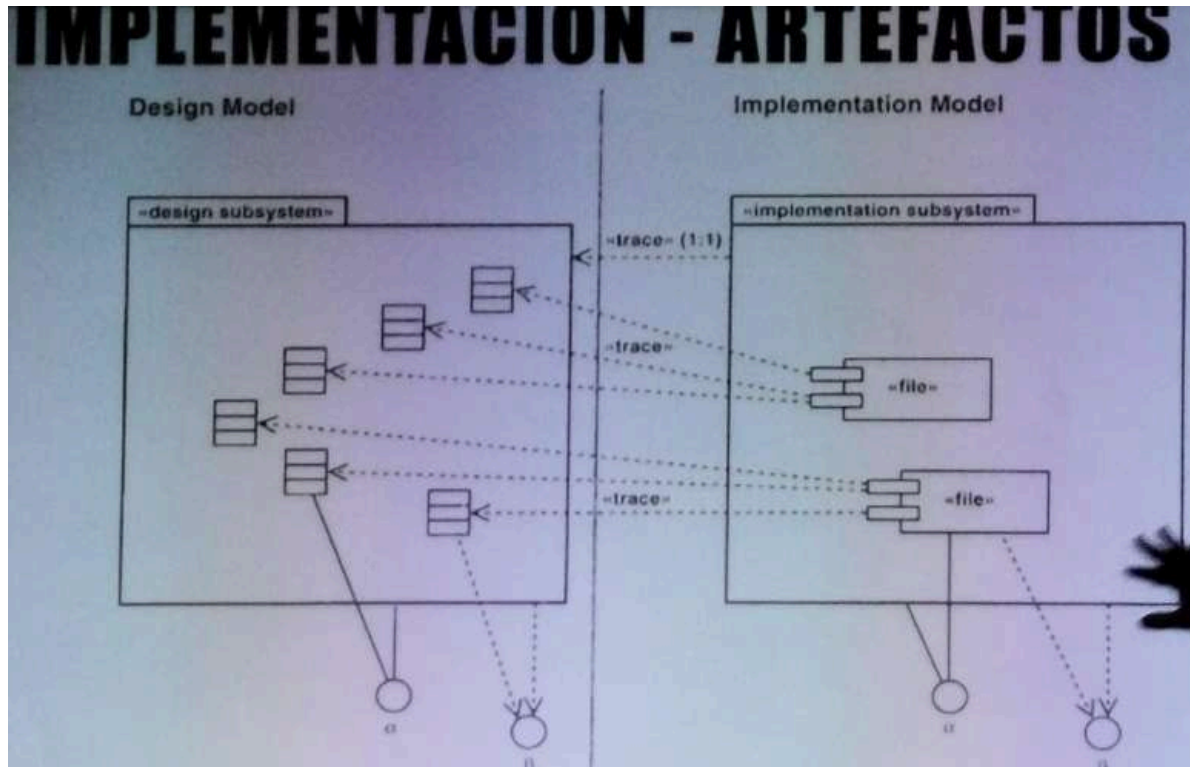
- Agrupamiento físico de elementos del sistema.
- Se manifiesta a través de un mecanismo de empaquetamiento concreto en un entorno de implementación determinado, como:
 - Un paquete en Java
 - Un ensamblado en .NET.
 - Un directorio de ficheros en un proyecto de C++
 - Un paquete en una herramienta de modelado como Rational Rose.
- Tienen una traza uno a uno (isomórfica) con los subsistemas de diseño.



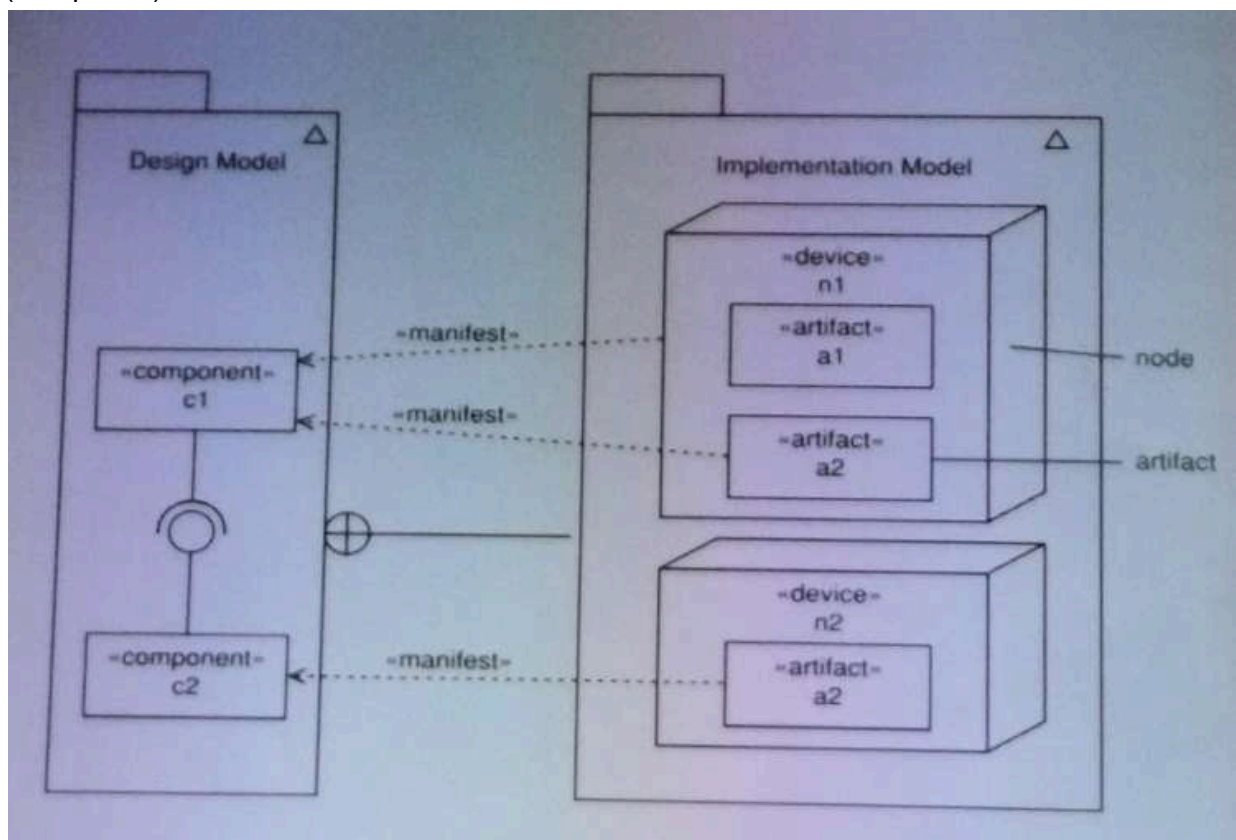
- acá decidimos explícitamente que esto va a estar ubicado en este lugar
- por cada subsistema de diseño voy a tener que tener un subsistema de implementación
- un archivo .java puede estar en un archivo .jar

- El subsistema de implementación debería definir dependencias análogas hacia otros subsistemas de implementación o interfaces (correspondientes).
- El subsistema de implementación debería proporcionar las mismas interfaces.
- El sistema de implementación debería definir qué componentes o, recursivamente, qué otros subsistemas de implementación dentro del subsistema deberían proporcionar las interfaces proporcionadas por el subsistema. Además, estos componentes (o subsistemas de implementación) contenidos deberían seguir la traza de las clases (o subsistemas de diseño) correspondientes en el subsistema de diseño que implementan.

Diagrama - Modelo Diseño y Modelo Implementación



(otro power)



-adentro del modelo de Diseño tenemos Componentes

IMPLEMENTACIÓN - ARTEFACTOS

- *Interfaz*
- Especifican las operaciones implementadas por componentes y subsistemas de implementación.
- Deben ser las mismas que en el modelo de diseño

-**no hay diferencia** entre el diseño e implementación, son las mismas

-el modelo de diseño es prácticamente lo que voy a implementar pero sin el código ni los archivos

IMPLEMENTACIÓN - ARTEFACTOS

- *Descripción de la arquitectura*
- Artefactos son considerados arquitectónicamente significativos:
 - Descomposición del modelo de implementación en subsistemas, sus interfaces, y dependencias entre ellos. (mismos que en diseño)
 - Componentes clave que siguen la traza de las clases de diseño significativas arquitectónicamente.

IMPLEMENTACIÓN - ARTEFACTOS

- **Plan de integración de construcciones**

- Describe la secuencia de **construcciones (builds)** necesarias en una iteración. Para cada una se incluye:
 - Funcionalidad esperada en las construcciones (Casos uso)
 - Qué partes del modelo de implementación son afectadas (Subsistemas y componentes)
- El SW se construye en pasos manejables, cada paso es una construcción (resulta en una pequeña integración o en errores de prueba)
- Una **construcción** es una versión ejecutable del sistema
- Los beneficios de la incrementalidad:
 - Una versión ejecutable puede ser **creada y probada temprano**
 - Los **defectos se detectan más fácilmente** durante la construcción
 - Las pruebas de integración tienden a ser **más profundas**

-para cada construcción vamos a tener, la funcionalidad, los casos de uso que vamos a implementar

-es muy importante el plan porque se va a decidir qué es lo que se va a implementar

3) cada build puede terminar bien o mal, cuando termina bien va a resultar en una pequeña integración. Si resulta mal termina en un error de prueba, ese error de prueba es un Artefacto. Tanto la construcción va a ser un artefacto como el error, el error se va usar como prueba

-hay algunas ventajas de que se haga de manera incremental

La **integración incremental** (apéndice C) es para la integración del sistema lo que el desarrollo iterativo controlado es para el desarrollo de software en general. Ambos se centran en un incremento bastante pequeño y manejable de la funcionalidad.

Un *plan de integración de construcciones* describe la secuencia de construcciones necesarias en una iteración. Más concretamente, un plan de este tipo describe lo siguiente para cada construcción:

- La funcionalidad que se espera que sea implementada en dicha construcción. Consiste en una lista de casos de uso o escenarios o parte de ellos, como se discutió en capítulos anteriores. Esta lista puede también referirse a otros requisitos adicionales.
- Las partes del modelo de implementación que están afectadas por la construcción. Esto es una lista de los subsistemas y componentes necesarios para implementar la funcionalidad supuesta por la construcción.

TRABAJADOR - IMPLEMENTACIÓN

IMPLEMENTACIÓN - TRABAJADORES

• Arquitecto

- Responsable de la integridad del modelo de implementación y asegurar que el modelo como un todo es correcto, completo, y legible.
- Responsable de
 - modelo de implementación
 - modelo de despliegue
 - descripción de la arquitectura.

• Ingeniero de componentes

- Define y mantiene el código fuente de uno o varios componentes, garantizando que el componente implementa la funcionalidad correcta.
- Mantiene la integridad de uno o varios subsistemas de implementación.
- Conviene que el mismo ingeniero sea responsable de los componentes contenidos en un subsistema

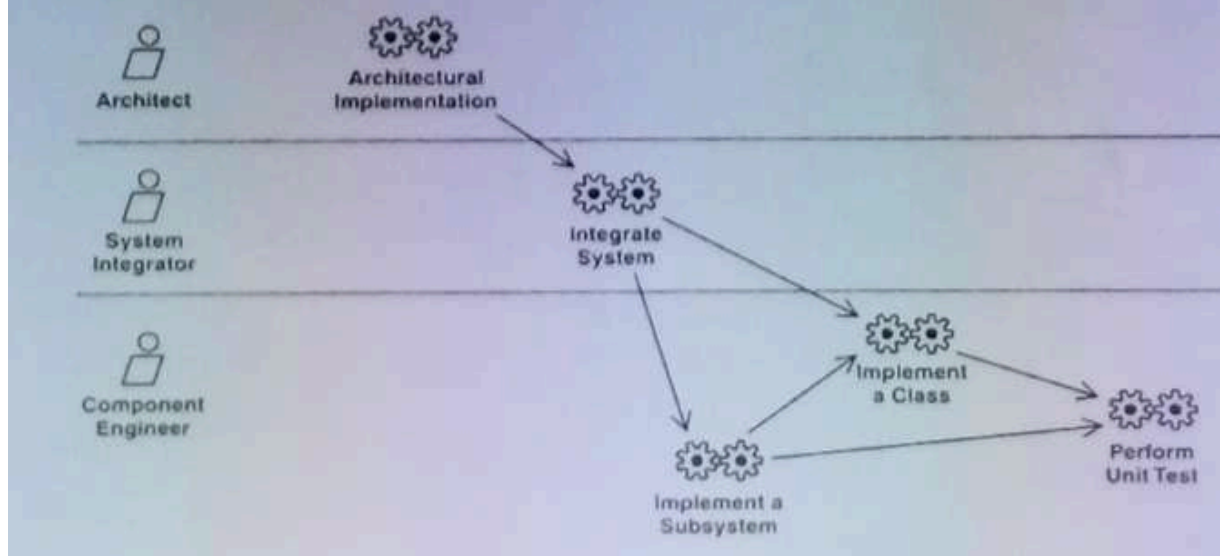
• Integrador de sistemas

- Responsable del plan de integración de construcciones.

Por último, un resultado importante de la implementación es la asignación de componentes ejecutables a nodos. El arquitecto es responsable de esta asignación, la cual se representa en la vista de la arquitectura del modelo de despliegue. Véase detalles en el Capítulo 9, Sección 9.3.7.

FLUJO DE TRABAJO - IMPLEMENTACIÓN

IMPLEMENTACIÓN – FLUJO DE TRABAJO



-el arquitecto define a alto nivel

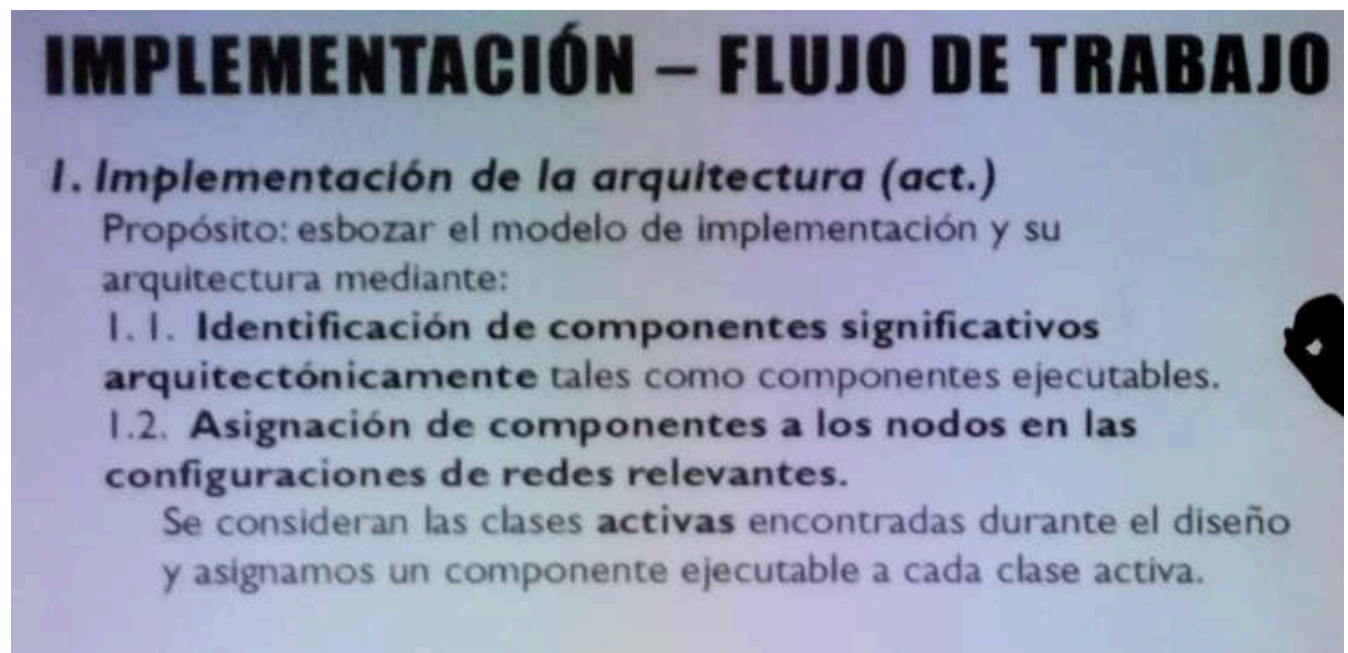
-el rol esencial de que es lo que se va a integrar o no, es el Integrador de sistema

- finalmente se hace el test unitario, se prueba cada componente por separado aislado de su contexto
- hay una cosa que no está reflejada aca, es que en la primer pasada no se hace la integración sino que se hace el plan
- primero se hace el plan, luego se implementa, se hacen las pruebas y luego recién se integra

El objetivo principal de la implementación es implementar el sistema. Este proceso es iniciado por el arquitecto³ esbozando los componentes clave en el modelo de implementación. A continuación, el integrador de sistemas planea las integraciones de sistema necesarias en la presente iteración como una secuencia de construcciones. Para cada construcción, el integrador de sistemas describe la funcionalidad que debería ser implementada y qué partes del modelo de implementación (es decir, subsistemas y componentes) se verán afectadas. Los requisitos sobre los subsistemas y componentes en la construcción son entonces implementados por ingenieros

de componentes. Los componentes resultantes son probados y pasados al integrador de sistemas para su integración. Entonces, el integrador de sistemas integra los nuevos componentes en una construcción y la pasa a los ingenieros de pruebas de integración para llevar a cabo pruebas de integración (véase Capítulo 11). A continuación, los desarrolladores inician la implementación de la siguiente construcción, tomando en consideración los defectos de la construcción anterior.

1. Implementación de la arquitectura



IMPLEMENTACIÓN – FLUJO DE TRABAJO

1. Implementación de la arquitectura (act.)

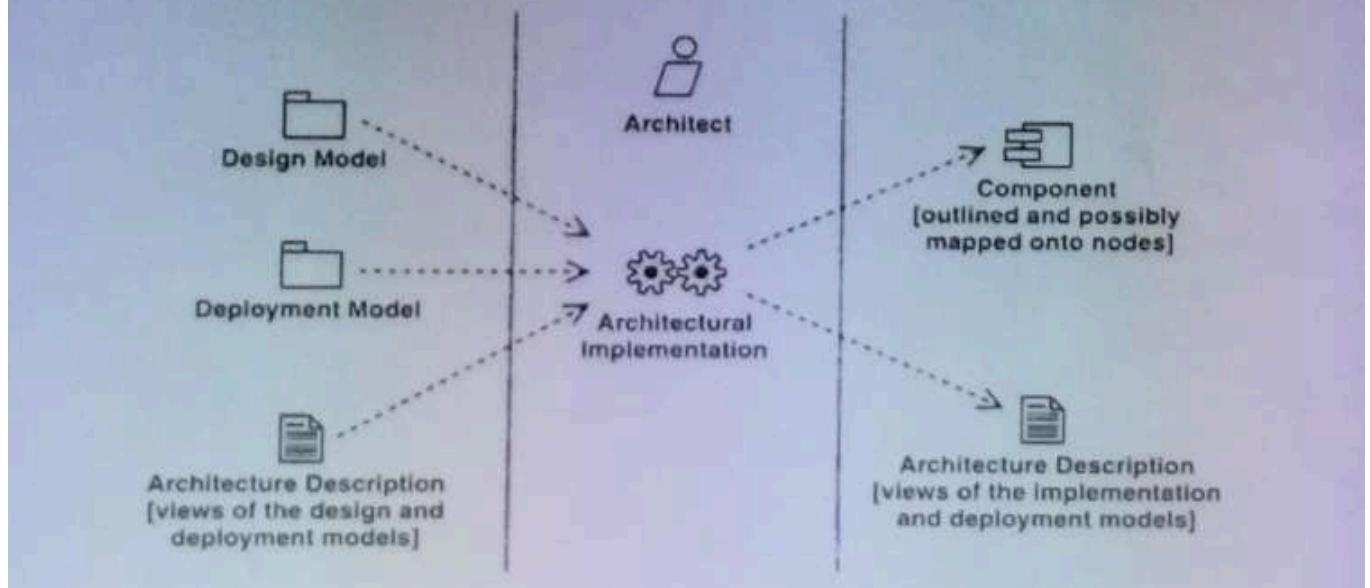
Propósito: esbozar el modelo de implementación y su arquitectura mediante:

- 1.1. **Identificación de componentes significativos arquitectónicamente** tales como componentes ejecutables.
- 1.2. **Asignación de componentes a los nodos en las configuraciones de redes relevantes.**

Se consideran las clases **activas** encontradas durante el diseño y asignamos un componente ejecutable a cada clase activa.

- clases activas, que se van a poder ejecutar concurrentemente

IMPLEMENTACIÓN – FLUJO DE TRABAJO



-lo que sale de aca son los componentes principales, no todos

-en el mapeo los componentes van a ser asignados a una ubicación determinada dentro del nodo

-lo que es el modelo de despliegue es lo mismo que se pone en la arquitectura. Es de los pocos modelos que no hay diferencia, van casi todos en ambos lugares

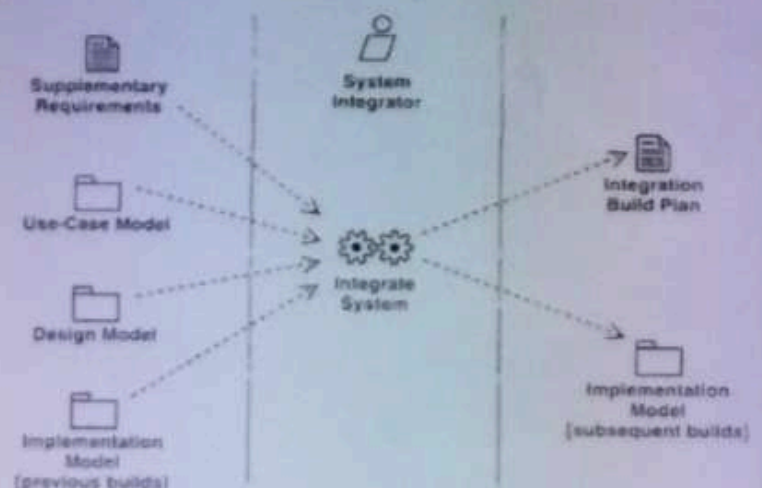
2. Integrar el sistema

IMPLEMENTACIÓN – FLUJO DE TRABAJO

2. Integrar el sistema (act.)

Los objetivos de la integración son:

- Crear un plan de integración de construcciones
- Integrar cada construcción antes de que sea sometida a pruebas de integración.



2.1 Planear la construcción subsecuente

IMPLEMENTACIÓN – FLUJO DE TRABAJO

2.1. Planear la construcción subsecuente

Criterios (facilitar la integración)

- La construcción debería agregar funcionalidad a las construcciones previas implementando casos de uso completos
- No debería incluir muchos componentes nuevos o refinados
- Las construcciones deberían ir desde las capas más bajas a las más altas
- Para cada potencial caso de uso a ser implementado
 - Identificar su realización de diseño, luego clases y subsistemas que participan, luego componentes y subsistemas de implementación, finalmente considerar el impacto

-voy a definir por cada CU las funcionalidad que voy a estar implementando, eso lo decidí el integrador

2.2 Integrar la construcción

IMPLEMENTACIÓN – FLUJO DE TRABAJO

2.2. Integrar la construcción

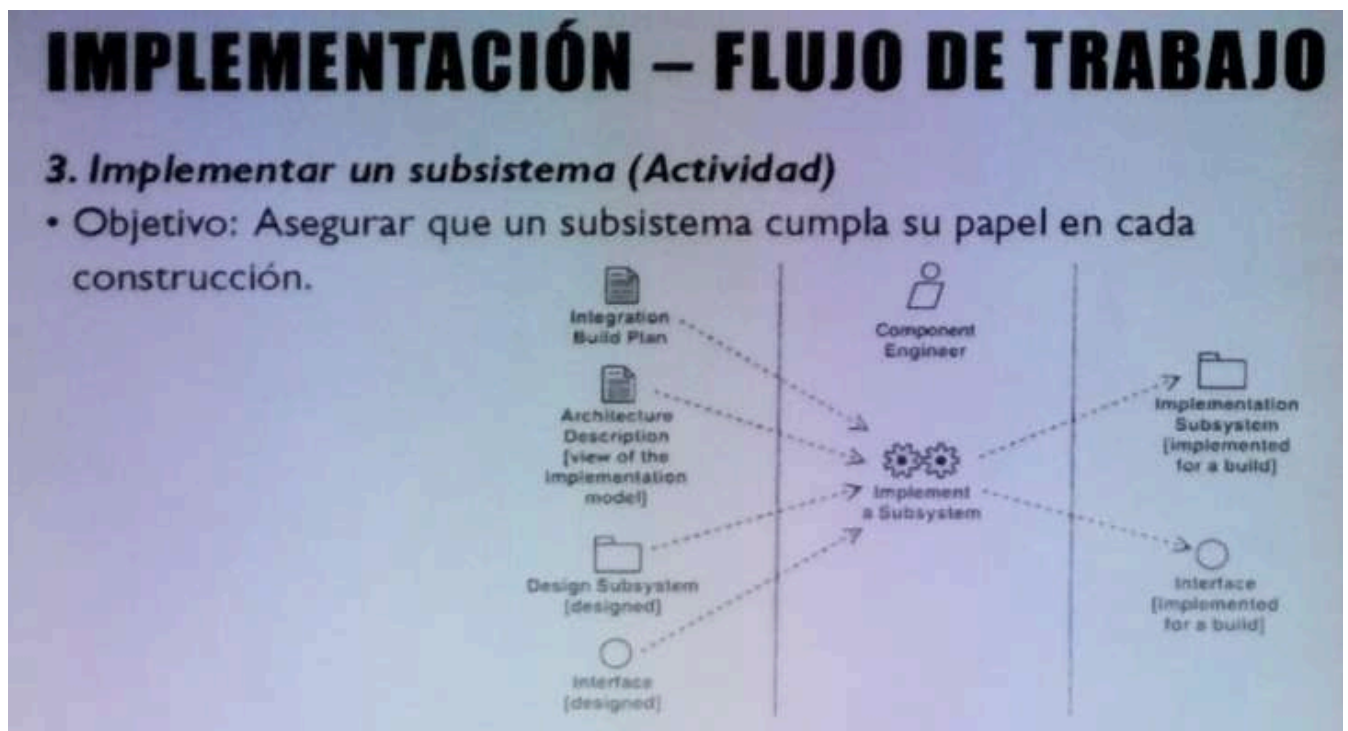
Si se planeó cuidadosamente lo anterior:

- Se recolectan las versiones correctas de subsistemas y componentes
- Se compilan y enlazan en una construcción
- Deben considerarse las dependencias de abajo hacia arriba para hacer la compilación

-a veces la integración tiene un orden para hacerse, hay dependencias que respetar

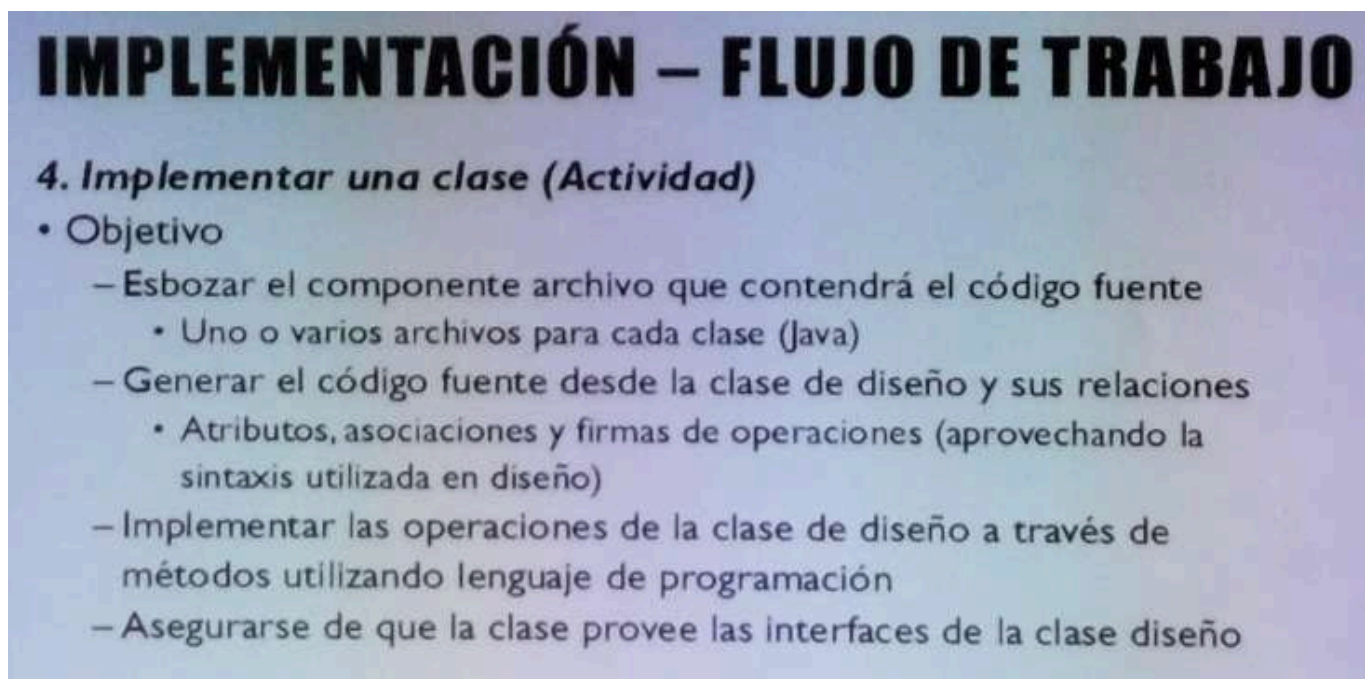
-en un sistema modelo en capas, primero se integran las capas inferiores y luego las capas superiores

3. Implementar un subsistema



3.1. Mantenimiento de los contenidos de los subsistemas

4. Implementar una clase (Actividad)



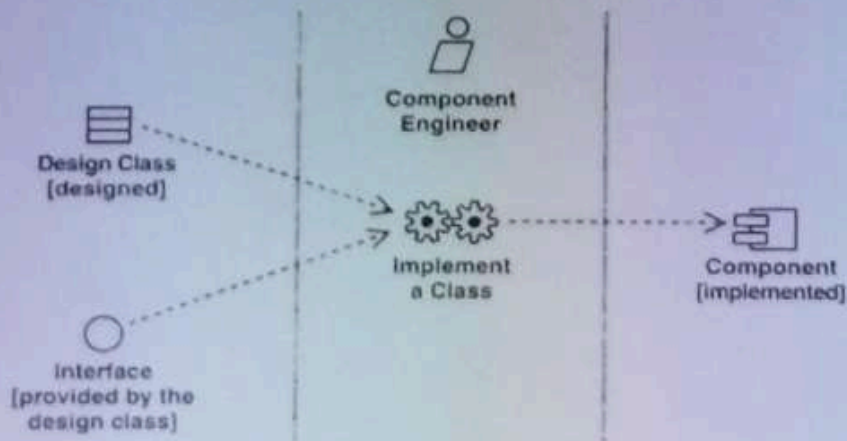
-debo integrar las clases en el lenguaje que elegí

-para cada operación que elija voy a tener que codificar el método

-el Ingeniero de componentes haría el rol de Programador

IMPLEMENTACIÓN – FLUJO DE TRABAJO

4. Implementar una clase (Actividad)



4.1 Esbozo de los componentes fichero

4.2 Generación de código a partir de una clase de diseño

4.3 Implementación de operaciones

5. Realizar prueba unitaria (act.)

IMPLEMENTACIÓN – FLUJO DE TRABAJO

5. Realizar prueba unitaria (act.)

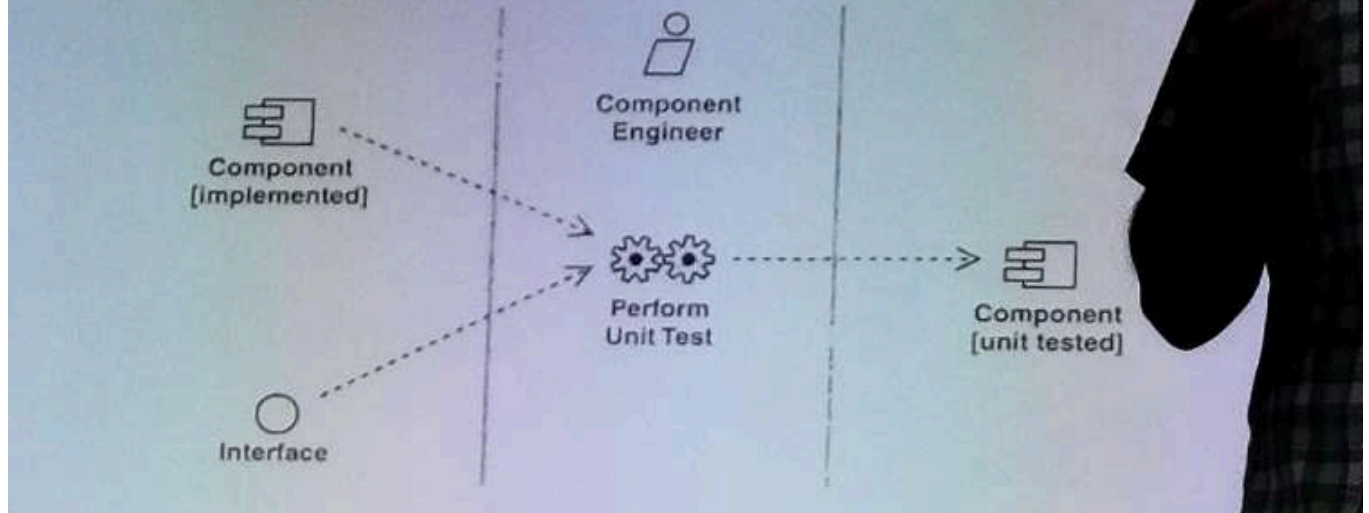
- Objetivo: probar los componentes implementados como unidades individuales.
- Existen dos tipos de prueba:
 - Prueba de especificación, o prueba de “caja negra”, que verifica el comportamiento de la unidad observable externamente.
 - Prueba de estructura o prueba de “caja blanca”, que verifica la implementación interna de la unidad.

-estas pruebas son altamente **automatizadas**

-Prueba Caja Negra: no le interesa cómo está codificado. Sino que para ciertas entradas devuelva ciertos valores. Esta es la más común

-Prueba Caja Blanca: le interesa cómo está codificado internamente. Son pruebas más exhaustivas por lo tanto **más costosas**. Si interesa ver cada parte del código

IMPLEMENTACIÓN – FLUJO DE TRABAJO



- aca aparece el componente implementado
- una prueba con las interfaces, es que sea compatible con la misma

5.1. Realización de pruebas de especificación (caja negra)

- Tienen en cuenta que devolverá para determinada entrada.
- Muestra como analizando el CU se pueden encontrar situaciones que cubren la gran mayoría de posibilidades sin tener que estar probando las infinitas combinaciones.
- Se utilizan las clases de equivalencia

Una clase de equivalencia es un conjunto de valores de entrada, estado o salida para los que se supone que un objeto se comporta de forma similar. Probando un componente para cada combinación de las clases de equivalencia de entradas, salidas y estados iniciales es posible obtener casi la misma cobertura “efectiva” de prueba que si se prueban individualmente cada una de las combinaciones de valores, pero con un esfuerzo considerablemente menor.

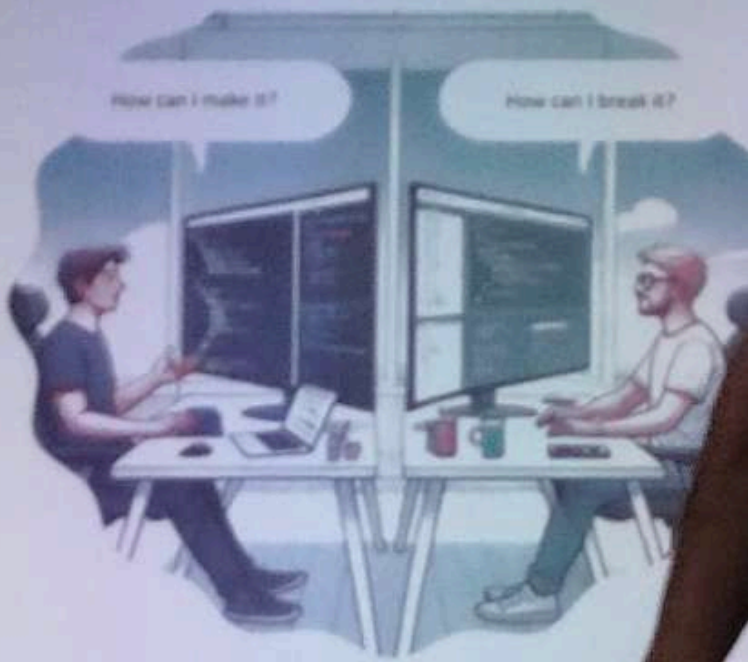
5.2. Realización de pruebas de estructura (caja blanca)

10.5.5.2. Realización de pruebas de estructura Las pruebas de estructura se realizan para verificar que un componente funciona internamente como se quería. El ingeniero de componentes debería asegurarse de probar todo el código durante las pruebas de estructura, lo que quiere decir que cada sentencia ha de ser ejecutada al menos una vez. El ingeniero de componentes debería también asegurarse de probar los caminos más importantes en el código. Entre estos caminos tenemos los que son seguidos más comúnmente, los caminos más críticos, los caminos menos conocidos de los algoritmos y otros caminos asociados con riesgos altos.

PRUEBA

PRUEBA

- Se verifica el resultado de cada construcción
 - Internas
 - Externas
- Empleado cuando se hacen pruebas de integración y de sistemas (implementación)
- Foco en la elaboración y construcción
- Transición por defectos



The illustration shows two developers sitting at desks with large monitors, working in a modern office environment. Above them are two speech bubbles: 'How can I make it?' on the left and 'How can I break it?' on the right, symbolizing the dual focus of testing in software development.

-la parte de prueba es la que nos permite verificar que el sistema cumple con los requisitos que realmente se pidieron, una parte de esa prueba se hace en los test unitarios al final de la implementación

-ahora se hace algo más allá de los test unitarios y **probar en su totalidad**

-los dos grandes pruebas que hay son: prueba de integración y prueba de sistemas

-la prueba está muy presente y acompaña la implementación

-en este caso recuerden que se va a necesitar de Pruebas en la fase de Elaboración, Construcción pero sobre todo en la **fase de Transición**

PROPÓSITOS - PRUEBA

PRUEBA - PROPÓSITOS

- Planificar las pruebas necesarias en cada iteración,
 - incluyendo las pruebas de **integración** y las **pruebas de sistema**.
- Diseñar e implementar pruebas creando:
 - los casos de prueba (especifican qué probar)
 - procedimientos de prueba (especifican cómo realizar las pruebas)
 - componentes de prueba para automatizar las pruebas.
- Realizar las pruebas.
 - Gestionar resultados
 - Volver a probar y reenviar a corregir

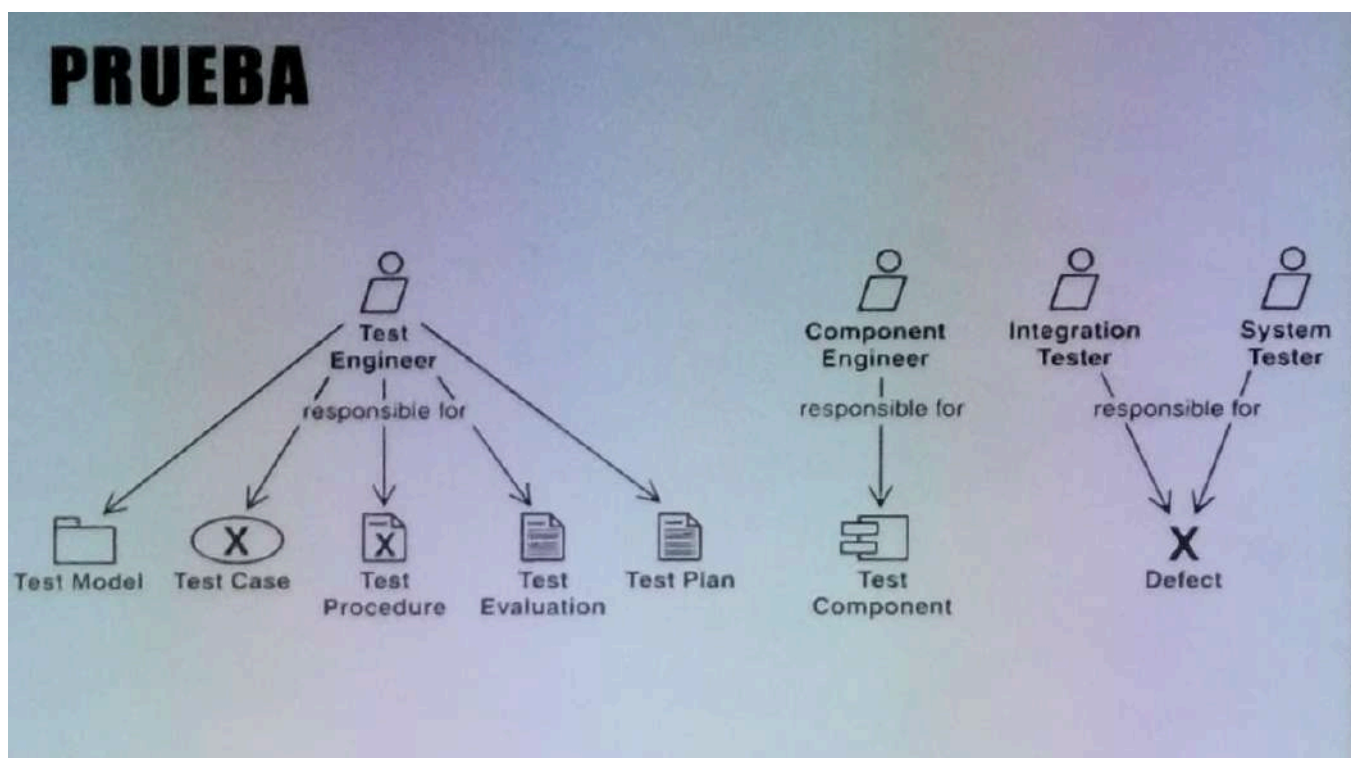
-también se debe planificar

-a veces no es posible probar todo, porque es costosa. Así que hay que probar lo que me da **mayor beneficio**

-hasta el final vamos a usar CU

-hay **3 partes importantes**: Casos de prueba, Procedimiento de prueba y Componente de prueba

Diagrama



- desaparece el Arquitecto
- el Ingeniero de componente, se va a encargar de programar el componente que va automatizar la prueba
- no hay **NINGÚN artefacto** igual a los de antes
- el único que existía antes era el Ingeniero de componentes, el **resto es NUEVO**

Durante la fase de inicio puede hacerse parte de la planificación inicial de las pruebas cuando se define el ámbito del sistema. Sin embargo, las pruebas se llevan a cabo sobre todo cuando una construcción (como un resultado de implementación) es sometida a pruebas de integración y de sistema. Esto quiere decir que la realización de pruebas se centra en las fases de elaboración, cuando se prueba la línea base ejecutable de la arquitectura, y de construcción, cuando el grueso del sistema está implementado. Durante la fase de transición el centro se desplaza hacia la corrección de defectos durante los primeros usos y a las pruebas de regresión. Véase Figura 11.2.

...a lo largo de las iteraciones, lo que significa que las últimas iteraciones requerirán un gran esfuerzo en pruebas de regresión. Es natural, por tanto, mantener el modelo de pruebas a lo largo del ciclo de vida del software completo, aunque el modelo de pruebas **cambia constantemente** debido a:

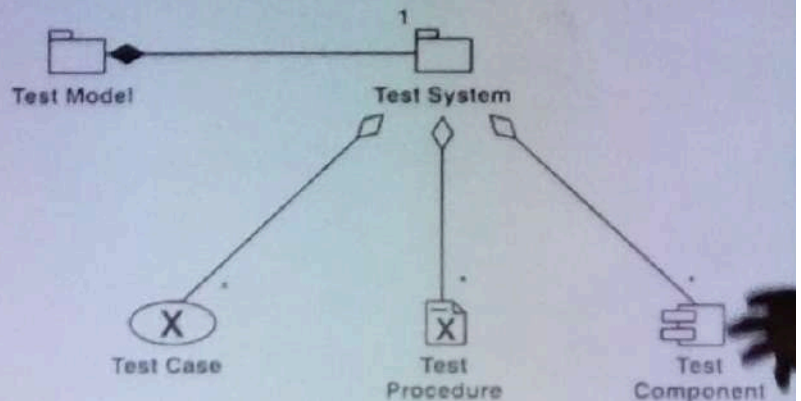
- La eliminación de casos de prueba obsoletos (y los correspondientes procedimientos de prueba y componentes de prueba).
- El refinamiento de algunos casos de prueba en casos de prueba de regresión.
- La creación de nuevos casos de uso para cada nueva construcción.

ARTEFACTOS

Modelo de pruebas

PRUEBA - ARTEFACTOS

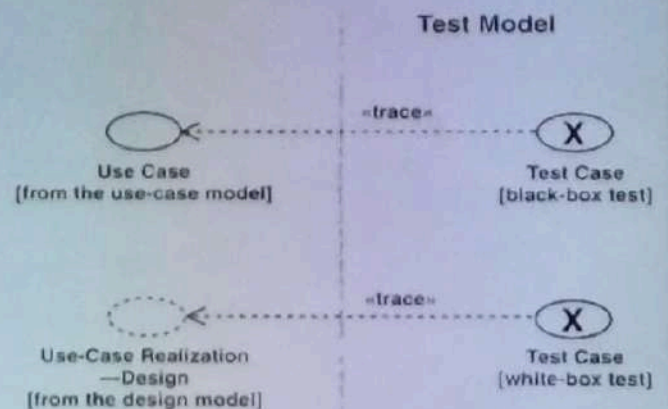
- **Modelo de pruebas**
- Describe como se prueban (integración y sistema) los componentes en el modelo de implementación.
- Es una colección de
 - casos de prueba,
 - procedimientos de prueba, y
 - componentes de prueba.



Caso de prueba

PRUEBA - ARTEFACTOS

- **Caso de prueba**
- Especifica una forma de probar el sistema, incluyendo:
 - qué probar
 - Requerimientos posibles de probar
 - Comúnmente:
 - Cómo probar un caso de uso o escenario (caja negra)
 - Cómo probar una realización de caso de uso diseño (caja blanca)
 - la entrada con la que se ha de probar
 - el resultado esperado y
 - las condiciones bajo las que ha de probarse.
- Los casos de prueba pueden parecerse mucho (escenarios mismo caso de uso)



-habíamos dicho que el caso de prueba que nos permite saber que vamos a probar, es una forma de probar nuestro sistema

-hay que definir cuales son los requerimientos, entonces definimos el CU

-nos define exactamente que vamos a estar probando, que CU vamos a estar probando.

Hay una relación de traza directa

-tenemos caja negra y blanca pero aplicado a un caso de prueba

-nos interesa saber que desencadena cada paso de los CU

-puede haber **muchos casos de prueba** para el mismo CU

- Un caso de prueba que especifica cómo probar un caso de uso o un escenario específico de un caso de uso. Un caso de prueba de este tipo incluye la verificación del resultado de la interacción entre los actores y el sistema, que se satisfacen las precondiciones y postcondiciones especificadas por el caso de uso y que se sigue la secuencia de acciones especificadas por el caso de uso. Obsérvese que un caso de prueba basado en un caso de uso especifica típicamente una prueba del sistema como “caja negra”, es decir, una prueba del comportamiento observable externamente del sistema.
- Un caso de prueba que especifica cómo probar una realización de caso de uso-diseño o un escenario específico de la realización. Un caso de prueba de este tipo puede incluir la verificación de la interacción entre los componentes que implementan dicho caso de uso. Obsérvese que los casos de prueba basados en una realización de caso de uso típicamente especifican una prueba del sistema como “caja blanca”, es decir, una prueba de la interacción interna entre los componentes del sistema.

Tipos de prueba

Se pueden especificar otros casos de prueba para probar el sistema como un todo. Por ejemplo:

- Las *pruebas de instalación* verifican que el sistema puede ser instalado en la plataforma del cliente y que el sistema funcionará correctamente cuando sea instalado.
- Las *pruebas de configuración* verifican que el sistema funciona correctamente en diferentes configuraciones; por ejemplo, en diferentes configuraciones de red.
- Las *pruebas negativas* intentan provocar que el sistema falle para poder así revelar sus debilidades. Los *ingenieros de pruebas* identifican los casos de prueba que intentan utilizar el sistema en formas para los que no ha sido diseñado, por ejemplo, utilizando configuraciones de red incorrectas, capacidad de hardware insuficiente o una carga de trabajo “imposible”¹.
- Las *pruebas de tensión o de estrés* identifican problemas con el sistema cuando hay recursos insuficientes o cuando hay competencia por los recursos.

(Esto es del capítulo 3 pero me parecía bien relacionarlo)

del ciclo de desarrollo. Tan pronto como se hayan capturado los casos de uso, es posible especificar los casos de prueba (pruebas de “caja negra”) y determinar el orden en el cual

realizarlos, integrarlos y probarlos. Más adelante, según se vayan realizando los casos de uso en el diseño, pueden detallarse las pruebas de los casos de uso (pruebas de “caja blanca”). Cada

Ejemplo

PRUEBA - ARTEFACTOS

• Caso de prueba (Ejemplo)

- Los ingenieros de pruebas sugieren un conjunto de casos de prueba para probar el caso de uso **Pagar factura**, en el que cada caso de prueba verificará un escenario del caso de uso.
- Uno de los casos de prueba propuestos es el pago de una factura por un pedido de 300 dólares de una bicicleta de montaña. A este caso de prueba lo llaman **Pay 300 - Mountain Bike**.
- Para completarse, el caso de prueba debe especificar **la entrada, el resultado esperado y otras condiciones** relevantes para verificar el escenario del caso de uso.

Entrada

PRUEBA - ARTEFACTOS

• Caso de prueba (Ejemplo - continúa)

• Entrada

- Existe un pedido válido de una bicicleta de montaña y se ha enviado al vendedor, **Crazy Mountaineer, Inc.** El precio de lista de la bicicleta es de **\$ 300**, incluidos los gastos de envío y manejo.
- El comprador ha recibido una confirmación de pedido (ID 98765) de una bicicleta de montaña. El precio confirmado de la bicicleta es de **\$300**, incluyendo gastos de envío y manipulación.
- El comprador ha recibido una factura (ID 12345).
La factura cumple con la confirmación del pedido de la bicicleta de montaña.
Esta es la única factura presente en el sistema.
El importe facturado debe ser un total de 300 USD y la factura debe estar en estado Pendiente.
La factura debe apuntar a una cuenta 22-222-2222, que debe recibir el dinero.
La cuenta tiene un saldo actual de **\$963,456.00**.
La cuenta debe ser propiedad del vendedor.
- La cuenta del comprador 11-111-1111 muestra un saldo de **\$350**.

Resultado y Condiciones

PRUEBA - ARTEFACTOS

- **Caso de prueba (Ejemplo - continúa)**

- **Resultado**

- El estado de la factura debe establecerse en Cerrado (para indicar que se ha pagado).
- La cuenta del comprador 11-111-1111 debe mostrar un saldo de \$50.
- El saldo de la cuenta del vendedor 22-222-2222 debería haber aumentado a \$963,756.00.

- **Condiciones**

- No se permite que otros casos de uso (instancias) accedan a las cuentas durante este caso de prueba.

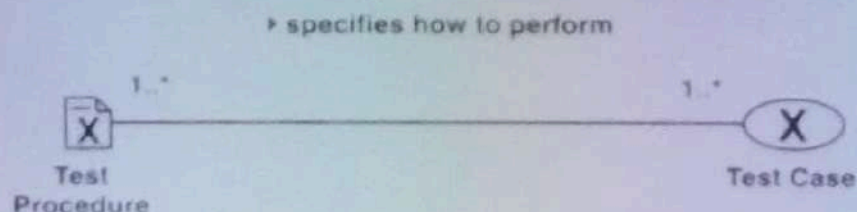
-las condiciones no son ni PRE ni POST, son medio se esta ejecutando. En este caso se asegura que no haya otras ejecuciones

Procedimiento prueba

PRUEBA - ARTEFACTOS

- **Procedimiento de prueba**

- Especifica cómo realizar uno o varios casos de prueba relacionados.
- Puede ser especificado para cada caso de prueba o reutilizado para varios casos de prueba
- Similar a la descripción de un camino de caso de uso, pero con información adicional

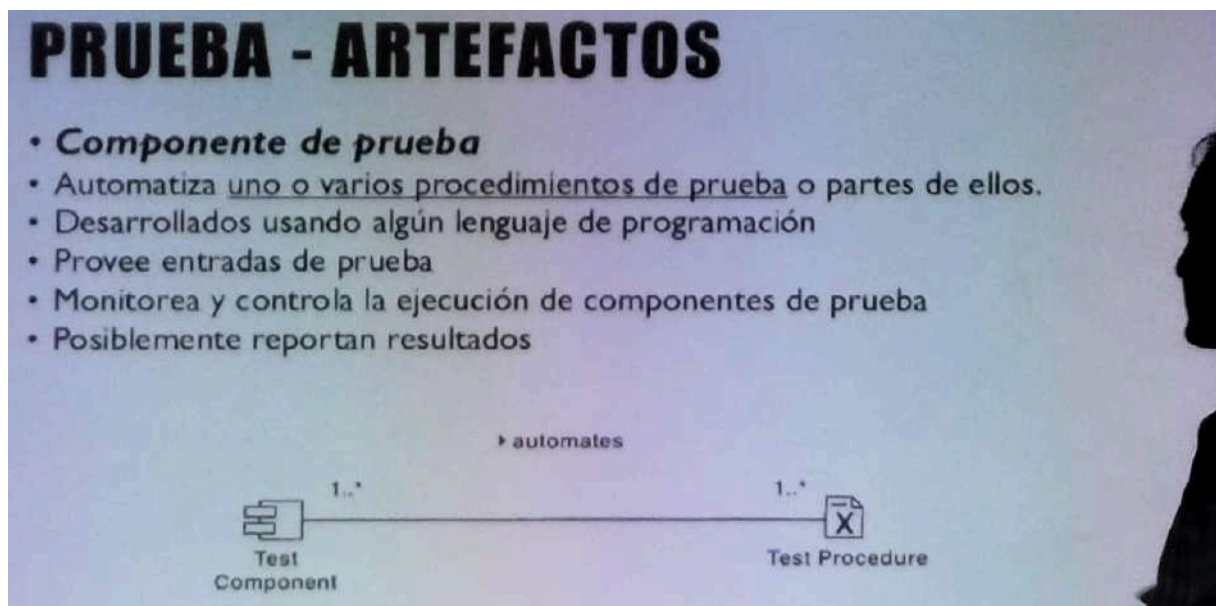


-es de lo menos importantes

-es como una especie de receta que tengo que seguir antes de hacer la prueba

-relación entre un caso de prueba y un procedimiento de prueba. Un procedimiento de prueba puede ser válido para muchos casos de prueba

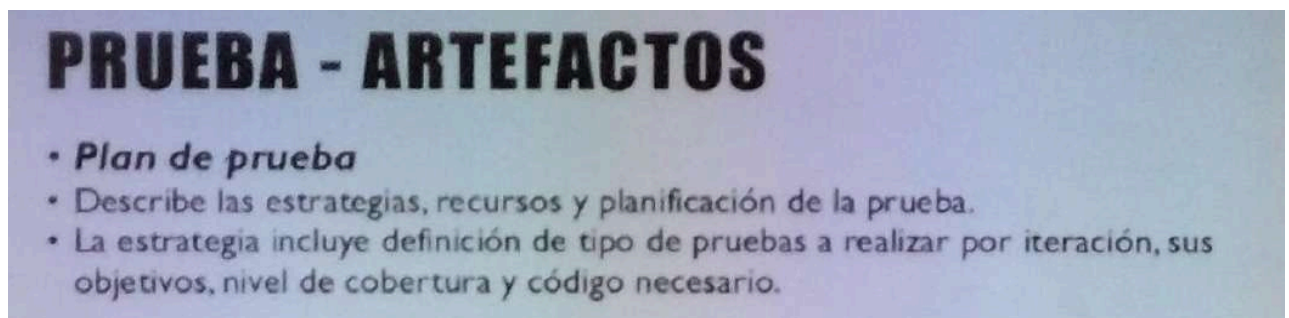
Componente de prueba



- me permite tratar de automatizar las pruebas, las más automatizables son los test unitarios. Y las menos automatizables son las de usabilidad
- tratar de sobrecargar lo más que se pueda para ver como funciona en base a eso

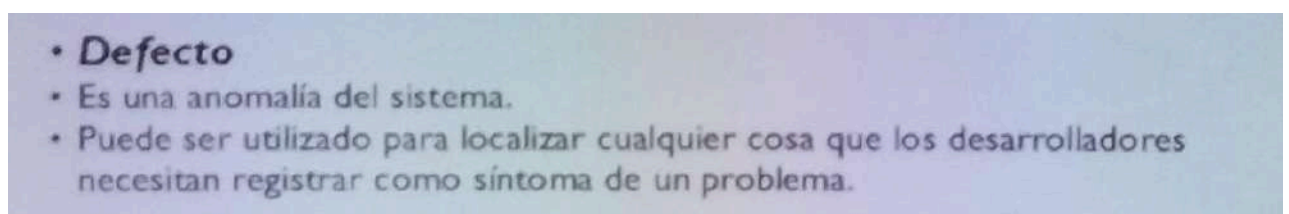
Los componentes de prueba pueden ser desarrollados utilizando un lenguaje de guiones o un lenguaje de programación, o pueden ser grabados con una herramienta de automatización de pruebas.

Plan de prueba



y porcentaje de pruebas que deberían ejecutarse con un resultado específico.

Defecto



- con ese Defecto debemos saber si se soluciono o no, por eso es necesario representarlo como un artefacto
- podría ser algo escrito

- por ejemplo un síntoma de un fallo software o un problema descubierto en una revisión
- de un problema en el sistema que ellos necesitan controlar y resolver.

Evaluación de prueba

- **Evaluación de prueba**
- Es una evaluación de los resultados de la prueba.

Una evaluación de prueba es una evaluación de los resultados de los esfuerzos de prueba, tales como la cobertura del caso de prueba, la cobertura de código y el estado de los defectos.

TRABAJADORES - PRUEBA

Ingeniero/Diseñador de pruebas

Ingeniero de Componentes

PRUEBA - TRABAJADORES

- **Ingeniero/Diseñador* de pruebas**
 - Responsable de la **integridad** del modelo de pruebas
 - **Planean** las pruebas.
 - **Selecciona y describen** los casos de prueba y procedimientos de prueba.
 - **Evaluar** la ejecución de las pruebas de integración y sistemas
- **Ingeniero de componentes**
 - Responsable de los **componentes de prueba** que automatizan algunos de los procedimientos de prueba.

-(en los libros, el ingeniero es lo mismo que el diseñador de prueba)

Obsérvese que los diseñadores de pruebas realmente no llevan a cabo las pruebas, sino que se dedican a la preparación y evaluación de las mismas. Otros dos tipos de trabajadores, los ingenieros de pruebas de integración y los ingenieros de pruebas de sistema, son los encargados de llevarlas a cabo.

11.4.2. Trabajador: ingeniero de componentes

Los ingenieros de componentes son responsables de los componentes de prueba que automatizan algunos de los procedimientos de prueba (no todos los procedimientos de prueba pueden ser automatizados). Esto es así porque la creación de dichos componentes puede necesitar de substanciales habilidades como programador (habilidades que también son precisas en el ingeniero de componentes en el flujo de trabajo de la implementación; véase Sección 10.4.2).

PRUEBA - TRABAJADORES

• Probador de integración

- Responsable de realizar las **pruebas de integración** que se necesitan para cada construcción producida en el flujo de implementación.
- Las pruebas de integración se realizan para probar que los **componentes integrados en una construcción interoperan correctamente**.
- Por ello normalmente derivan de casos de prueba vinculados con realizaciones de casos de uso diseño
- Documentan los defectos que aparecen como resultado de las pruebas de integración.

-el test de integración hay gente por separada que lo hace

-en base al plan de integración, aquellos componentes que deben funcionar de manera conjunta si da el resultado esperado

-**no esta tan asociado** a los **CU**

Obsérvese que un ingeniero de pruebas de integración prueba el resultado, es decir, una construcción, **creado por el integrador de sistemas en el flujo de trabajo de la implementación**. Es por esto por lo que en algunos proyectos se decide que estos trabajadores sean el mismo individuo o grupo de individuos para minimizar el solapamiento en el conocimiento necesario.

• Probador de sistema

- Responsable de realizar las **pruebas de sistema** necesarias sobre una construcción que muestra el resultado de una iteración completa.
- Las pruebas de sistema se llevan a cabo para **verificar interacciones entre los actores y el sistema (como un todo)**
- Derivados de casos de prueba que especifican como probar casos de uso.
- No necesitan saber del funcionamiento interno del sistema

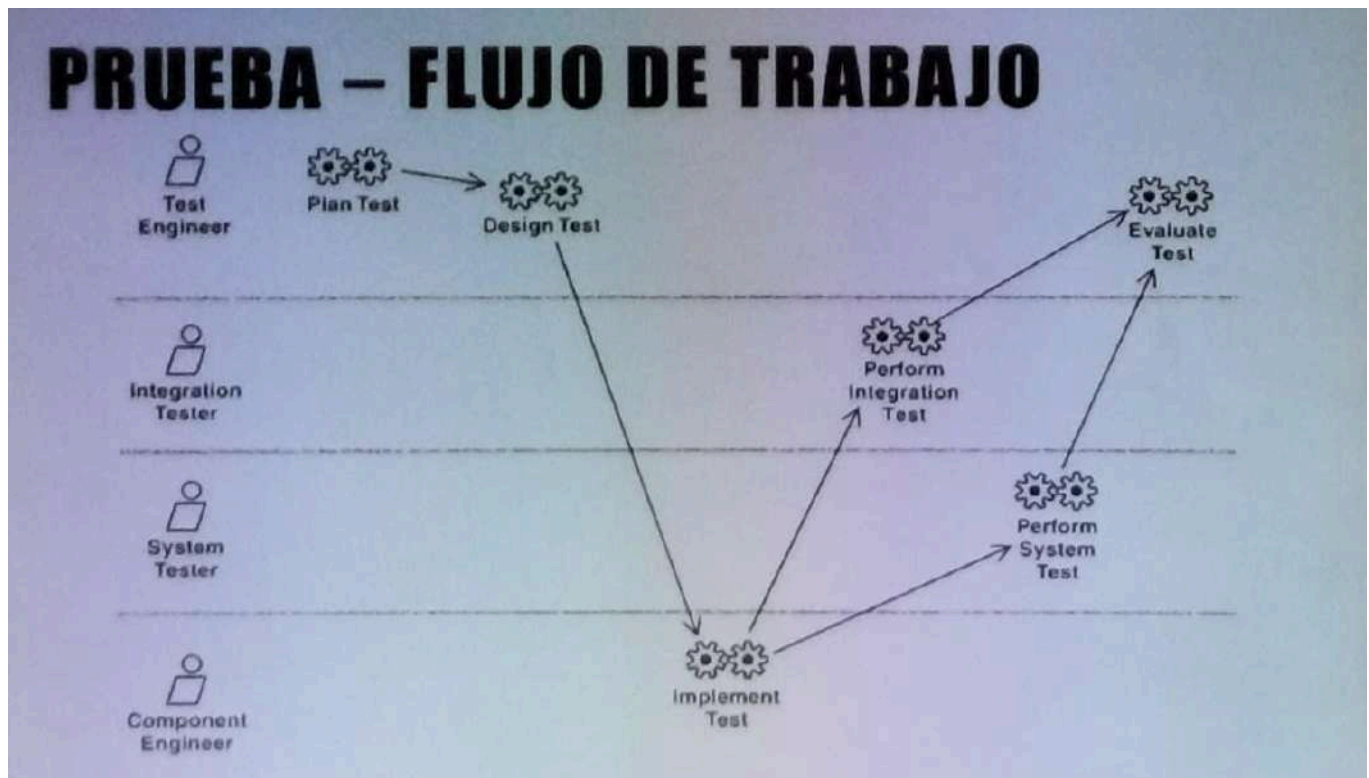
-verificar que el sistema interactúe correctamente con el Usuario del Sistema, trata de ver que la interacción sea correcta

-normalmente esto viene **asociado** con los **CU**

-normalmente al ejecutar la prueba si existiese vienen un conjunto de Defectos

El ingeniero de pruebas de sistema **se encarga de documentar los defectos** en los resultados de las pruebas de sistema.

FLUJO DE TRABAJO

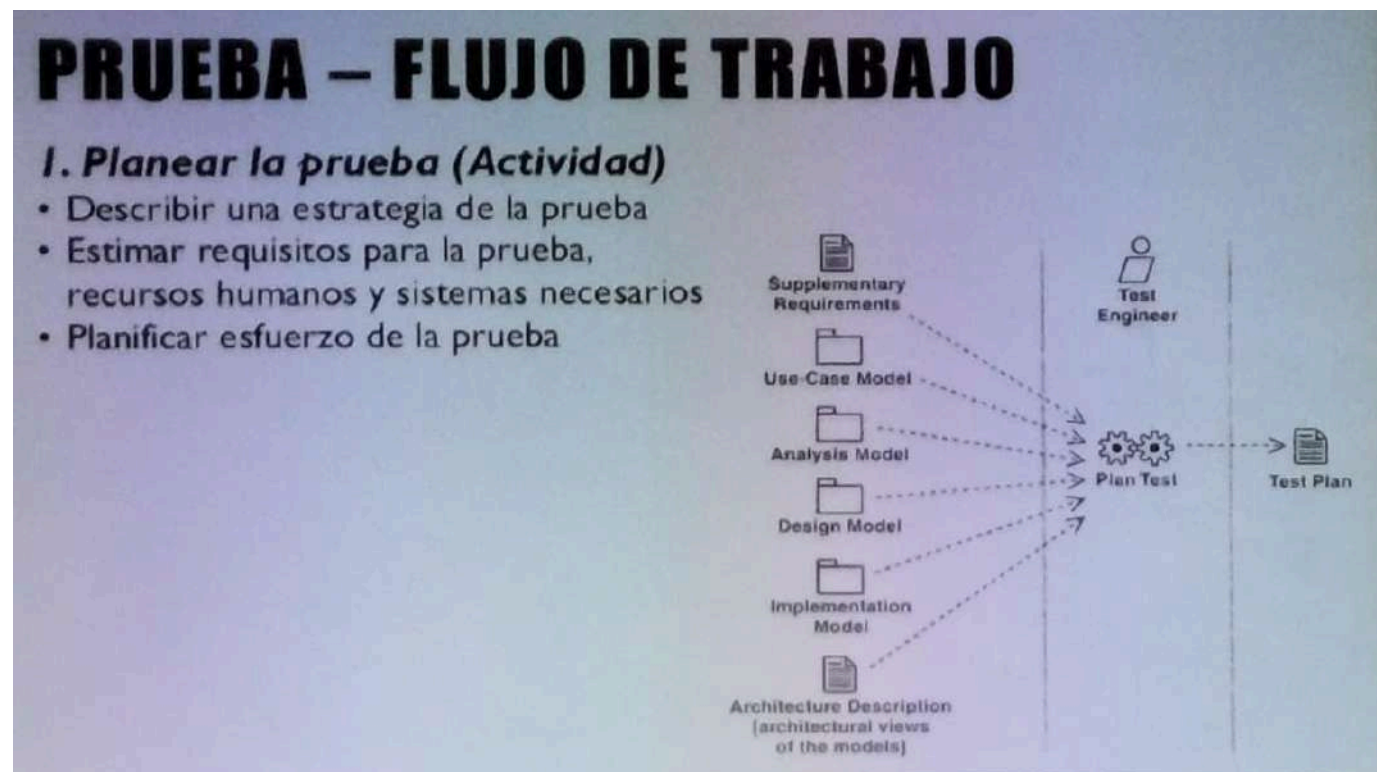


- finalmente cada tester se encarga de hacer la prueba correcta
- después de hacer la prueba viene una actividad de evaluación
- estas son generalidades

El principal objetivo de la prueba es realizar y evaluar las pruebas como se describe en el modelo de pruebas. Los ingenieros de pruebas inician esta tarea planificando el esfuerzo de prueba en cada iteración, y describen entonces los casos de prueba necesarios y los procedimientos de prueba correspondientes para llevar a cabo las pruebas. Si es posible, los ingenieros de componentes crean a continuación los componentes de prueba para automatizar algunos de los procedimientos de prueba. Todo esto se hace para cada construcción entregada como resultado del flujo de trabajo de implementación.

Con estos casos, procedimientos y componentes de prueba como entrada, los ingenieros de pruebas de integración y de sistema prueban cada construcción y detectan cualquier defecto que encuentren en ellos. Los defectos sirven como realimentación tanto para otros flujos de trabajo, como el de diseño y el de implementación, como para los ingenieros de pruebas para que lleven a cabo una evaluación sistemática de los resultados de las pruebas.

1. Planear la prueba



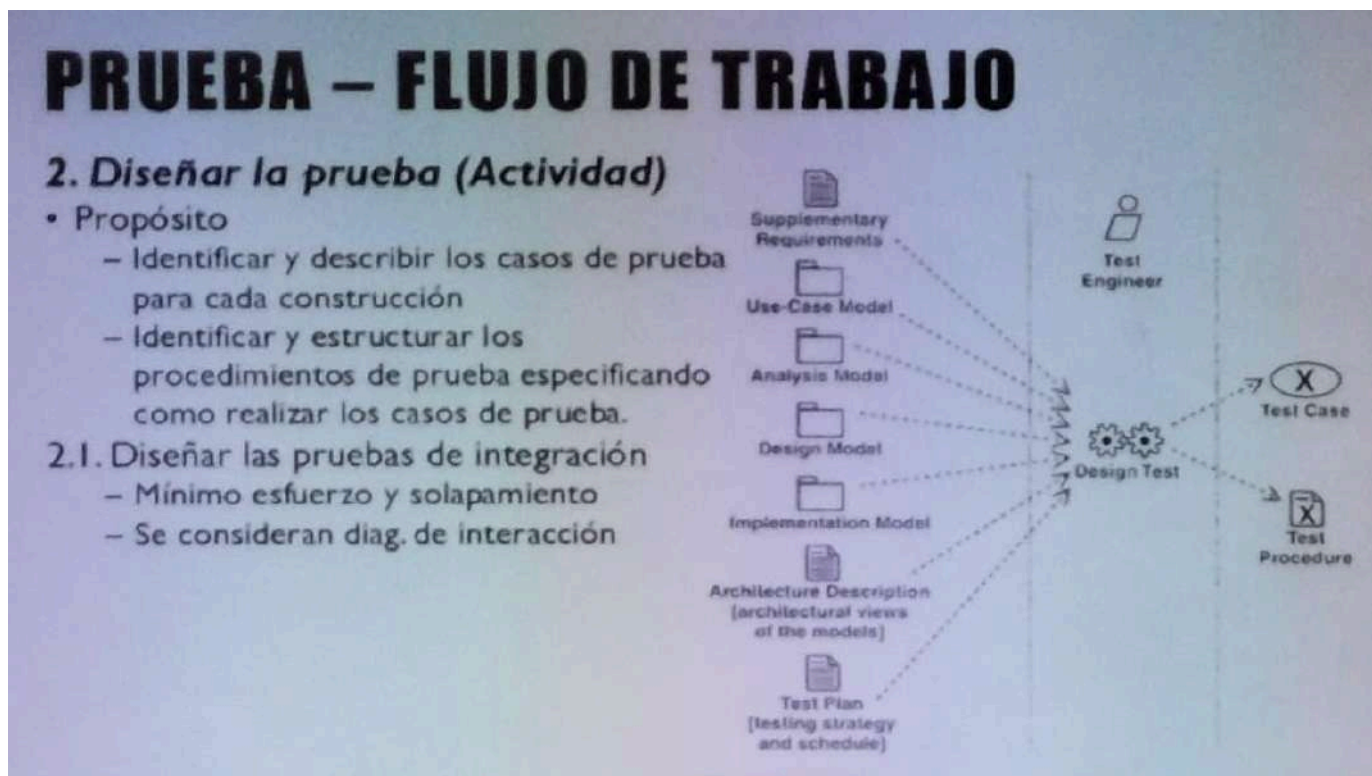
-falta el modelo de Despliegue

-la planificación de la prueba trata de hacer más eficiente la prueba, que recursos económicos, materiales y humanos se van a necesitar. Se indica a que se le va a prestar mayor atención

Los diseñadores de pruebas desarrollan una estrategia de prueba para la iteración, es decir, deciden qué tipo de pruebas ejecutar, cómo ejecutar dichas pruebas, cuándo ejecutarlas y cómo determinar si el esfuerzo de prueba tiene éxito.

El desarrollo, realización y evaluación de cada caso de prueba, procedimiento de prueba y componente de prueba lleva algún tiempo y cuesta algún dinero. Sin embargo, ningún sistema puede ser probado completamente; por tanto, deberíamos identificar los casos, procedimientos y componentes de prueba con un mayor retorno a la inversión en términos de mejora de calidad [3]. El principio general consiste en desarrollar casos y procedimientos de prueba con un solapamiento mínimo para probar los casos de uso más importantes y probar los requisitos que están asociados a los riesgos más altos.

2. Diseñar la prueba



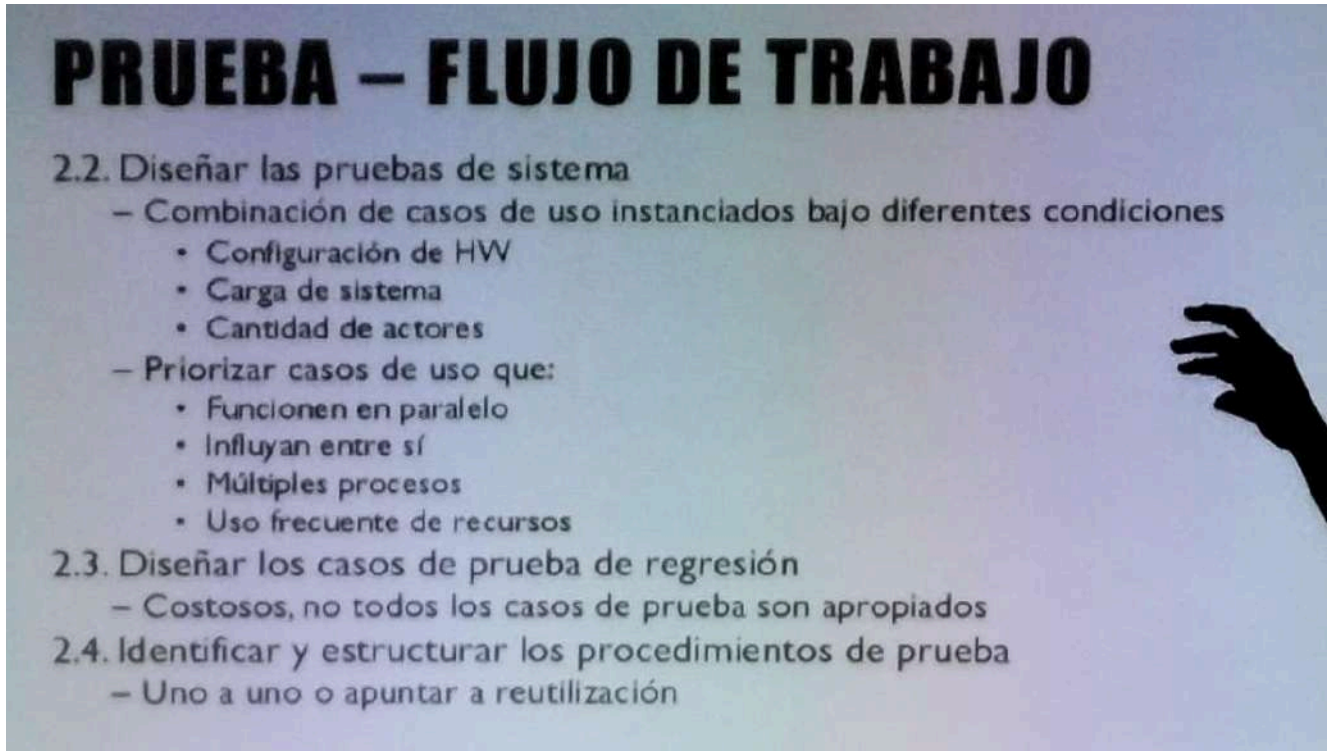
-trata de cubrir todo el código lo más que existe sin solapamiento, tratando de probar la mayor cantidad

-el único que no produce es el Componente de prueba

(Usa todos los modelos menos el de implementación, los requisitos adicionales y la salida de la actividad anterior que sería el Plan de prueba)

11.5.2.1. Diseño de los casos de prueba de integración Los casos de prueba de integración se utilizan para verificar que los componentes interaccionan entre sí de la forma apropiada después de haber sido integrados en una construcción (véase Sección 10.5.2). La mayoría de los casos de prueba de integración pueden ser derivados de las realizaciones de casos de uso-diseño, ya que las realizaciones de casos de uso describen cómo interaccionan las clases y los objetos, y por tanto cómo interaccionan los componentes.

2.2 Diseñar las pruebas de sistema



PRUEBA – FLUJO DE TRABAJO

2.2. Diseñar las pruebas de sistema

- Combinación de casos de uso instanciados bajo diferentes condiciones
 - Configuración de HW
 - Carga de sistema
 - Cantidad de actores
- Priorizar casos de uso que:
 - Funcionen en paralelo
 - Influyan entre sí
 - Múltiples procesos
 - Uso frecuente de recursos

2.3. Diseñar los casos de prueba de regresión

- Costosos, no todos los casos de prueba son apropiados

2.4. Identificar y estructurar los procedimientos de prueba

- Uno a uno o apuntar a reutilización

-se trata la configuración de HW, carga de sistema y cantidad de actores. Todo eso es importante

-se priorizan los CU que van a poder funcionar en paralelo, porque son los que pueden tener **mayores inconvenientes**

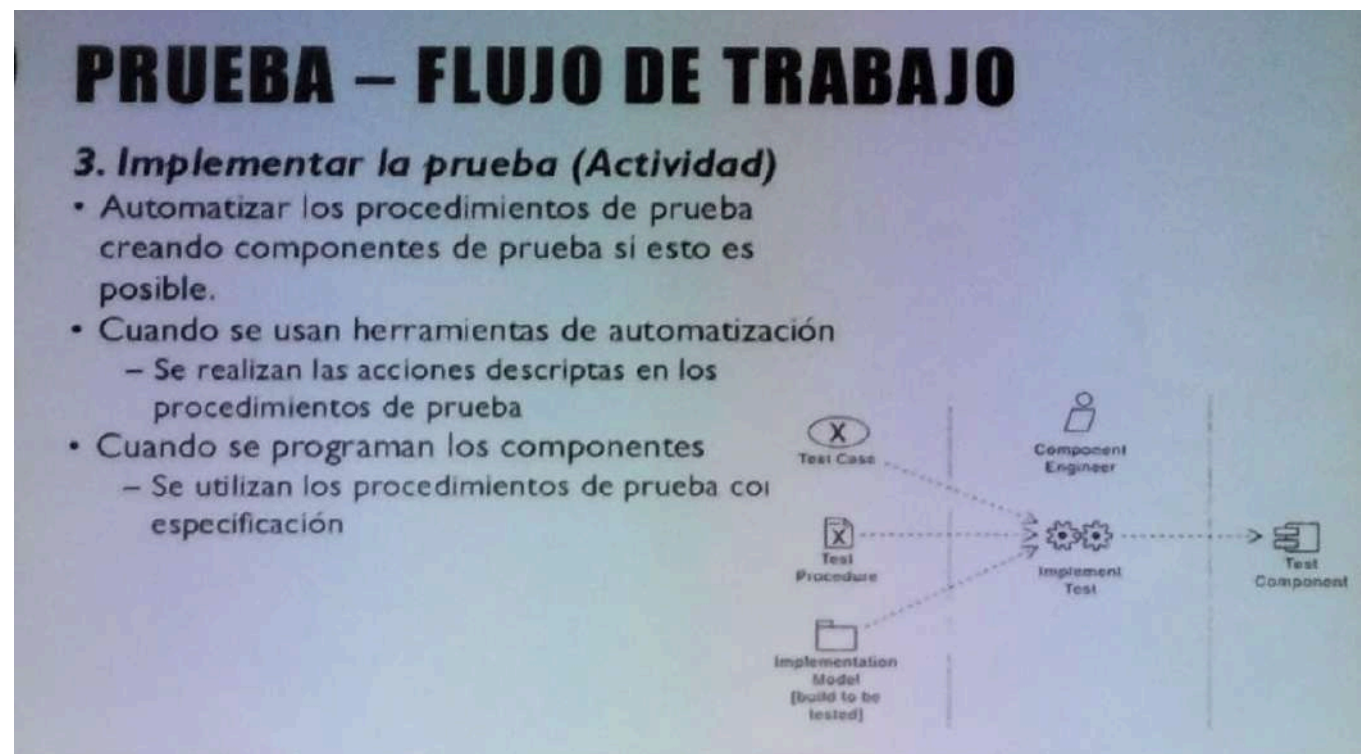
2.3 Diseñar los casos de prueba de regresión

-Se trata de probar el sistema hacia atrás, puede haber casos donde necesite probar todo el sistema completo si sigue funcionando

11.5.2.3. Diseño de los casos de prueba de regresión Algunos casos de prueba de construcciones anteriores pueden ser usados para pruebas de regresión en construcciones subsiguientes, aunque no todos los casos de prueba son adecuados para pruebas de regresión. Para ser adecuados y contribuir a la calidad del sistema los casos de prueba han de ser suficientemente flexibles para ser resistentes a cambios en el software que está siendo probado. La flexibilidad requerida en un caso de prueba de regresión puede suponer un esfuerzo de desarrollo extra, lo que significa que debemos ser cuidadosos y convertirlos en casos de prueba de regresión únicamente cuando el esfuerzo merezca la pena.

Básicamente debe ser un caso de prueba que pueda cambiar y convenga seguir probándolo, porque si al primer cambio ya queda obsoleto esto es muy costoso y no conviene

3. Implementar la prueba



(Usa los 2 artefactos de la actividad anterior + el modelo de Implementación preparado para probarse)

- Cuando programamos los componentes de prueba explícitamente usamos los procedimientos de prueba como especificaciones iniciales. Obsérvese que dicho esfuerzo de programación podría requerir personal con buenas habilidades como programador.

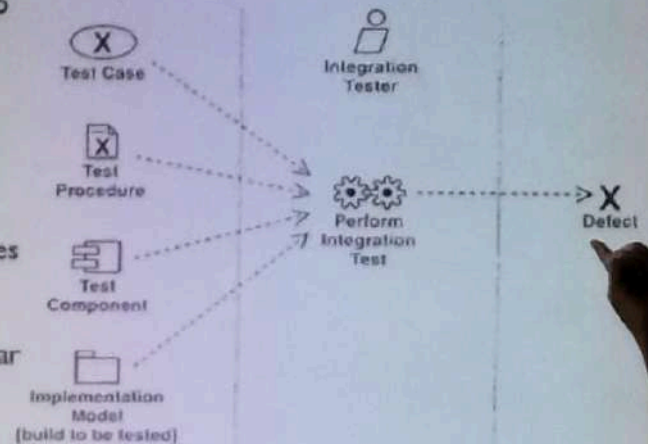
Los componentes de prueba usan a menudo grandes cantidades de datos de entrada para ser probados y producen grandes cantidades de datos de salida como resultado de las pruebas. Es útil poder visualizar estos datos de forma clara e intuitiva de manera que puedan especificarse correctamente y los resultados de las pruebas puedan ser interpretados. Utilizamos para ello hojas de cálculo y bases de datos.

4. Efectuar prueba de integración

PRUEBA – FLUJO DE TRABAJO

4. Efectuar pruebas de integración (Actividad)

1. Realizar las pruebas de integración ejecutando los procedimientos o componentes de prueba correspondientes.
2. Comparar los resultados de las pruebas con los resultados esperados e investigar resultados no esperados.
3. Informar defectos a los ingenieros de componentes responsables de los componentes que registran fallas.
4. Informar los defectos a los diseñadores de pruebas, quienes usarán los defectos para evaluar los resultados de las pruebas.



-con suerte no hay defectos

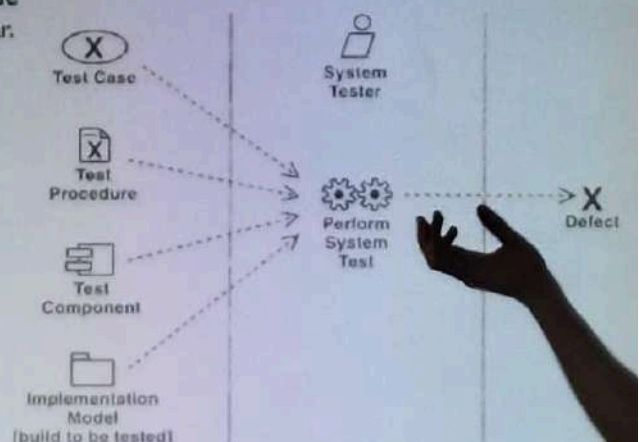
(La entrada es la misma que actividad anterior y se agrega la salida del anterior)

5. Realizar prueba del sistema

PRUEBA – FLUJO DE TRABAJO

5. Realizar prueba del sistema (Actividad)

- Una vez finalizadas las pruebas de integración se realizan las pruebas de sistema de forma similar.



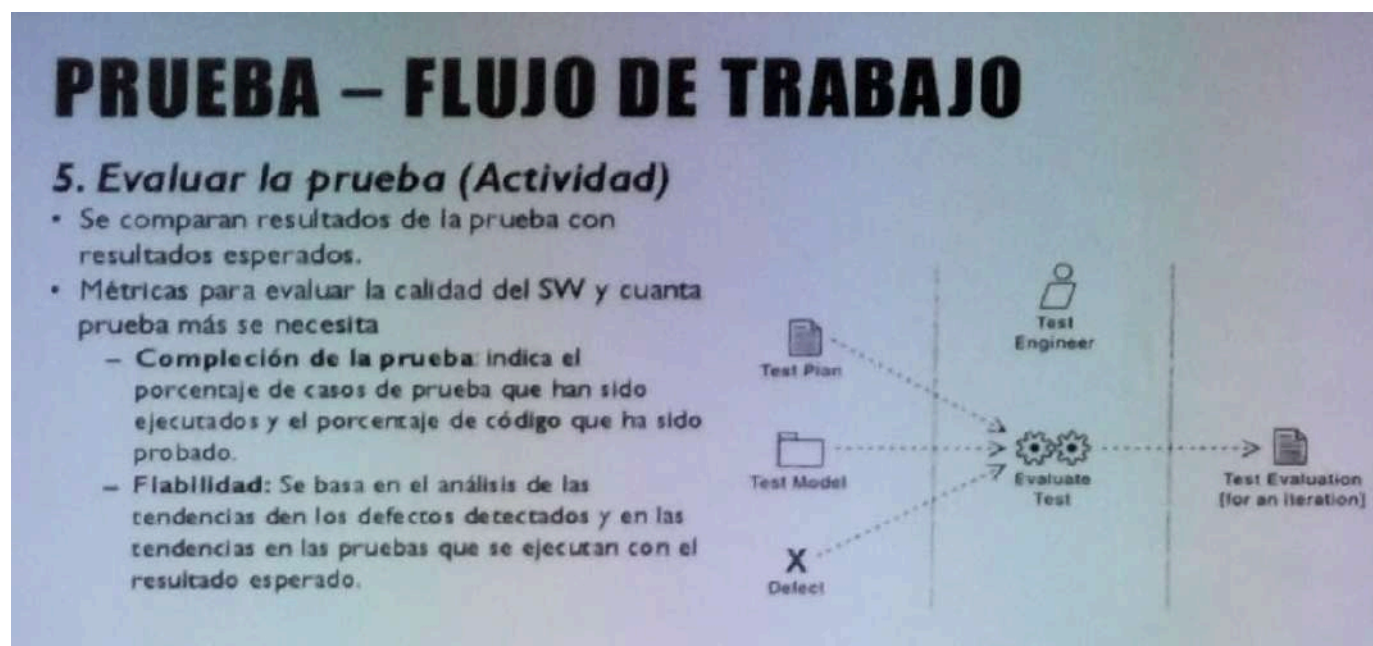
(La entrada y salida es la misma que la actividad anterior)

El propósito de la prueba de sistema es el realizar las pruebas de sistema (véase Sección 11.5.1.2) necesarias en cada iteración y el recopilar los resultados de las pruebas (véase Figura 11.14).

La prueba de sistema puede empezar cuando las pruebas de integración indican que el sistema satisface los objetivos de calidad de integración fijados en el plan de prueba de la iteración actual; por ejemplo, el 95 por ciento de los casos de prueba de integración se ejecutan con el resultado esperado.

La prueba de sistema se realiza de forma análoga a la forma en que se realiza la prueba de integración (véase Sección 11.5.4).

6. Evaluar la prueba



-hay que enfocarnos en la completitud de la prueba

-y por otro lado que tan confiable es la prueba que se realizó

-esto se hace en base al plan que teníamos y los defectos que surgieron

(La entrada es distinta a todos los otros, va a tener todo lo que se estuvo probando y sus defectos, en base a eso hace la evaluación)

El propósito de la evaluación de la prueba es el de evaluar los esfuerzos de prueba en una iteración (véase Figura 11.15).

Para determinar la fiabilidad del sistema, los diseñadores de pruebas crean diagramas de las **tendencias de los defectos**, donde representan la distribución de tipos específicos de defectos (como defectos nuevos o fatales) a lo largo del tiempo. La prueba puede crear también diagramas de tendencias que representan el **porcentaje de pruebas ejecutadas con éxito** (es decir, ejecuciones de pruebas que generan los resultados esperados) a lo largo del tiempo.

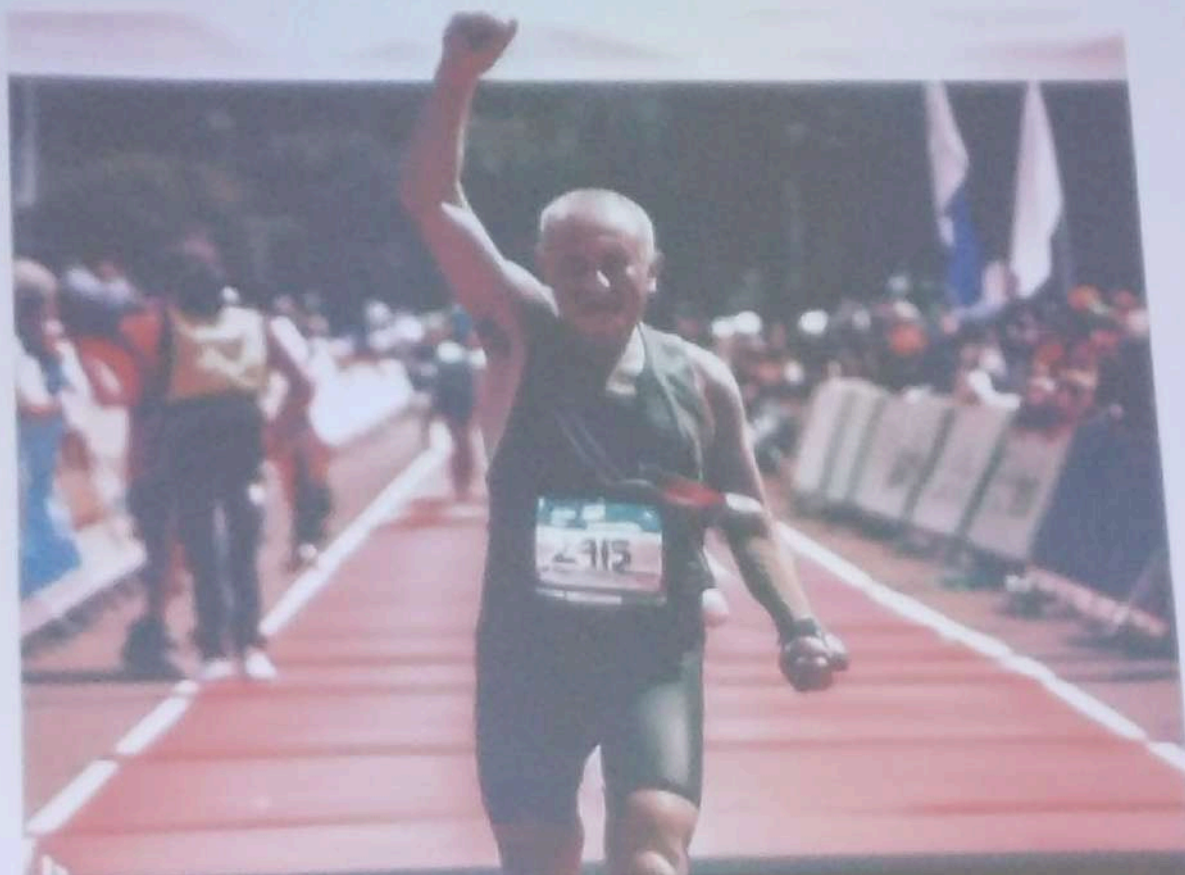
Las tendencias de los defectos siguen a menudo patrones que se repiten en varios proyectos. Por ejemplo, normalmente el número de defectos nuevos generados cuando se prueba un sistema se incrementa bastante rápidamente tan pronto como comienza la prueba, después se mantiene estable durante un tiempo y finalmente empieza a caer lentamente. Por tanto, comparando la tendencia actual con tendencias similares en proyectos anteriores es posible predecir el esfuerzo necesario para alcanzar un nivel de calidad aceptable.

Basándose en el análisis de la tendencia de los defectos los diseñadores de pruebas pueden sugerir otras acciones, como por ejemplo:

- Realizar pruebas adicionales para localizar más defectos, si la fiabilidad medida sugiere que el sistema no está suficientemente maduro.
- Relajar el criterio para las pruebas, si los objetivos de calidad para la iteración actual se pusieron demasiado altos.
- Aislar las partes del sistema que parecen tener una calidad aceptable y entregarlas como resultado de la iteración actual. Las partes que no cumplieron los criterios de calidad han de ser revisados y probadas de nuevo.

Los diseñadores de pruebas documentan la compleción de la prueba, su fiabilidad y sugieren acciones en una descripción de la evaluación de la prueba.

PROCESO UNIFICADO



SE TERMINOO CINE!!!!